Data Structures (in C++)

- Graphs and Graph Algorithms -

부산대학교 정보·의생명공학대학
정보컴퓨터공학부

Graphs

Graphs

- **Graph**
  - A way of representing relationships that exist between pairs of objects
  - A set of objects, called *vertices* (or *nodes*), with a collection of pairwise connections between them, called *edges* (or *arcs*)
  - A graph G is a set V of vertices and a collection E of edges
  - An edge (u, v) is said to be
    - *directed* from u to v if the pair (u, v) is ordered with u preceding v
    - *undirected* if the pair (u, v) is not ordered

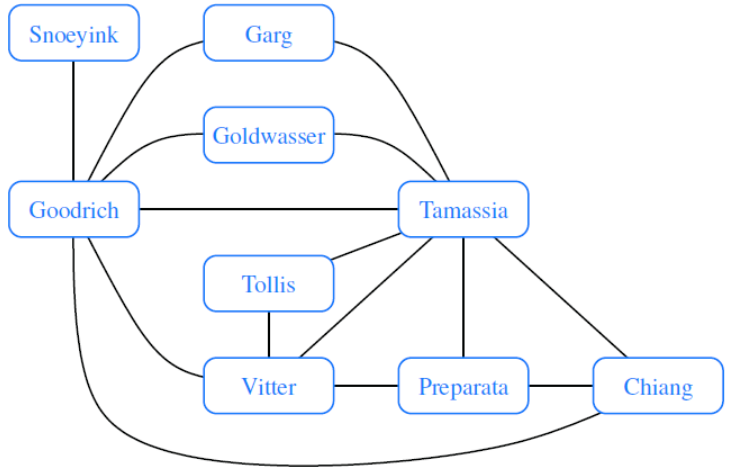


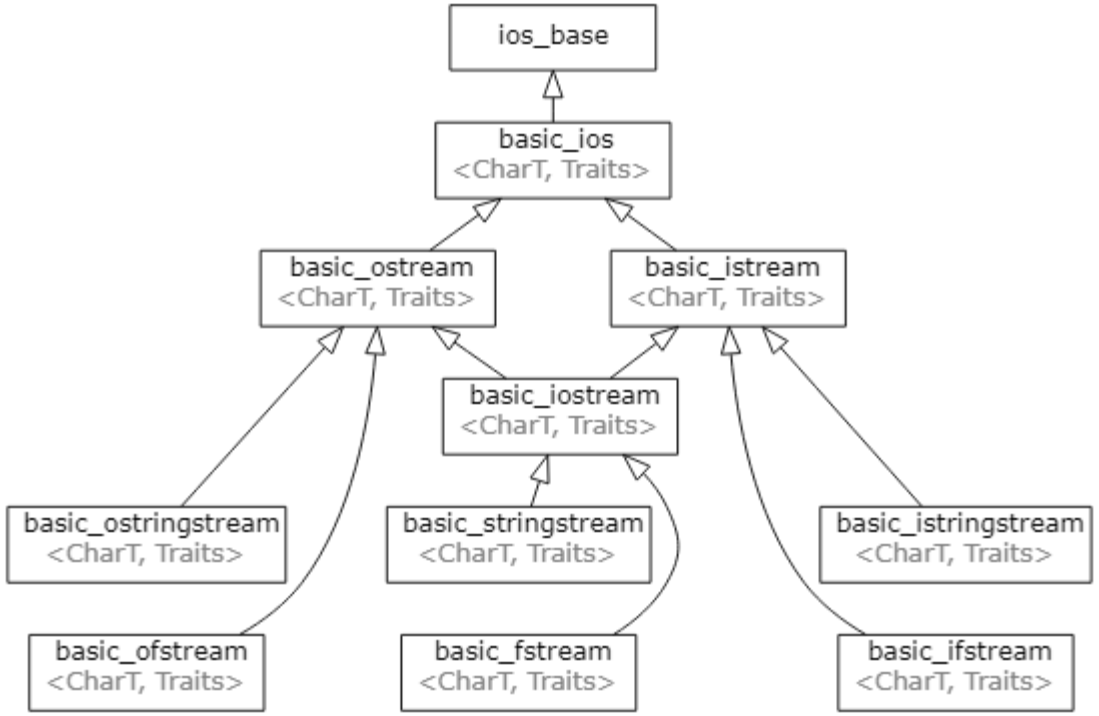
Figure 14.1: Graph of coauthorship among some authors.

  - A graph is an *undirected graph* if all the edges in a graph are undirected
  - A graph is a *directed graph* (or *digraph*) if all the edges are directed
  - A graph that has both directed and undirected edges is called a *mixed graph*

Graphs

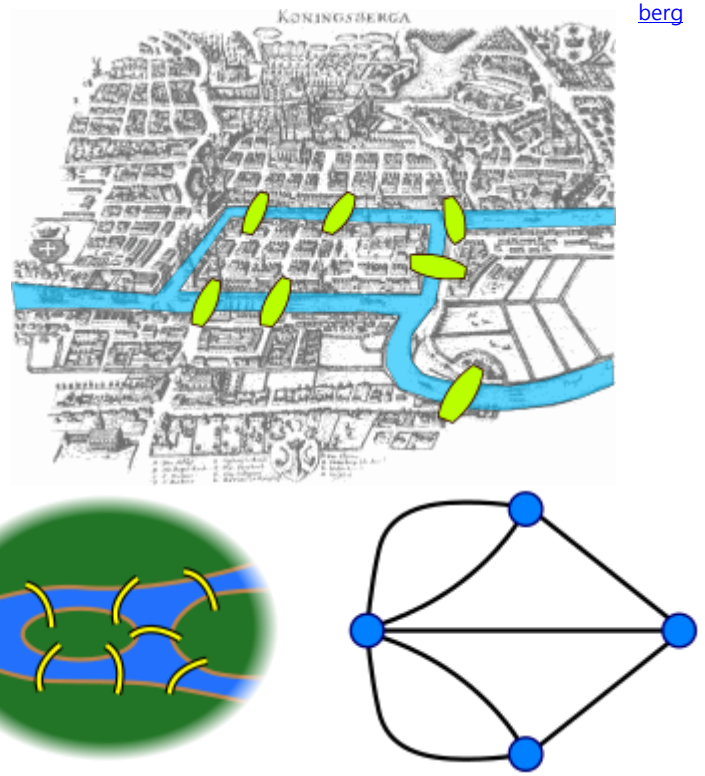
- **Graph Examples**

https://en.cppreference.com/w/cpp/io



C++ Inheritance Diagram

https://en.wikipedia.org/wiki/Seven_Bridges_of_K%C3%B6nigsberg



Seven Bridges of Königsberg

Graphs

- **Graph**
  - The two vertices joined by an edge are called the *end vertices* (or *endpoints*) of the edge
    - The first endpoint is its *origin* and the other is the *destination* of a directed edge
  - Two vertices u and v are *adjacent* if there is an edge between them
  - An edge is *incident* on a vertex if the vertex is one of the edge's endpoints
  - The *incoming/outgoing* edges of a vertex are the directed edges whose *destination/origin* is that vertex
  - The *degree* of a vertex v, denoted $\deg(v)$, is the number of incident edges of v
    - *In-degree* $indeg(v)$: the number of the incoming edges
    - *Out-degree* $outdeg(v)$: the number of the outgoing edges

Graphs

- **Graph Examples**

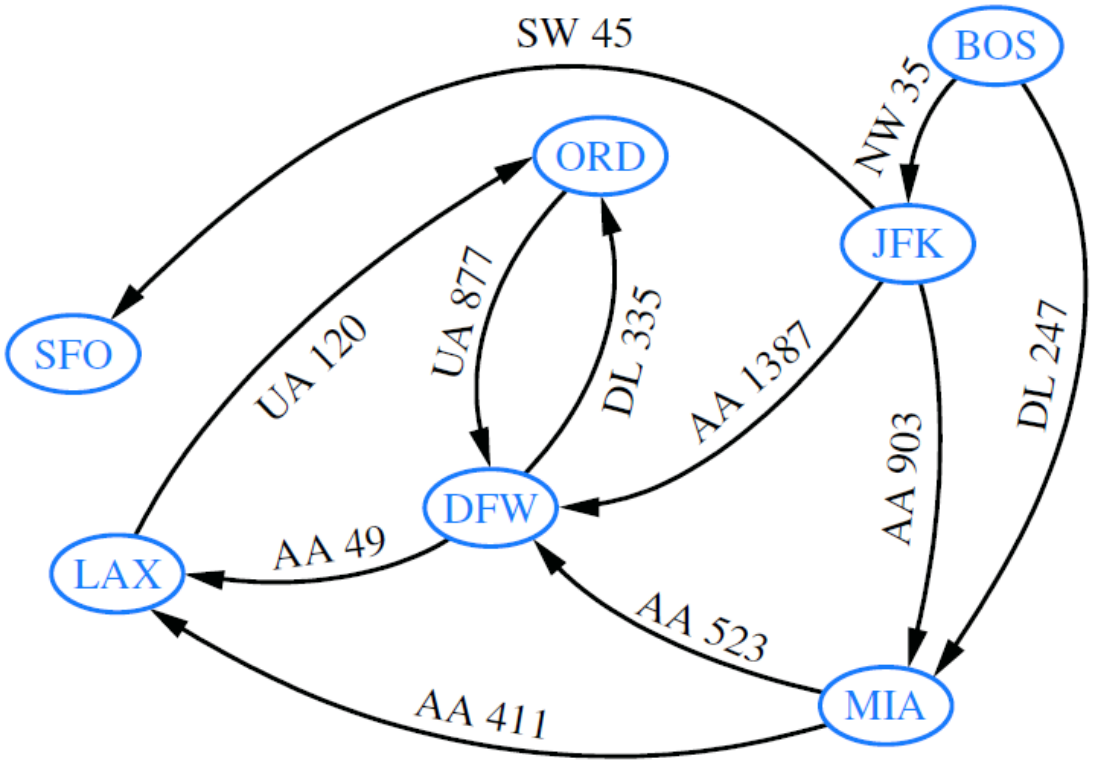
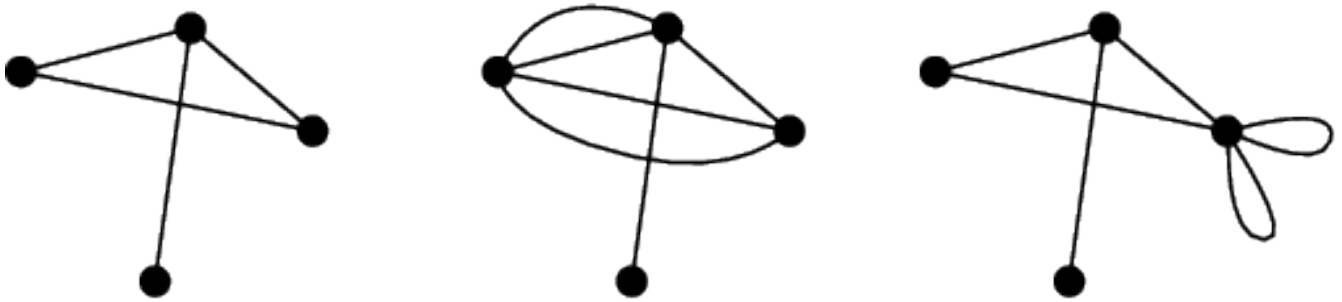


Figure 14.2: Example of a directed graph representing a flight network. The endpoints of edge UA 120 are LAX and ORD; hence, LAX and ORD are adjacent. The in-degree of DFW is 3, and the out-degree of DFW is 2.

Graphs

- **Graph**
  - *Parallel edges* (or *multiple edges*):
    - Two undirected edges with the same end vertices
    - Two directed edges with the same origin and destination
  - A *self-loop* connects a vertex to itself
  - Graphs without parallel edges or self-loops are said to be *simple*

https://mathworld.wolfram.com/SimpleGraph.html



simple graph

nonsimple graph with multiple edges

nonsimple graph with loops

Graphs

- **Graph**
  - A *path* is a sequence of alternating vertices and edges that starts at a vertex and ends at a vertex
    - Each edge is incident to its predecessor and successor vertex
    - A path is *simple* if each vertex in the path is distinct
  - A *cycle* is a path that starts and ends at the same vertex
    - Includes at least one edge
    - A cycle is simple if each vertex in the cycle is distinct except for the first and last one
  - A *directed path/cycle* is a path/cycle such that all edges are directed and traversed along their direction
  - A directed graph is *acyclic* if it has no directed cycles

Graphs

- **Graph**
  - Given vertices u and v of a (directed) graph G, u *reaches* v (or v is *reachable* from u) if G has a (directed) path from u to v
  - A graph is *connected* if there is a path between any two vertices of the graph
  - A directed graph is *strongly connected* if for any two vertices u and v, u reaches v and v reaches u
  - A *subgraph* of a graph G is a graph H whose vertices and edges are subsets of those of G
  - A *spanning subgraph* of G is a subgraph of G that contains all the vertices of the graph G

Graphs

- **Graph Examples**

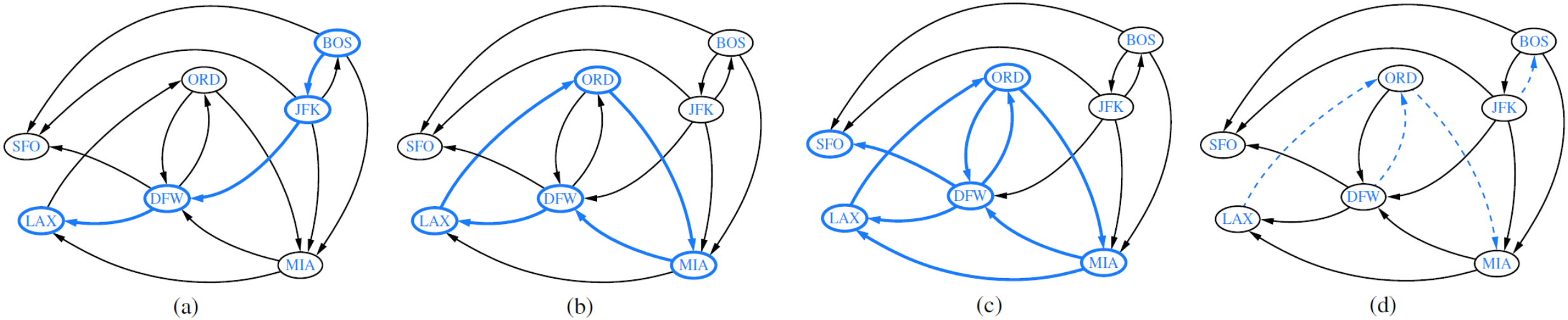


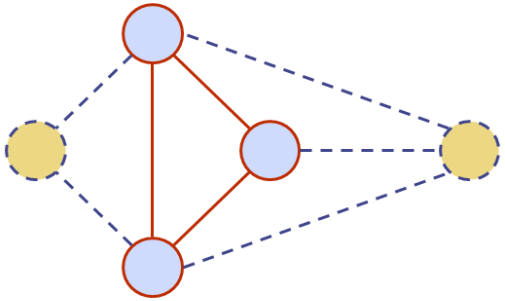
Figure 14.3: Examples of reachability in a directed graph: (a) a directed path from BOS to LAX is highlighted; (b) a directed cycle (ORD, MIA, DFW, LAX, ORD) is highlighted; its vertices induce a strongly connected subgraph; (c) the subgraph of the vertices and edges reachable from ORD is highlighted; (d) the removal of the dashed edges results in a directed acyclic graph.

Graphs

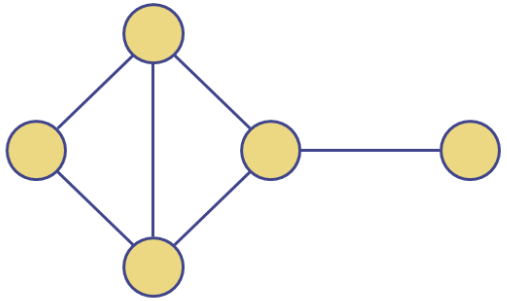
- **Graph**
  - If a graph G is not connected, its maximal connected subgraphs are called the *connected components* of G
  - A *forest* is a graph without cycles
  - A *tree* is a connected graph without cycles (*i.e.*, connected forest)
  - A *spanning tree* of a graph is a spanning subgraph that is a tree

Graphs

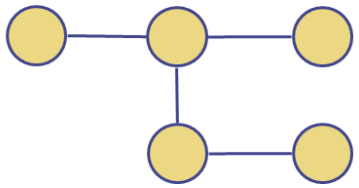
- **Graph**



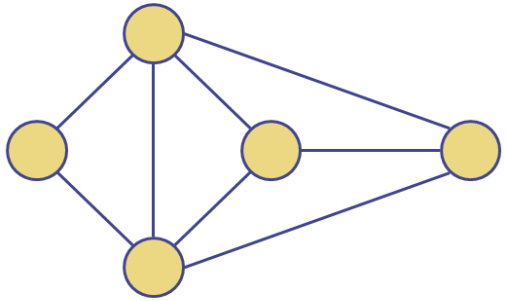
Subgraph



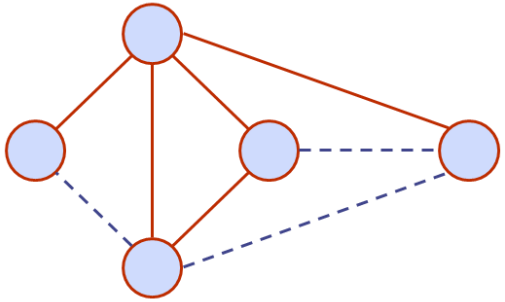
Connected graph



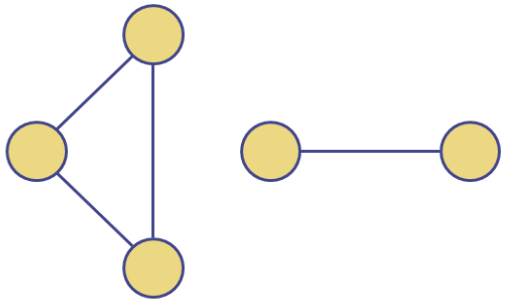
Tree



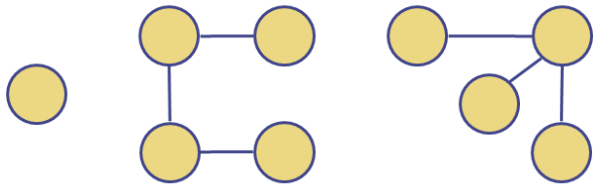
Graph



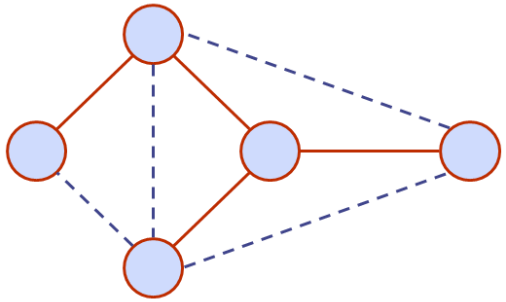
Spanning subgraph



Non connected graph with two connected components



Forest



Spanning tree

- **Graph Properties**

Proposition 14.8: *If G is a graph with m edges and vertex set V, then*

$$\sum_{v \text{ in } V} deg(v) = 2m.$$

Justification: An edge (u, v) is counted twice in the summation above; once by its endpoint u and once by its endpoint v. Thus, the total contribution of the edges to the degrees of the vertices is twice the number of edges. ■

Proposition 14.9: *If G is a directed graph with m edges and vertex set V, then*

$$\sum_{v \text{ in } V} indeg(v) = \sum_{v \text{ in } V} outdeg(v) = m.$$

Justification: In a directed graph, an edge (u, v) contributes one unit to the out-degree of its origin u and one unit to the in-degree of its destination v. Thus, the total contribution of the edges to the out-degrees of the vertices is equal to the number of edges, and similarly for the in-degrees. ■

- **Graph Properties**

Proposition 14.10: *Let G be a simple graph with n vertices and m edges. If G is undirected, then $m \leq n(n-1)/2$, and if G is directed, then $m \leq n(n-1)$.*

Justification: Suppose that G is undirected. Since no two edges can have the same endpoints and there are no self-loops, the maximum degree of a vertex in G is $n-1$ in this case. Thus, by Proposition 14.8, $2m \leq n(n-1)$. Now suppose that G is directed. Since no two edges can have the same origin and destination, and there are no self-loops, the maximum in-degree of a vertex in G is $n-1$ in this case. Thus, by Proposition 14.9, $m \leq n(n-1)$. ■

There are a number of simple properties of trees, forests, and connected graphs.

Proposition 14.11: *Let G be an undirected graph with n vertices and m edges.*

- *If G is connected, then $m \geq n-1$.*
- *If G is a tree, then $m = n-1$.*
- *If G is a forest, then $m \leq n-1$.*

Graph ADT

- **Graph ADT Operations**

Each Vertex object u supports the following operations, which provide access to the vertex's element and information regarding incident edges and adjacent vertices.

operator*(): Return the element associated with u.

incidentEdges(): Return an edge list of the edges incident on u.

isAdjacentTo(v): Test whether vertices u and v are adjacent.

Each Edge object e supports the following operations, which provide access to the edge's end vertices and information regarding the edge's incidence relationships.

operator*(): Return the element associated with e.

endVertices(): Return a vertex list containing e's end vertices.

opposite(v): Return the end vertex of edge e distinct from vertex v; an error occurs if e is not incident on v.

isIncidentOn(v): Test whether e is incident on v.

Graph ADT

- **Graph ADT Operations**

Finally, the full graph ADT consists of the following operations, which provide access to the lists of vertices and edges, and provide functions for modifying the graph.

vertices():	Return a vertex list of all the vertices of the graph.
edges():	Return an edge list of all the edges of the graph.
insertVertex(x):	Insert and return a new vertex storing element x.
insertEdge(v, w, x):	Insert and return a new undirected edge with end vertices v and w and storing element x.
eraseVertex(v):	Remove vertex v and all its incident edges.
eraseEdge(e):	Remove edge e.

Data Structures for Graphs

- **The Edge List Structure**
  - The simplest representation of a graph G
  - All vertex objects are stored in an unordered list V
  - All edge objects are stored in an unordered list E

Vertex Objects

The vertex object for a vertex v storing element x has member variables for:

- A copy of x
- The position (or entry) of the vertex-object in collection V

The distinguishing feature of the edge list structure is not how it represents vertices, but the way in which it represents edges. In this structure, an edge e of G storing an element x is explicitly represented by an edge object. The edge objects are stored in a collection E, which would typically be a vector or node list.

Edge Objects

The edge object for an edge e storing element x has member variables for:

- A copy of x
- The vertex positions associated with the endpoint vertices of e
- The position (or entry) of the edge-object in collection E

Data Structures for Graphs

- **The Edge List Structure**

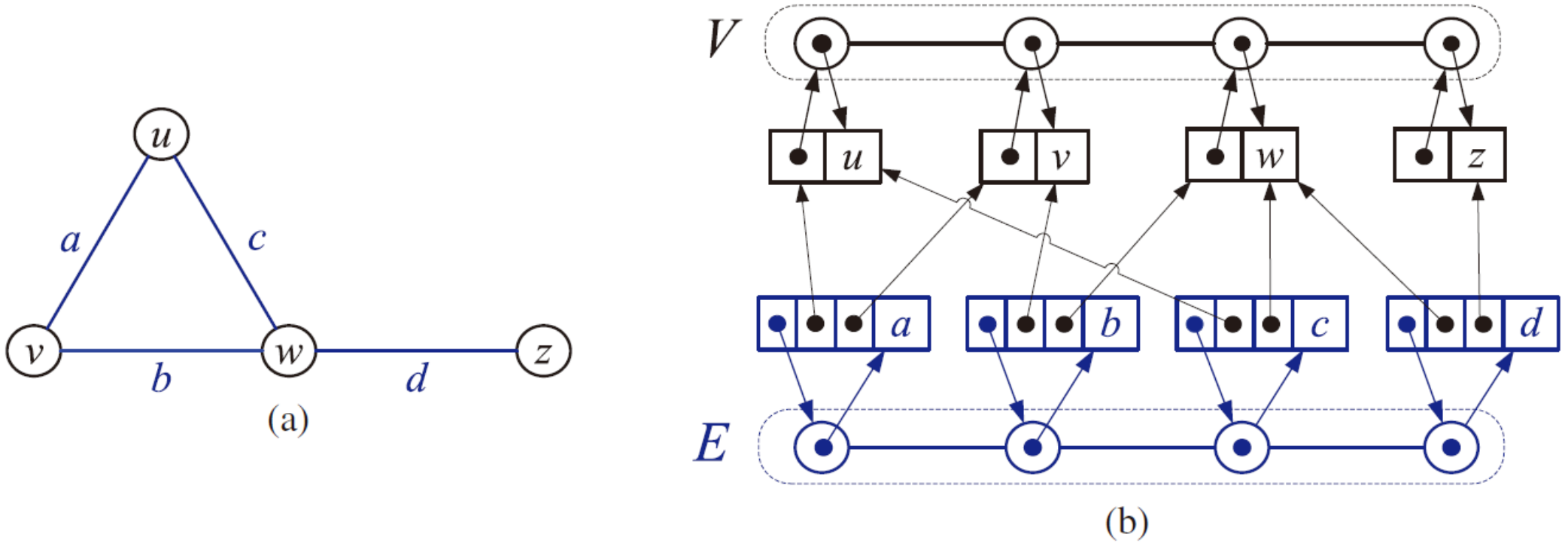


Figure 13.3: (a) A graph G. (b) Schematic representation of the edge list structure for G. We visualize the elements stored in the vertex and edge objects with the element names, instead of with actual references to the element objects.

Data Structures for Graphs

▪ Performance of the Edge List Structure

- Methods vertices() and edges() are implemented by using the iterators for V and E, respectively, to enumerate the elements of the lists.
- Methods incidentEdges and isAdjacentTo all take $O(m)$ time, since to determine which edges are incident upon a vertex v we must inspect all edges.
- Since the collections V and E are lists implemented with a doubly linked list, we can insert vertices, and insert and remove edges, in $O(1)$ time.
- The update function eraseVertex(v) takes $O(m)$ time, since it requires that we inspect all the edges to find and remove those incident upon v.

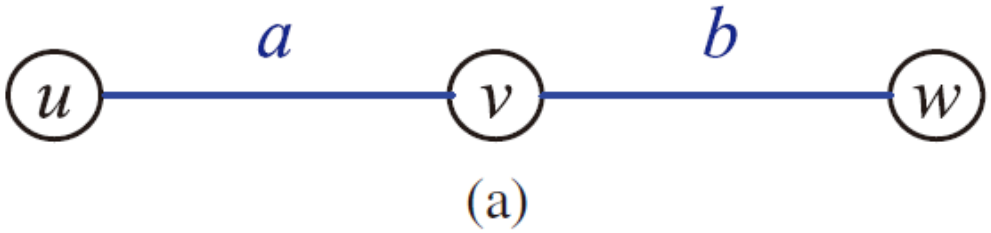
Operation	*Time*
vertices	$O(n)$
edges	$O(m)$
endVertices, opposite	$O(1)$
incidentEdges, isAdjacentTo	$O(m)$
isIncidentOn	$O(1)$
insertVertex, insertEdge, eraseEdge,	$O(1)$
eraseVertex	$O(m)$

Table 13.1: Running times of the functions of a graph implemented with the edge list structure. The space used is $O(n+m)$, where n is the number of vertices and m is the number of edges.

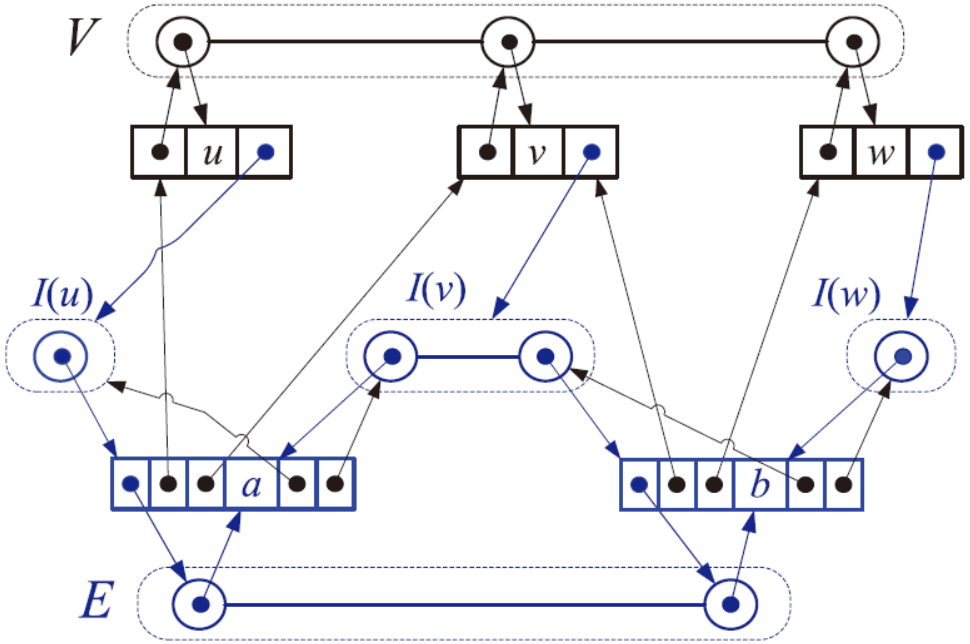
Data Structures for Graphs

- **The Adjacency List Structure**
  - Adds extra information to the edge list that supports direct access to the incident edges of each vertex

- A vertex object v holds a reference to a collection $I(v)$, called the ***incidence collection*** of v, whose elements store references to the edges incident on v.
- The edge object for an edge e with end vertices v and w holds references to the positions (or entries) associated with edge e in the incidence collections $I(v)$ and $I(w)$.



(a)



(b)

Figure 13.4: (a) A graph G. (b) Schematic representation of the adjacency list structure of G. As in Figure 13.3, we visualize the elements of collections with names.

Data Structures for Graphs

▪ Performance of the Adjacency List Structure

- Methods vertices() and edges() are implemented by using the iterators for V and E, respectively, to enumerate the elements of the lists.
- Method v.incidentEdges() takes time proportional to the number of incident vertices of v, that is, $O(\deg(v))$ time.
- Method v.isAdjacentTo(w) can be performed by inspecting either the incidence collection of v or that of w. By choosing the smaller of the two, we get $O(\min(\deg(v), \deg(w)))$ running time.
- Method eraseVertex(v) takes $O(\deg(v))$ time.

Operation	*Time*
vertices	$O(n)$
edges	$O(m)$
endVertices, opposite	$O(1)$
v.incidentEdges()	$O(\deg(v))$
v.isAdjacentTo(w)	$O(\min(\deg(v), \deg(w))$
isIncidentOn	$O(1)$
insertVertex, insertEdge, eraseEdge,	$O(1)$
eraseVertex(v)	$O(\deg(v))$

Table 13.2: Running times of the functions of a graph implemented with the adjacency list structure. The space used is $O(n+m)$, where n is the number of vertices and m is the number of edges.

Data Structures for Graphs

- **The Adjacency Matrix Structure**
  - Augments the edge list with a matrix A
  - The vertices are interpreted as the integers in the set $\{0, 1, \ldots, n-1\}$
  - The edges are pairs of such integers
  - Stores references to edges in the cells of a two-dimensional $n \times n$ array A

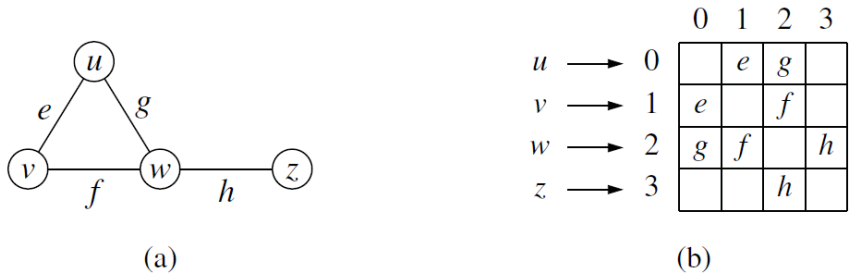


Figure 14.7: (a) An undirected graph G; (b) a schematic representation of the auxiliary adjacency matrix structure for G, in which n vertices are mapped to indices 0 to $n-1$. Although not diagrammed as such, we presume that there is a unique Edge instance for each edge, and that it maintains references to its endpoint vertices. We also assume that there is a secondary edge list (not pictured), to allow the edges( ) method to run in $O(m)$ time, for a graph with m edges.

Data Structures for Graphs

▪ The Adjacency Matrix Structure

- A vertex object v stores a distinct integer i in the range $0, 1, \ldots, n-1$, called the ***index*** of v.
- We keep a two-dimensional $n \times n$ array A such that the cell $A[i, j]$ holds a reference to the edge (v, w), if it exists, where v is the vertex with index i and w is the vertex with index j. If there is no such edge, then $A[i, j] = null$.

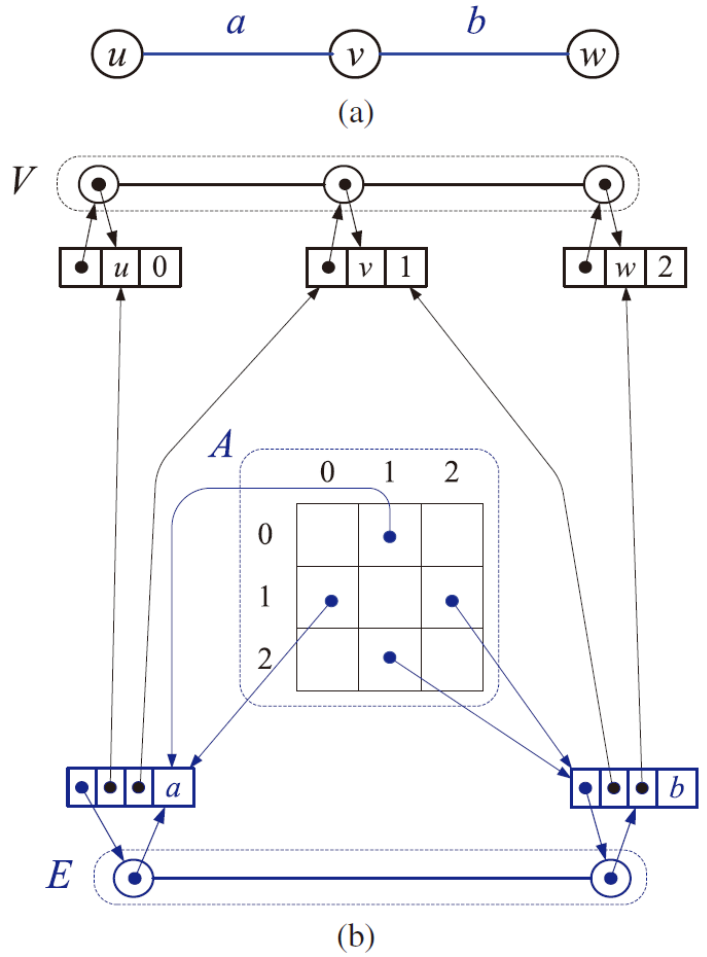


Figure 13.5: (a) A graph G without parallel edges. (b) Schematic representation of the simplified adjacency matrix structure for G.

Data Structures for Graphs

- **Performance of the Adjacency Matrix Structure**

Operation	*Time*
vertices	$O(n)$
edges	$O(m)$
endVertices, opposite	$O(1)$
isAdjacentTo, isIncidentOn	$O(1)$
incidentEdges	$O(n)$
insertEdge, eraseEdge,	$O(1)$
insertVertex, eraseVertex	$O(n^2)$

Generation of a new matrix

Table 13.3: Running times for a graph implemented with an adjacency matrix.

Graph Traversals

- **Graph Traversal**
  - A systematic procedure for exploring a graph by examining all of its vertices and edges

- Computing a path from vertex u to vertex v, or reporting that no such path exists.
- Given a start vertex s of G, computing, for every vertex v of G, a path with the minimum number of edges between s and v, or reporting that no such path exists.
- Testing whether G is connected.
- Computing a spanning tree of G, if G is connected.
- Computing the connected components of G.
- Identifying a cycle in G, or reporting that G has no cycles.

- Computing a directed path from vertex u to vertex v, or reporting that no such path exists.
- Finding all the vertices of $\vec{G}$ that are reachable from a given vertex s.
- Determine whether $\vec{G}$ is acyclic.
- Determine whether $\vec{G}$ is strongly connected.

Depth-First Search

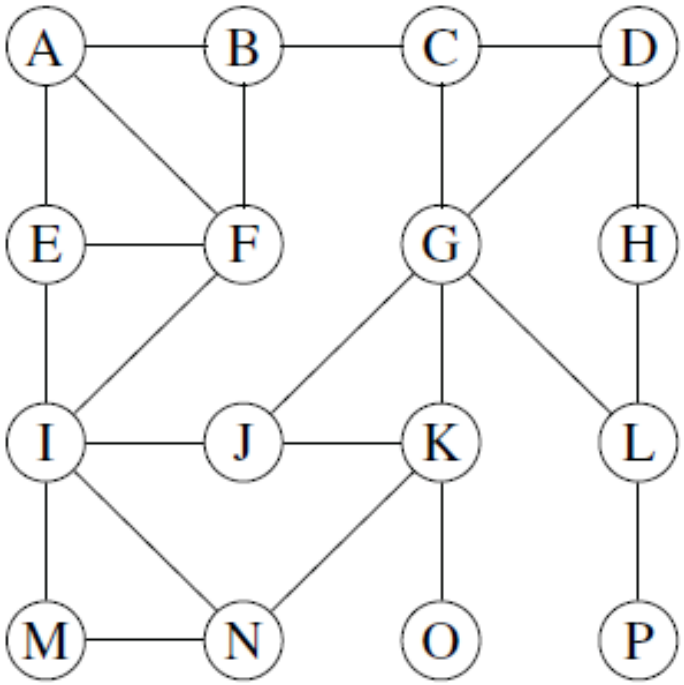
- **Depth-First Search (DFS)**
  - Explores each branch as far as possible
  - If a *dead-end* is encountered, trace back to continue to the next branch (*i.e.*, *Backtracking*)
  - *Discovery edges* (or *tree edges*): The edges used to discover new vertices
  - *Back edges*: The edges lead to already visited vertices

Algorithm DFS(G, u):
Input: A graph G and a vertex u of G
Output: A collection of vertices reachable from u, with their discovery edges
Mark vertex u as visited.
for each of u's outgoing edges, $e = (u, v)$ **do**
  if vertex v has not been visited **then**
    Record edge e as the discovery edge for vertex v.
    Recursively call DFS(G, v).

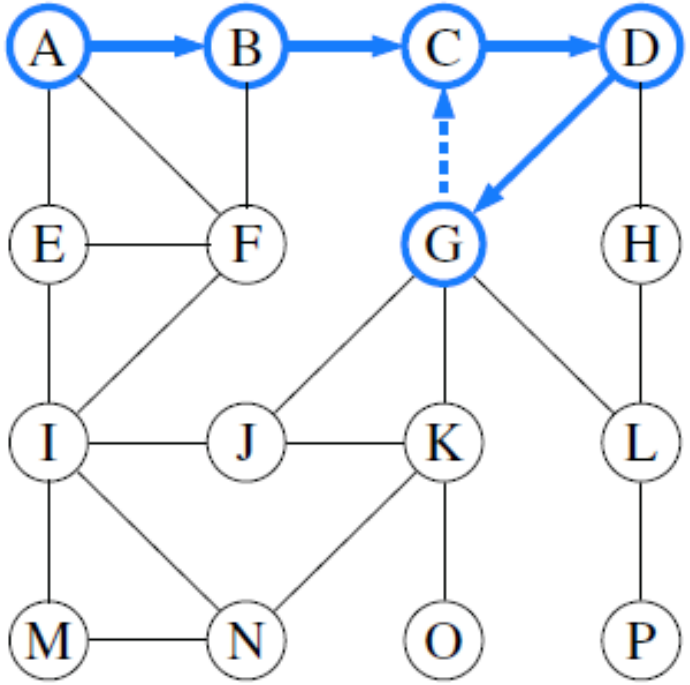
Code Fragment 14.4: The DFS algorithm.

Depth-First Search

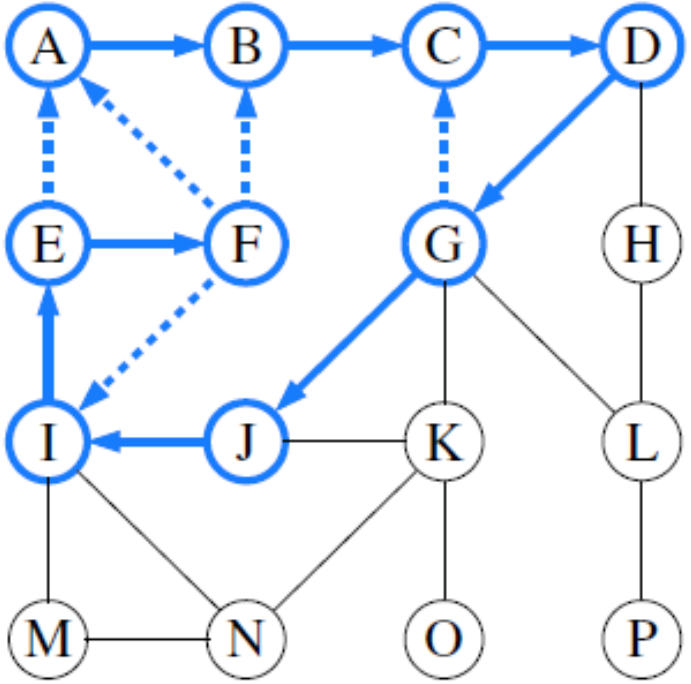
- **DFS Example**



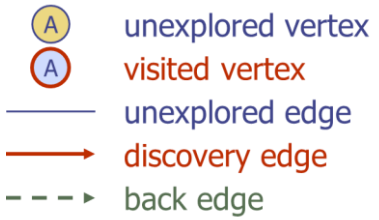
(a)



(b)

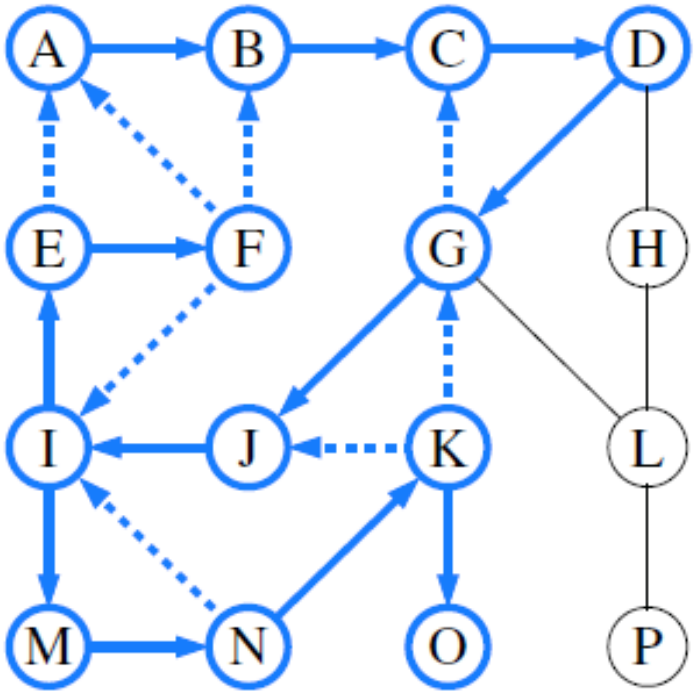


(c)

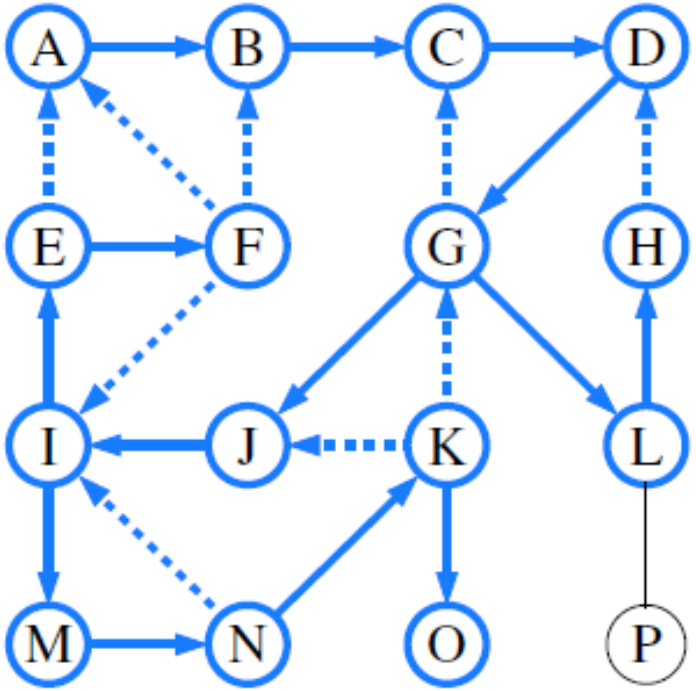


Depth-First Search

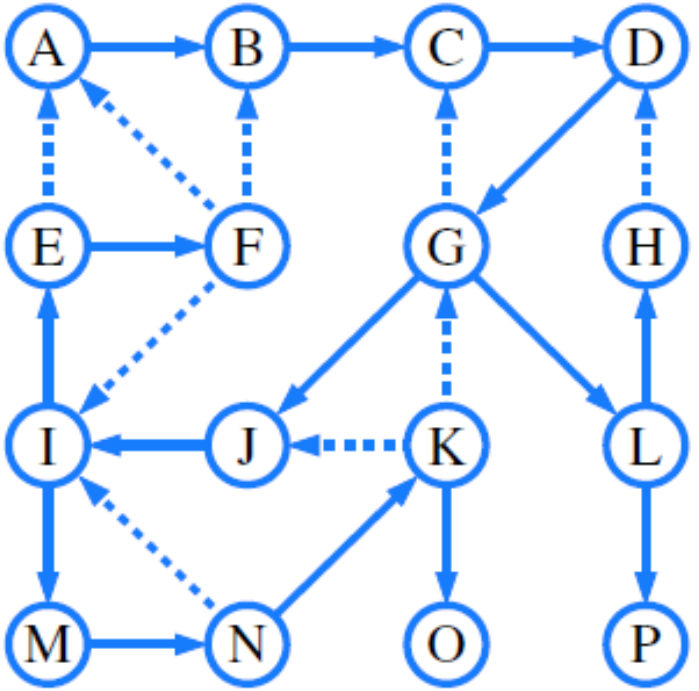
- **DFS Example**



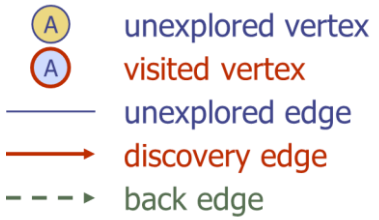
(d)



(e)



(f)



Depth-First Search

- **DFS Property**

Proposition 14.12: *Let G be an undirected graph on which a DFS traversal starting at a vertex s has been performed. Then the traversal visits all vertices in the connected component of s, and the discovery edges form a spanning tree of the connected component of s.*

Justification: Suppose there is at least one vertex w in s's connected component not visited, and let v be the first unvisited vertex on some path from s to w (we may have $v = w$). Since v is the first unvisited vertex on this path, it has a neighbor u that was visited. But when we visited u, we must have considered the edge (u, v); hence, it cannot be correct that v is unvisited. Therefore, there are no unvisited vertices in s's connected component.

Since we only follow a discovery edge when we go to an unvisited vertex, we will never form a cycle with such edges. Therefore, the discovery edges form a connected subgraph without cycles, hence a tree. Moreover, this is a spanning tree because, as we have just seen, the depth-first search visits each vertex in the connected component of s. ■

Depth-First Search

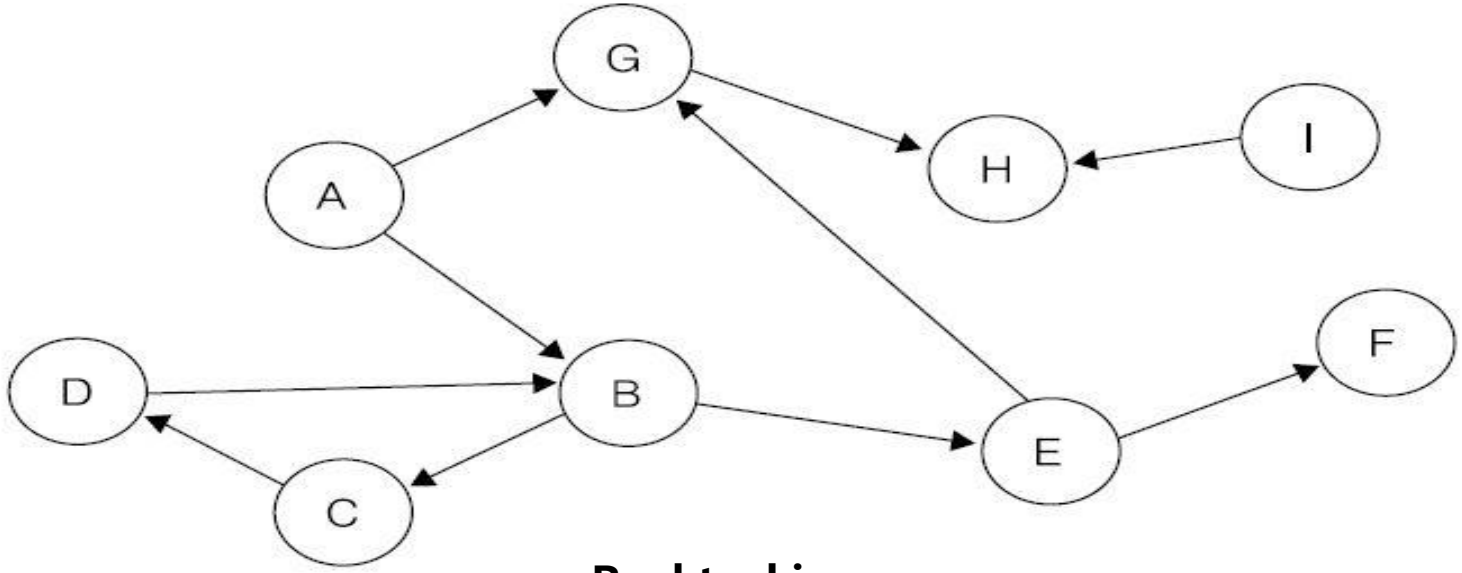
- **DFS Property**

In terms of its running time, depth-first search is an efficient method for traversing a graph. Note that DFS is called exactly once on each vertex, and that every edge is examined exactly twice, once from each of its end vertices. Thus, if n_s vertices and m_s edges are in the connected component of vertex s, a DFS starting at s runs in $O(n_s + m_s)$ time, provided the following conditions are satisfied:

- The graph is represented by a data structure such that creating and iterating through the list generated by v.incidentEdges() takes $O(\text{degree}(v))$ time, and e.opposite(v) takes $O(1)$ time. The adjacency list structure is one such structure, but the adjacency matrix structure is not.
- We have a way to "mark" a vertex or edge as explored, and to test if a vertex or edge has been explored in $O(1)$ time. We discuss ways of implementing DFS to achieve this goal in the next section.

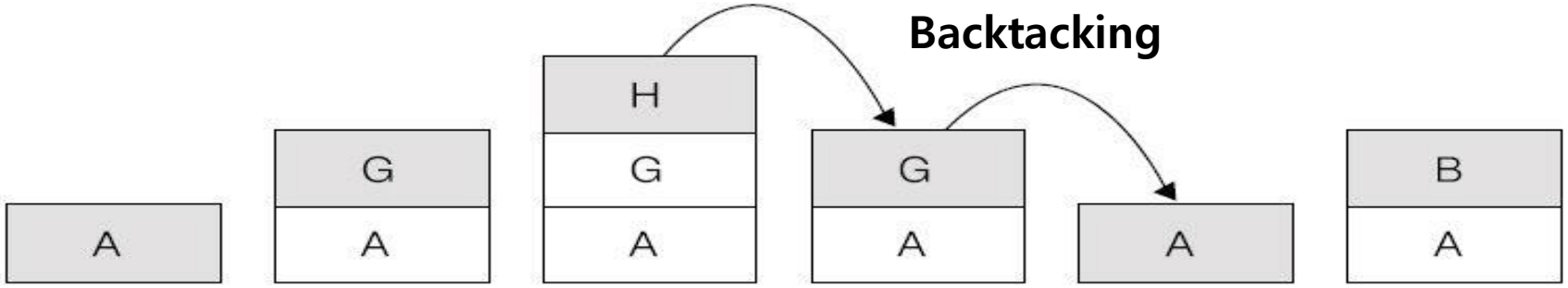
Depth-First Search

- **DFS using a Stack**
  - Search for a path from A to F



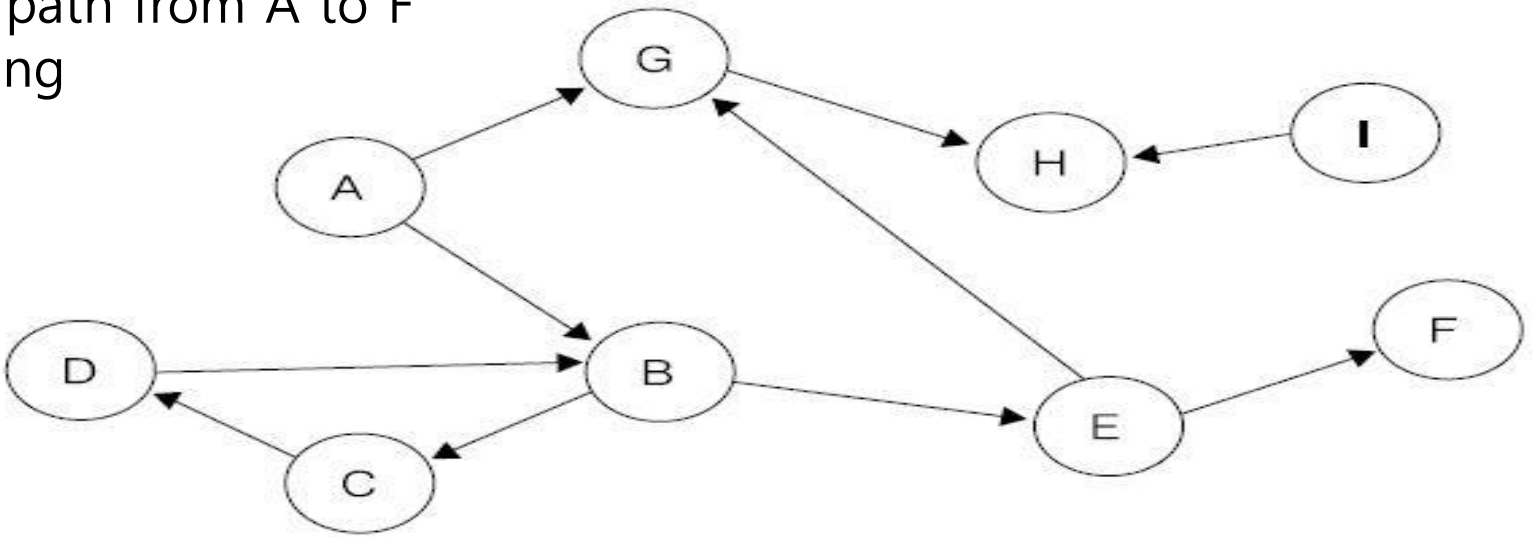
Backtacking

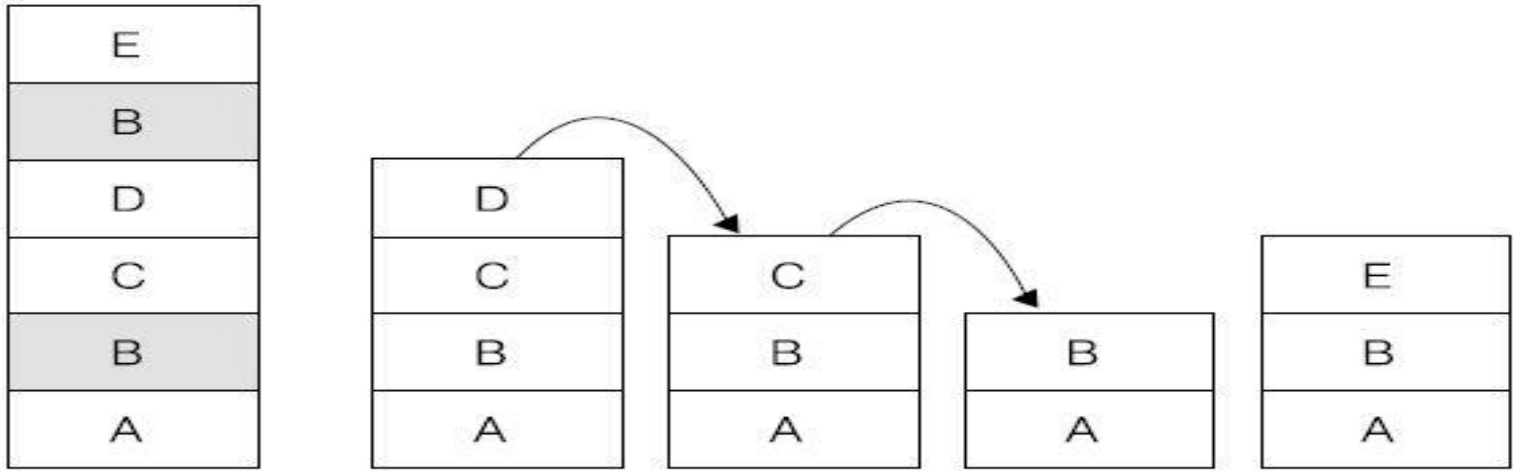
Backtacking



Depth-First Search

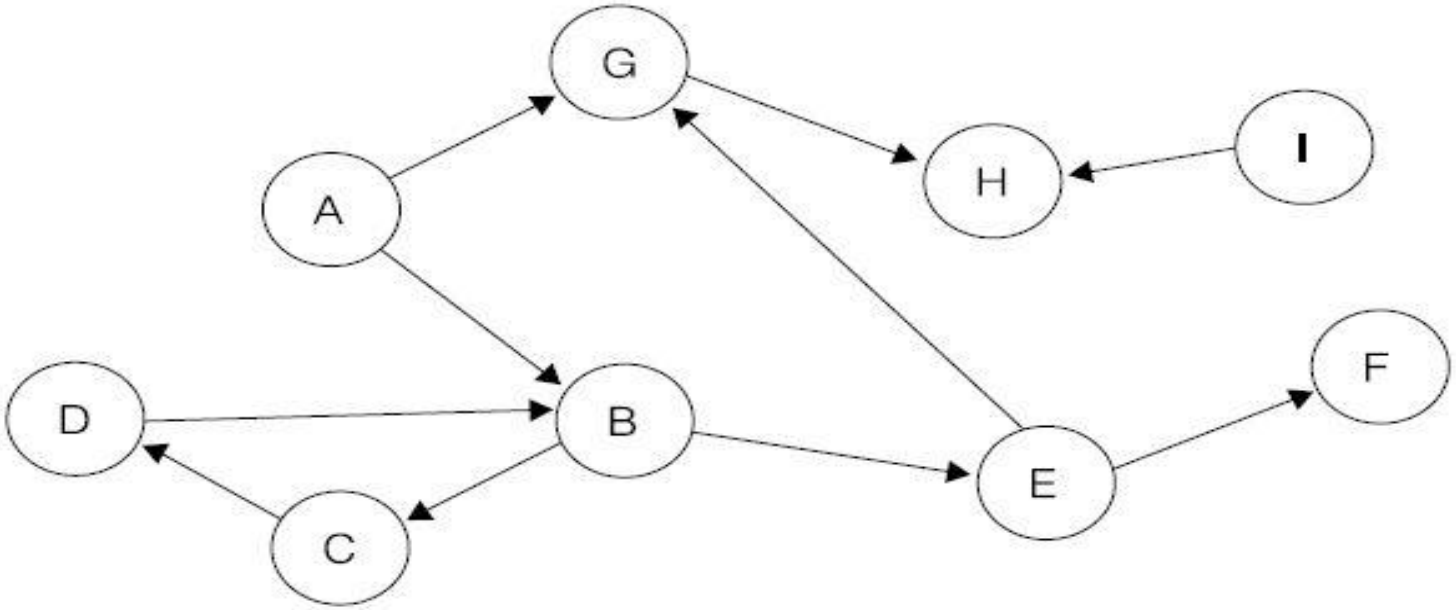
- **DFS using a Stack**
  - Search for a path from A to F
  - Avoid revisiting

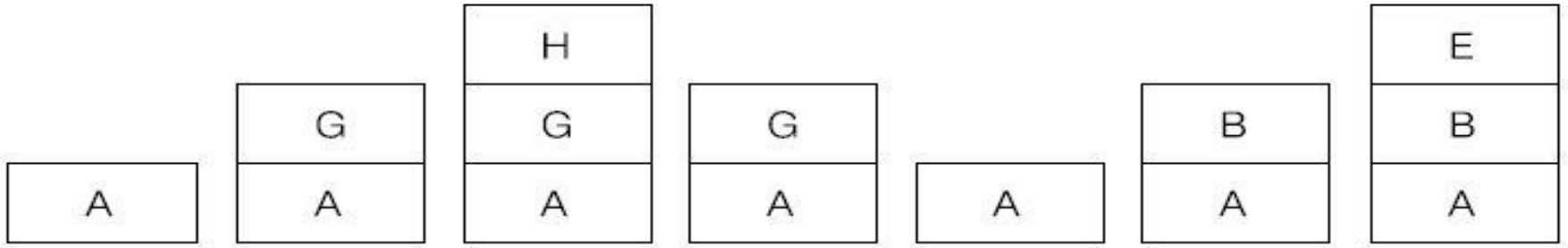




Depth-First Search

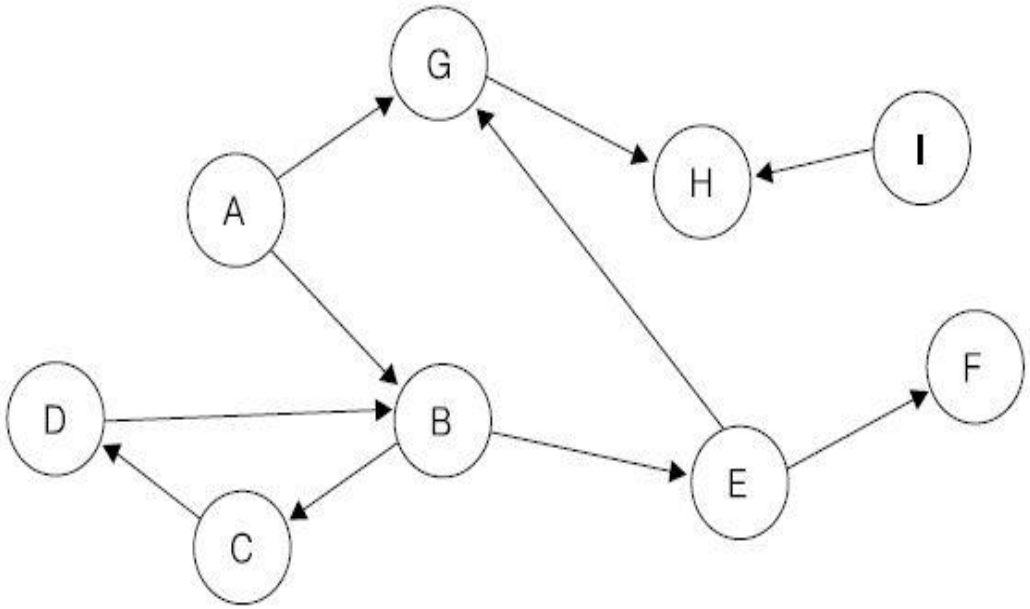
- **DFS using a Stack**
  - Search for a path from A to F

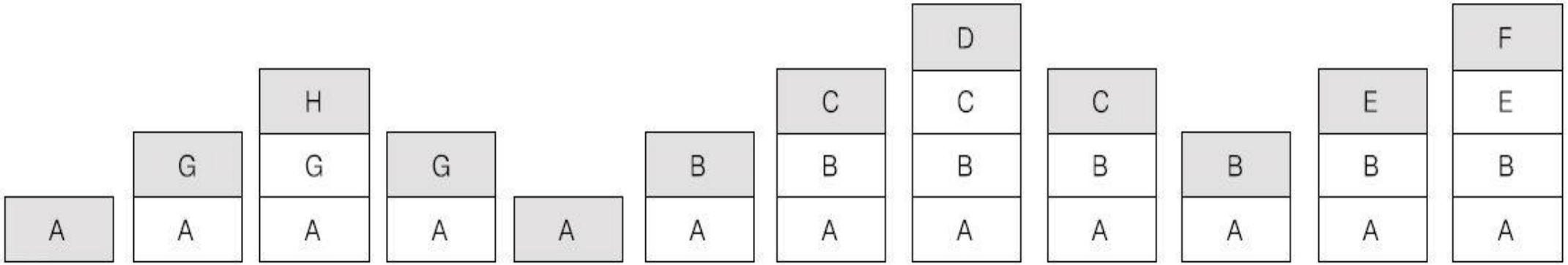




Depth-First Search

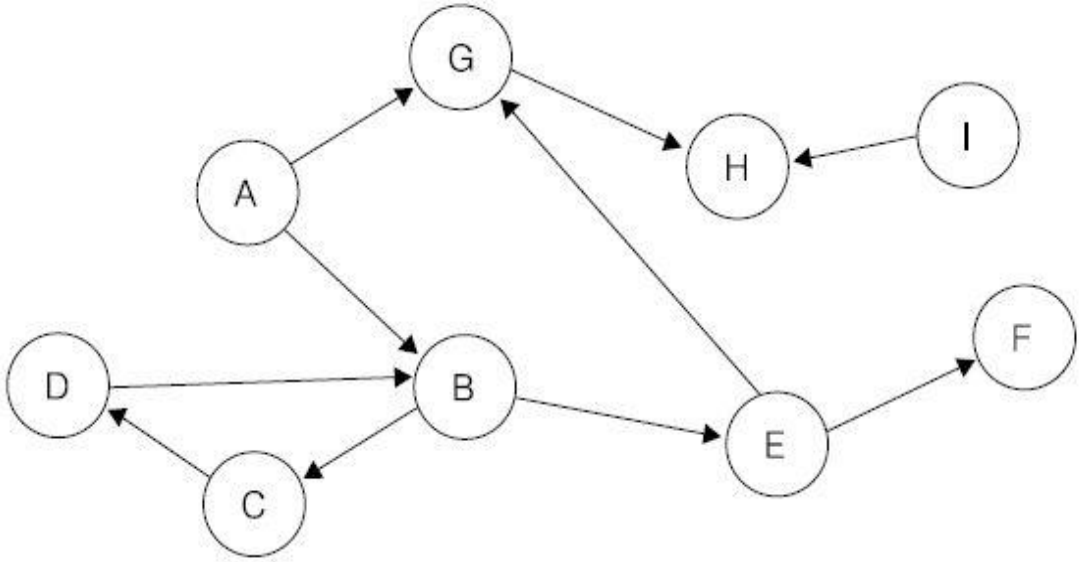
- **DFS using a Stack**
  - Search for a path from A to F
  - Case 1

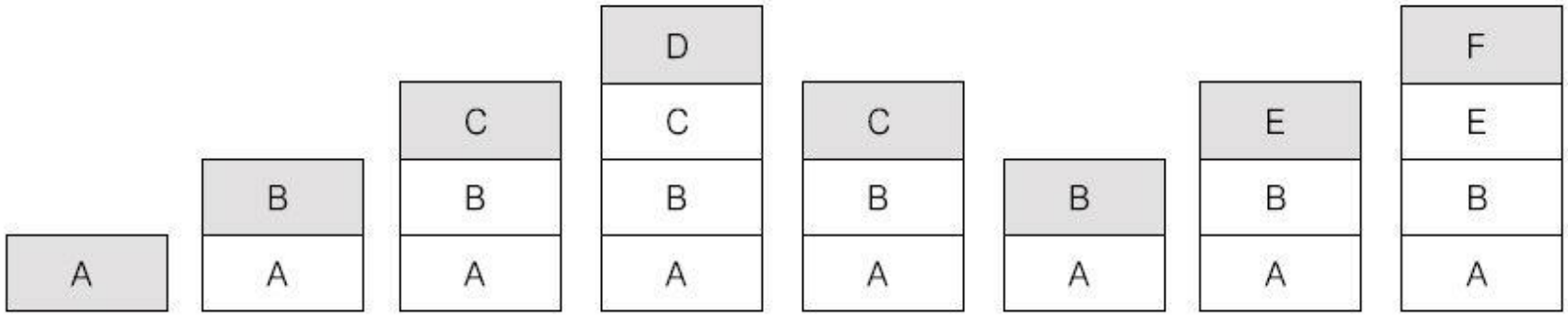




Depth-First Search

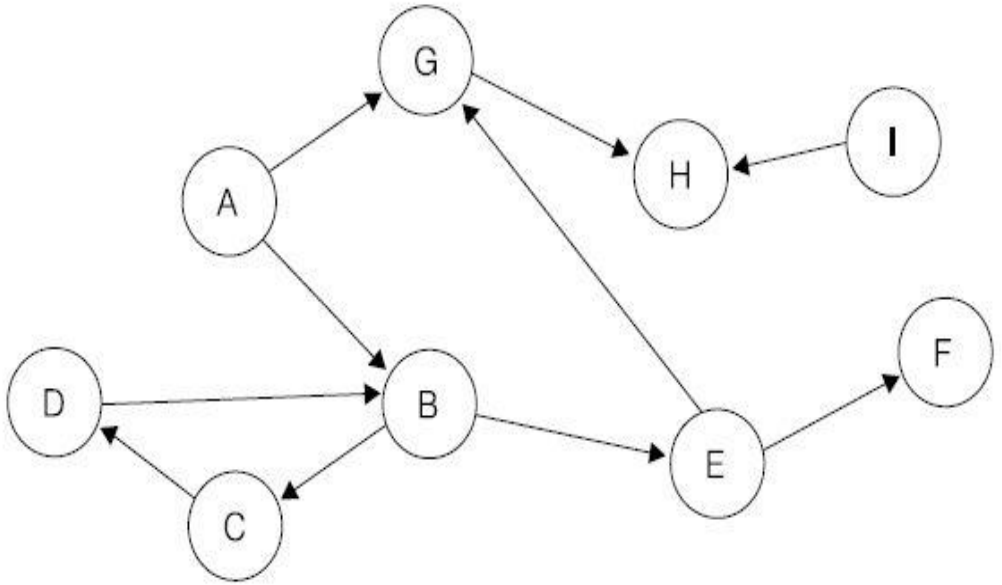
- **DFS using a Stack**
  - Search for a path from A to F
  - Case 2

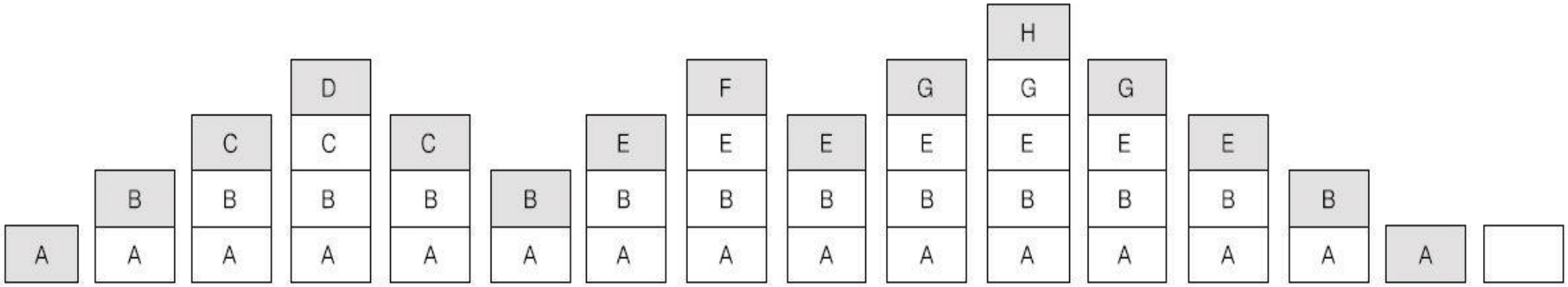




Depth-First Search

- **DFS using a Stack**
  - Search for a path from A to K
  - No such path exists





DFS Implementation

- **The Decorator Pattern**
  - Adds additional *functionalities* or *attributes* (*i.e.*, *decorations*) to existing objects
  - Each decoration is identified by a key identifying this decoration and a value associated with the key
  - An object is decorable if the following functions are supported:

$\text{set}(a, x)$: Set the value of attribute a to x.

$\text{get}(a)$: Return the value of attribute a.

DFS Implementation

- **The Decorator Pattern**

```
Object* yes = new Object;          // decorator values
Object* no = new Object;
Decorator v;                       // a decorable object
// ...
v.set("visited", yes);             // set "visited" attribute
// ...
if (v.get("visited") == yes) cout << "v was visited";
else cout << "v was not visited";
```

```
class Decorator {
private:                                          // member data
  std::map<string,Object*> map;                   // the map
public:
  Object* get(const string& a)                    // get value of attribute
    { return map[a]; }
  void set(const string& a, Object* d)            // set value
    { map[a] = d; }
};
```

Code Fragment 13.2: A C++ implementation of class Decorator.

DFS Implementation

- **DFS Traversal using Decorators**

Algorithm DFS(G, v):

Input: A graph G with decorable vertices and edges, a vertex v of G, such that all vertices and edges have been decorated with the status value of *unvisited*

Output: A decoration of the vertices of the connected component of v with the value *visited* and of the edges in the connected component of v with values *discovery* and *back*, according to a depth-first search traversal of G

```
v.set("status", visited)
for all edges e in v.incidentEdges() do
  if e.get("status") = unvisited then
    w ← e.opposite(v)
    if w.get("status") = unvisited then
      e.set("status", discovered)
      DFS(G, w)
    else
      e.set("status", back)
```

Code Fragment 13.3: DFS on a graph with decorable edges and vertices.

DFS Implementation

- **C++ Implementation**
  - A generic DFS class contains the following virtual functions:

- startVisit(v): called at the start of the visit of v
- traverseDiscovery(e, v): called when a discovery edge e out of v is traversed
- traverseBack(e, v): called when a back edge e out of v is traversed
- isDone(): called to determine whether to end the traversal early
- finishVisit(v): called when we are finished exploring from v

```
template <typename G>
class DFS {                                          // generic DFS
protected:                                           // local types
  typedef typename G::Vertex Vertex;                 // vertex position
  typedef typename G::Edge Edge;                     // edge position
  // ...insert other typename shortcuts here
protected:                                           // member data
  const G& graph;                                    // the graph
  Vertex start;                                      // start vertex
  Object *yes, *no;                                  // decorator values
protected:                                           // member functions
  DFS(const G& g);                                   // constructor
  void initialize();                                 // initialize a new DFS
  void dfsTraversal(const Vertex& v);                // recursive DFS utility
                                                     // overridden functions
  virtual void startVisit(const Vertex& v) {}        // arrived at v
                                                     // discovery edge e
  virtual void traverseDiscovery(const Edge& e, const Vertex& from) {}
                                                     // back edge e
  virtual void traverseBack(const Edge& e, const Vertex& from) {}
  virtual void finishVisit(const Vertex& v) {}       // finished with v
  virtual bool isDone() const { return false; }      // finished?
  // ...insert marking utilities here
};
```

Code Fragment 13.4: A generic implementation of depth-first search.

DFS Implementation

- **C++ Implementation**

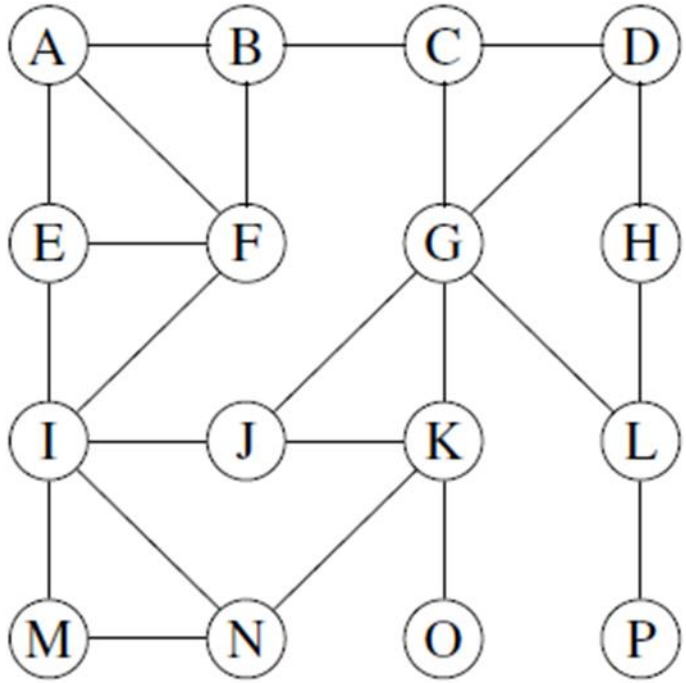
```
protected:                                          // marking utilities
  void visit(const Vertex& v)          { v.set("visited", yes); }
  void visit(const Edge& e)            { e.set("visited", yes); }
  void unvisit(const Vertex& v)        { v.set("visited", no); }
  void unvisit(const Edge& e)          { e.set("visited", no); }
  bool isVisited(const Vertex& v)      { return v.get("visited") == yes; }
  bool isVisited(const Edge& e)        { return e.get("visited") == yes; }
```

Code Fragment 13.5: Vertex and edge marking utilities, which are part of DFS.

```
/* DFS⟨G⟩ :: */                                     // constructor
  DFS(const G& g)
    : graph(g), yes(new Object), no(new Object) {}

/* DFS⟨G⟩ :: */                                     // initialize a new DFS
  void initialize() {
    VertexList verts = graph.vertices();
    for (VertexItor pv = verts.begin(); pv != verts.end(); ++pv)
      unvisit(*pv);                                 // mark vertices unvisited
    EdgeList edges = graph.edges();
    for (EdgeItor pe = edges.begin(); pe != edges.end(); ++pe)
      unvisit(*pe);                                 // mark edges unvisited
  }
```

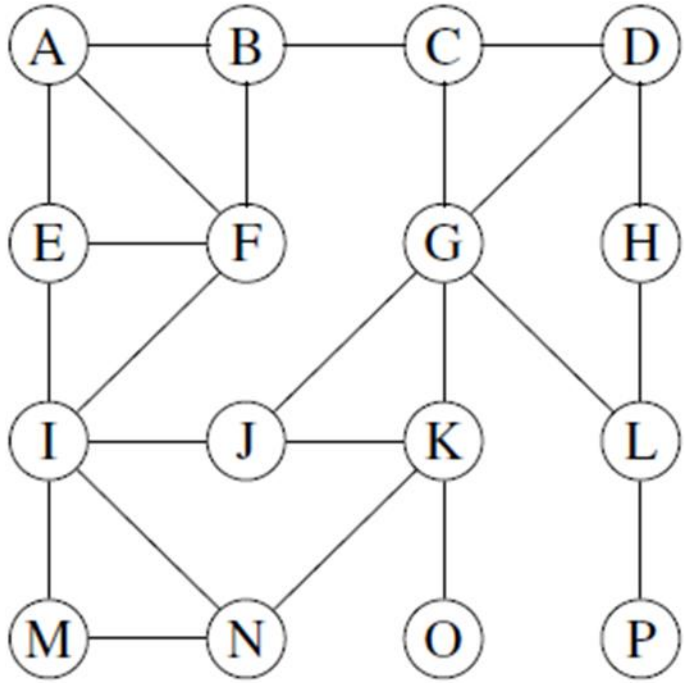
Code Fragment 13.6: The class constructor for DFS and the initialization function.



DFS Implementation

- **C++ Implementation**

```
/* DFS⟨G⟩ :: */                                  // recursive traversal
  void dfsTraversal(const Vertex& v) {
    startVisit(v);    visit(v);                  // visit v and mark visited
    EdgeList incident = v.incidentEdges();
    EdgeItor pe = incident.begin();
    while (!isDone() && pe != incident.end()) {  // visit v's incident edges
      Edge e = *pe++;
      if (!isVisited(e)) {                       // discovery edge?
        visit(e);                                // mark it visited
        Vertex w = e.opposite(v);                // get opposing vertex
        if (!isVisited(w)) {                     // unexplored?
          traverseDiscovery(e, v);               // let's discover it
          if (!isDone()) dfsTraversal(w);        // continue traversal
        }
        else traverseBack(e, v);                 // process back edge
      }
    }
    if (!isDone()) finishVisit(v);               // finished with v
  }
```



Code Fragment 13.7: The function dfsTraversal, which implements a generic DFS traversal.

DFS Implementation

- **C++ Implementation**
  - Counts the number of connected components of a graph

```
template <typename G>
class Components : public DFS<G> {              // count components
private:
  int nComponents;                              // num of components
public:
  Components(const G& g): DFS<G>(g) {}          // constructor
  int operator()();                             // count components
};
```

Code Fragment 13.8: The class Components, which extends the DFS class in order to count the number of components of a graph by overloading the "()" operator.

```
Components components(G);      // declare a Components object
int nc = components();         // compute the number of components
```

```
/* Components⟨G⟩ :: */                                   // count components
 int operator()() {
   initialize();                                         // initialize DFS
   nComponents = 0;                                      // init vertex count
   VertexList verts = graph.vertices();
   for (VertexItor pv = verts.begin(); pv != verts.end(); ++pv) {
     if (!isVisited(*pv)) {                              // not yet visited?
       dfsTraversal(*pv);                                // visit
       nComponents++;                                    // one more component
     }
   }
   return nComponents;
 }
```

Code Fragment 13.9: The overloaded () operator for class Components, which computes the number of connected components of a graph.

DFS Implementation

- **C++ Implementation**
  - Finds a path between a given source and target vertices

```
template <typename G>
class FindPath : public DFS<G> {                    // find a path by DFS
private:                                            // local data
  VertexList path;                                  // the path
  Vertex target;                                    // the target vertex
  bool done;                                        // is target found?
protected:                                          // overridden functions
  void startVisit(const Vertex& v);                 // visit vertex
  void finishVisit(const Vertex& v);                // finished with vertex
  bool isDone() const;                              // done yet?
public:
  FindPath(const G& g) : DFS<G>(g) {}               // constructor
                                                    // find path from s to t
  VertexList operator()(const Vertex& s, const Vertex& t);
};
```

Code Fragment 13.10: The class FindPath, which extends the DFS class in order to compute a path from source vertex s to target vertex t.

```
/* FindPath⟨G⟩ :: */                               // find path from s to t
  VertexList operator()(const Vertex& s, const Vertex& t) {
    initialize();                                   // initialize DFS
    path.clear();                                   // clear the path
    target = t;                                     // save the target
    done = false;
    dfsTraversal(s);                                // traverse starting at s
    return path;                                    // return the path
  }
```

Code Fragment 13.11: The overloaded () operator for class FindPath, which returns a path from s to t.

```
/* FindPath⟨G⟩ :: */                               // visit vertex
  void startVisit(const Vertex& v) {
    path.push_back(v);                              // insert into path
    if (v == target) done = true;                   // reached target vertex
  }
/* FindPath⟨G⟩ :: */                               // finished with vertex
  void finishVisit(const Vertex& v)
    { if (!done) path.pop_back(); }                 // remove last vertex
/* FindPath⟨G⟩ :: */                               // done yet?
  bool isDone() const
    { return done; }
```

Code Fragment 13.12: The overridden functions used by class FindPath.

DFS Implementation

- **C++ Implementation**
  - Finds a cycle in the connected component of a given start vertex *s*

```
template <typename G>
class FindCycle : public DFS<G> {
private:                                        // local data
  EdgeList cycle;                               // cycle storage
  Vertex cycleStart;                            // start of cycle
  bool done;                                    // cycle detected?
protected:                                      // overridden functions
  void traverseDiscovery(const Edge& e, const Vertex& from);
  void traverseBack(const Edge& e, const Vertex& from);
  void finishVisit(const Vertex& v);            // finished with vertex
  bool isDone() const;                          // done yet?
public:
  FindCycle(const G& g) : DFS<G>(g) {}          // constructor
  EdgeList operator()(const Vertex& s);         // find a cycle
};
```

Code Fragment 13.13: The class FindCycle, which extends the DFS class in order to compute a cycle in the connected component of a given start vertex *s*.

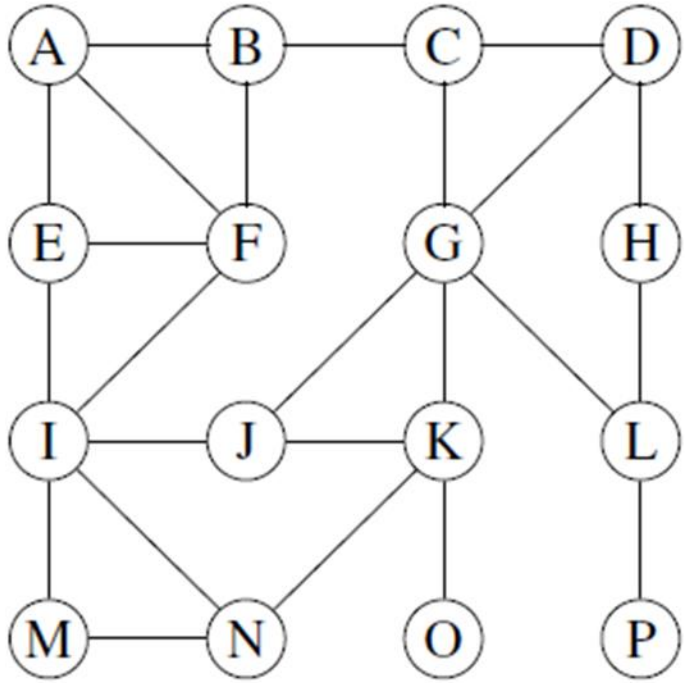
```
/* FindCycle⟨G⟩ :: */                          // find a cycle
  EdgeList operator()(const Vertex& s) {
    initialize();                               // initialize DFS
    cycle = EdgeList(); done = false;           // initialize members
    dfsTraversal(s);                            // do the search
    if (!cycle.empty() && s != cycleStart) {    // found a cycle?
      EdgeItor pe = cycle.begin();
      while (pe != cycle.end()) {               // search for prefix
        if ((pe++)->isIncidentOn(cycleStart)) break; // last edge of prefix?
      }
      cycle.erase(cycle.begin(), pe);           // remove prefix
    }
    return cycle;                               // return the cycle
  }
```

Code Fragment 13.14: The function of class FindCycle, which computes the cycle.

DFS Implementation

- **C++ Implementation**
  - Finds a cycle in the connected component of a given start vertex *s*

```
/* FindCycle⟨G⟩ :: */                                  // discovery edge e
  void traverseDiscovery(const Edge& e, const Vertex& from)
    { if (!done) cycle.push_back(e); }                  // add edge to list
/* FindCycle⟨G⟩ :: */                                  // back edge e
  void traverseBack(const Edge& e, const Vertex& from) {
    if (!done) {                                        // no cycle yet?
      done = true;                                      // cycle now detected
      cycle.push_back(e);                               // insert final edge
      cycleStart = e.opposite(from);                    // save starting vertex
    }
  }
/* FindCycle⟨G⟩ :: */                                  // finished with vertex
  void finishVisit(const Vertex& v) {
    if (!cycle.empty() && !done)                        // not building a cycle?
      cycle.pop_back();                                 // remove this edge
  }
/* FindCycle⟨G⟩ :: */                                  // done yet?
  bool isDone() const
    { return done; }
```



Code Fragment 13.15: The overridden functions used by class FindCycle.

Breadth-First Search

- **Breadth-First Search (BFS)**
  - Traverses a connected component of a graph
  - Defines a spanning tree
  - Proceeds in rounds and subdivides the vertices into *levels*
  - Starts at vertex s at level 0 and explores vertices
    at the same distance from s
    (*i.e.*, vertices at the same level)
  - A cross edge connects two nodes such that they do not have any ancestor and a descendant relationship between them

Algorithm BFS(s):

```
initialize collection L_0 to contain vertex s
i ← 0
while L_i is not empty do
    create collection L_{i+1} to initially be empty
    for all vertices v in L_i do
        for all edges e in v.incidentEdges() do
            if edge e is unexplored then
                w ← e.opposite(v)
                if vertex w is unexplored then
                    label e as a discovery edge
                    insert w into L_{i+1}
                else
                    label e as a cross edge
    i ← i + 1
```

Breadth-First Search

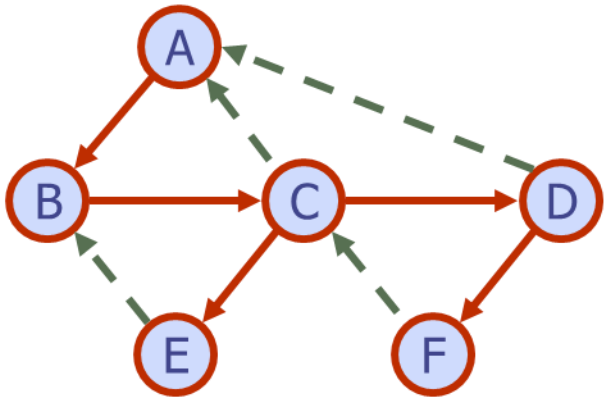
- **DFS vs. BFS**

Back edge (v,w)

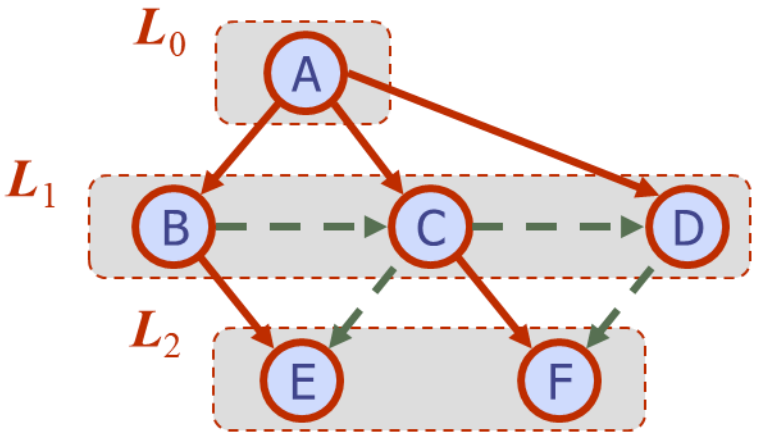
- w is an ancestor of v in the tree of discovery edges

Cross edge (v,w)

- w is in the same level as v or in the next level



DFS



BFS

Breadth-First Search

- **BFS Example**

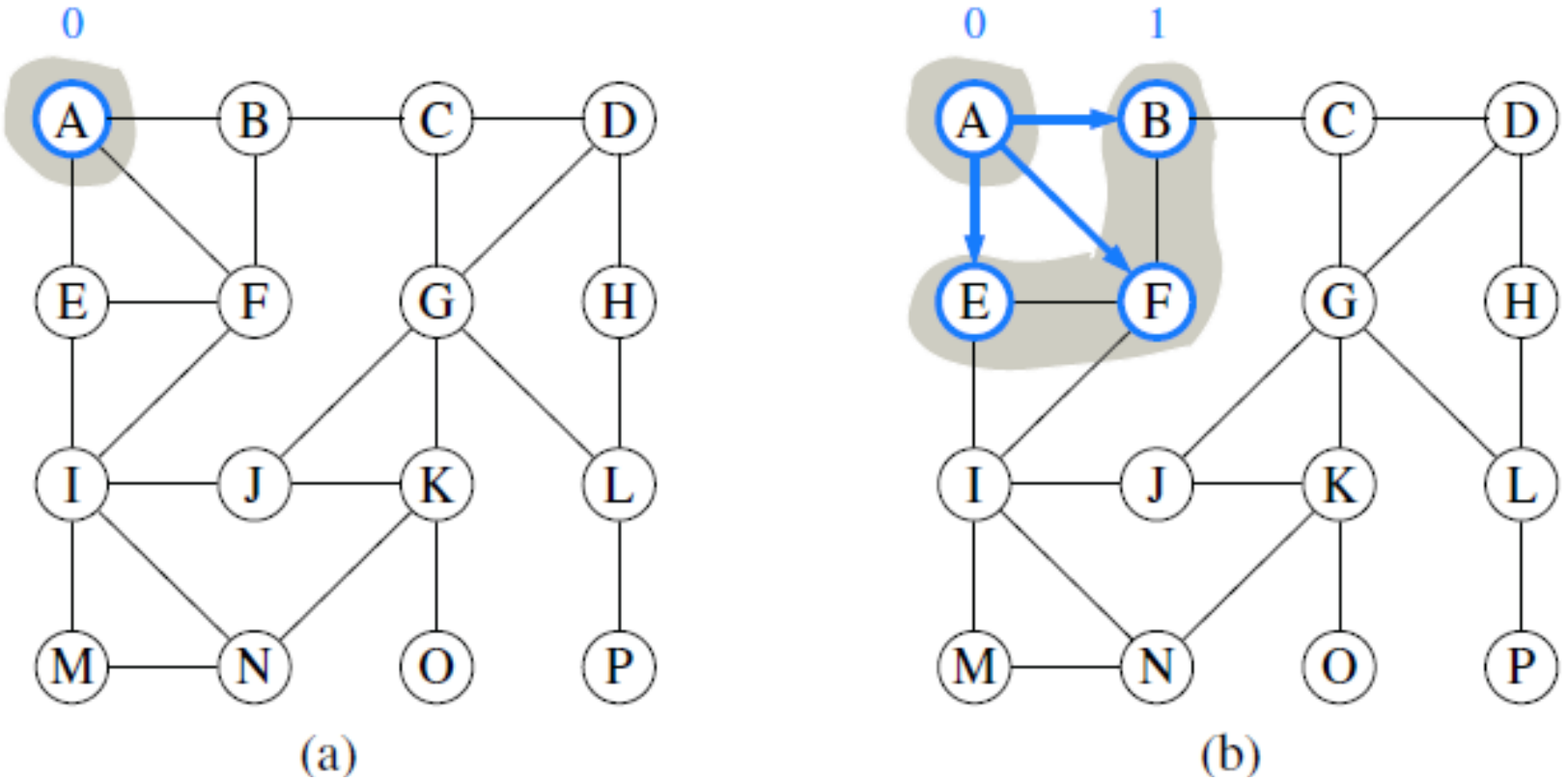


Figure 14.10: Example of breadth-first search traversal, where the edges incident to a vertex are considered in alphabetical order of the adjacent vertices. The discovery edges are shown with solid lines and the nontree (cross) edges are shown with dashed lines: (a) starting the search at A; (b) discovery of level 1; (c) discovery of level 2; (d) discovery of level 3; (e) discovery of level 4; (f) discovery of level 5.

Breadth-First Search

- **BFS Example**

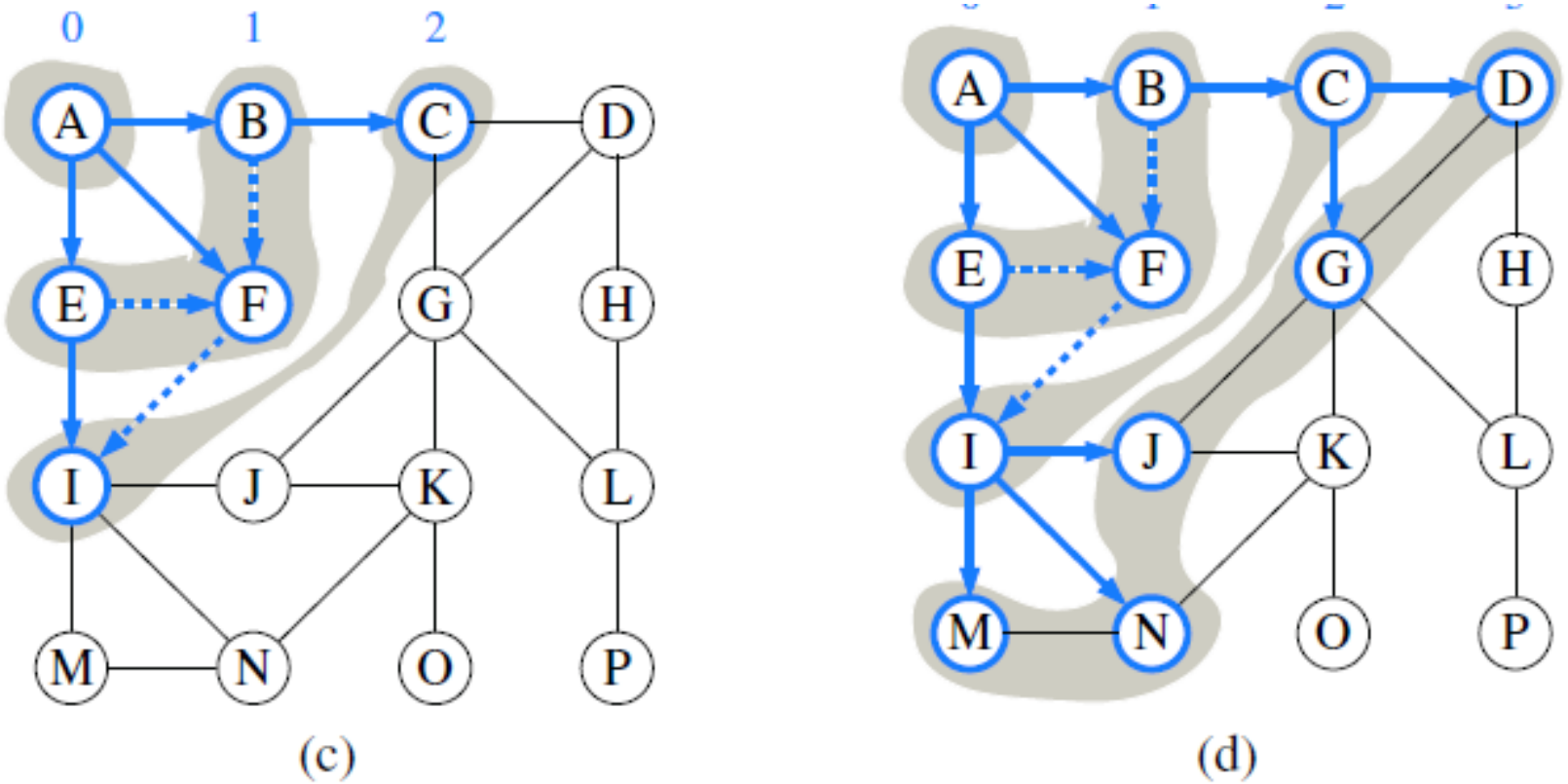
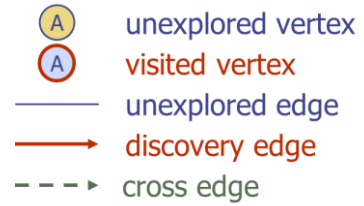


Figure 14.10: Example of breadth-first search traversal, where the edges incident to a vertex are considered in alphabetical order of the adjacent vertices. The discovery edges are shown with solid lines and the nontree (cross) edges are shown with dashed lines: (a) starting the search at A; (b) discovery of level 1; (c) discovery of level 2; (d) discovery of level 3; (e) discovery of level 4; (f) discovery of level 5.



Breadth-First Search

- **BFS Example**

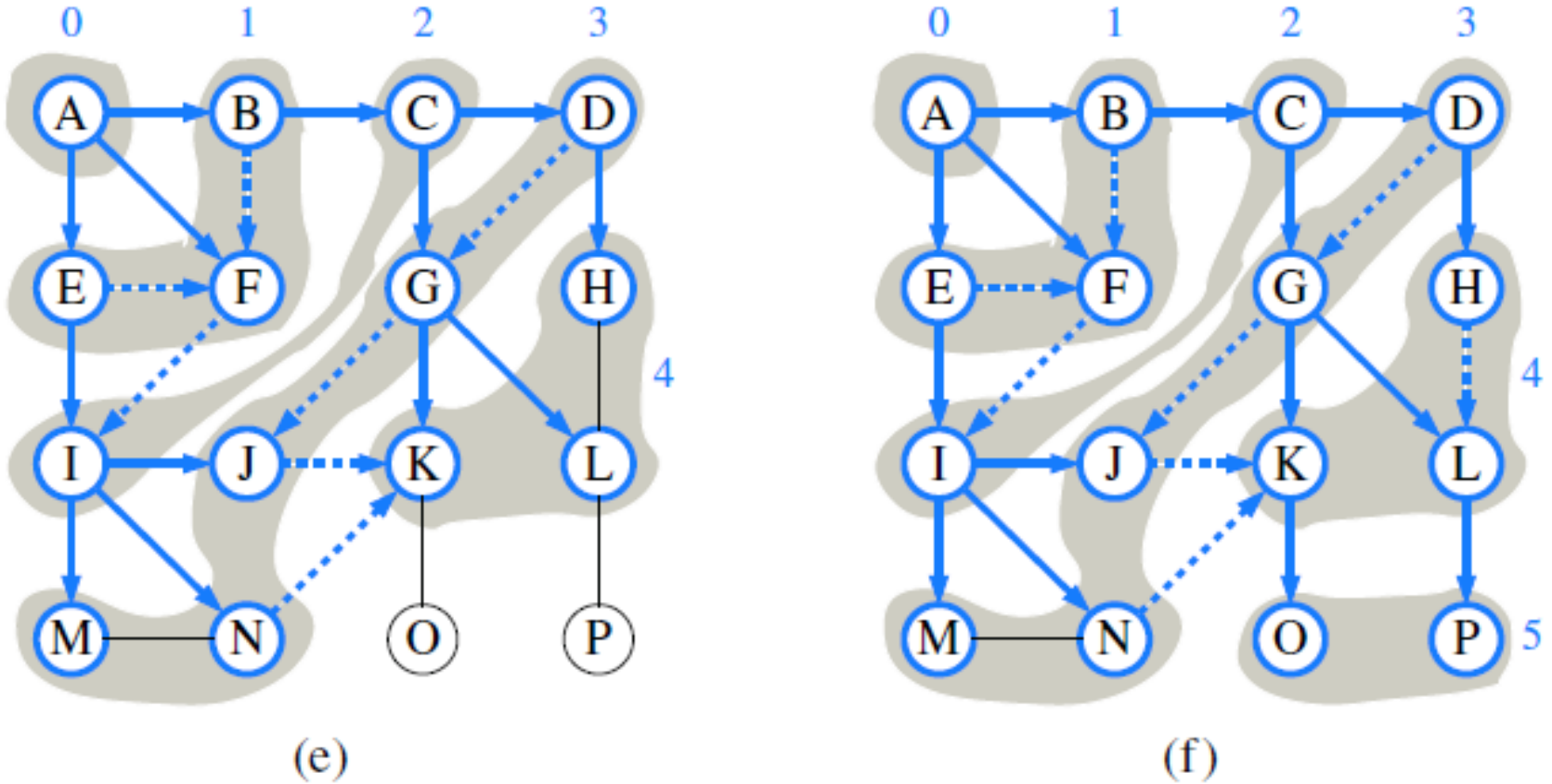
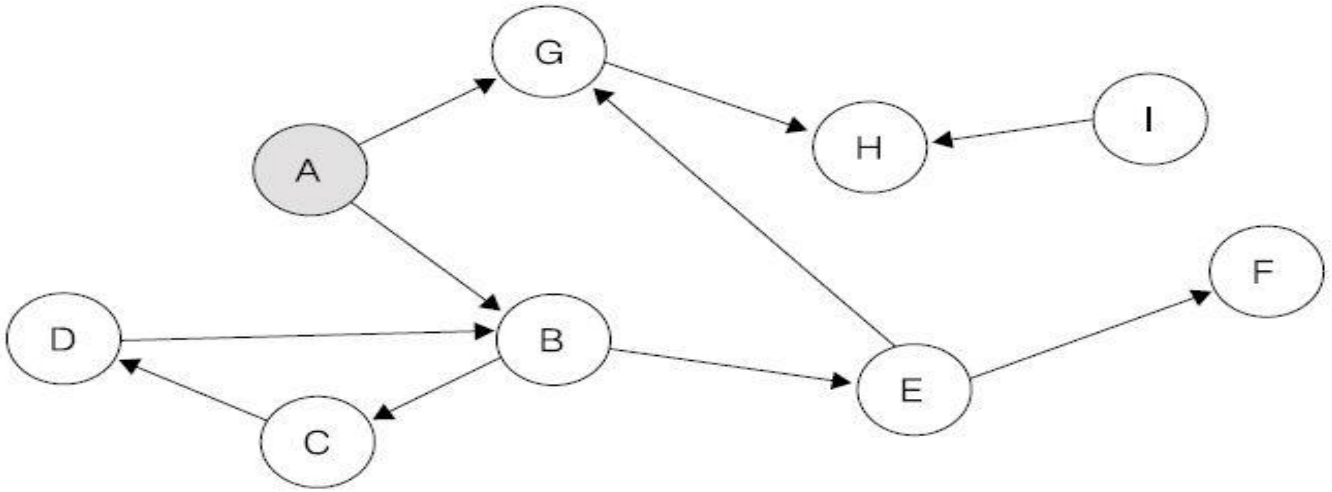
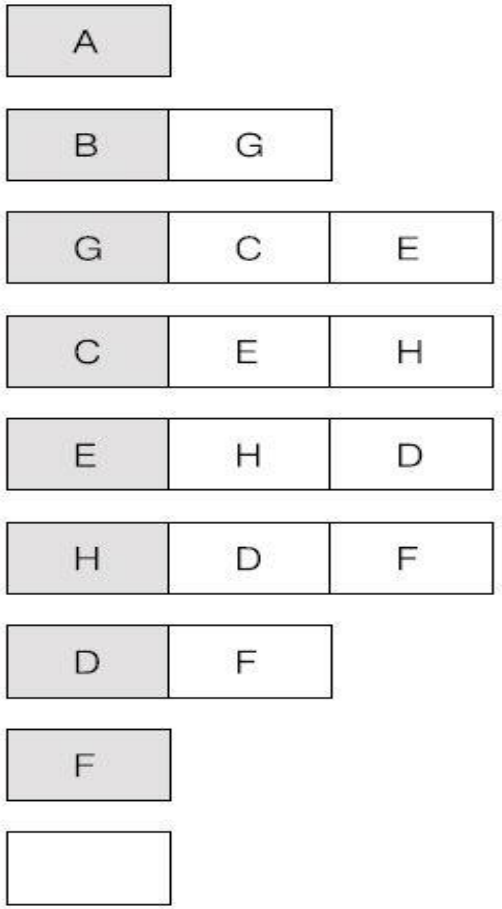


Figure 14.10: Example of breadth-first search traversal, where the edges incident to a vertex are considered in alphabetical order of the adjacent vertices. The discovery edges are shown with solid lines and the nontree (cross) edges are shown with dashed lines: (a) starting the search at A; (b) discovery of level 1; (c) discovery of level 2; (d) discovery of level 3; (e) discovery of level 4; (f) discovery of level 5.

Breadth-First Search

- **BFS using a Queue**





Breadth-First Search

- **BFS Property**

Proposition 14.16: *Let G be an undirected or directed graph on which a BFS traversal starting at vertex s has been performed. Then*

- *The traversal visits all vertices of G that are reachable from s.*
- *For each vertex v at level i, the path of the BFS tree T between s and v has i edges, and any other path of G from s to v has at least i edges.*
- *If (u,v) is an edge that is not in the BFS tree, then the level number of v can be at most 1 greater than the level number of u.*

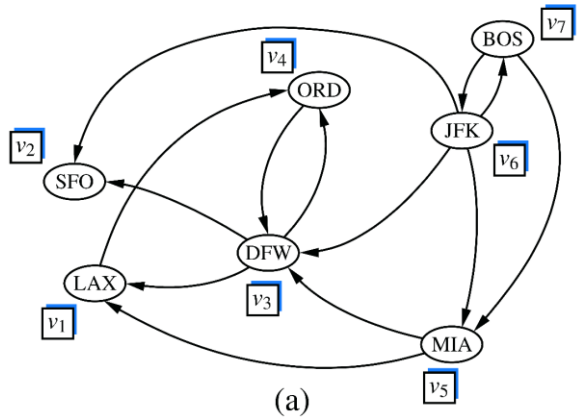
The analysis of the running time of BFS is similar to the one of DFS, with the algorithm running in $O(n+m)$ time, or more specifically, in $O(n_s+m_s)$ time if n_s is the number of vertices reachable from vertex s, and $m_s \leq m$ is the number of incident edges to those vertices. To explore the entire graph, the process can be restarted at another vertex, akin to the DFSComplete method of Code Fragment 14.7. The actual path from vertex s to vertex v can be reconstructed using the constructPath method of Code Fragment 14.6

Proposition 14.17: *Let G be a graph with n vertices and m edges represented with the adjacency list structure. A BFS traversal of G takes $O(n+m)$ time.*

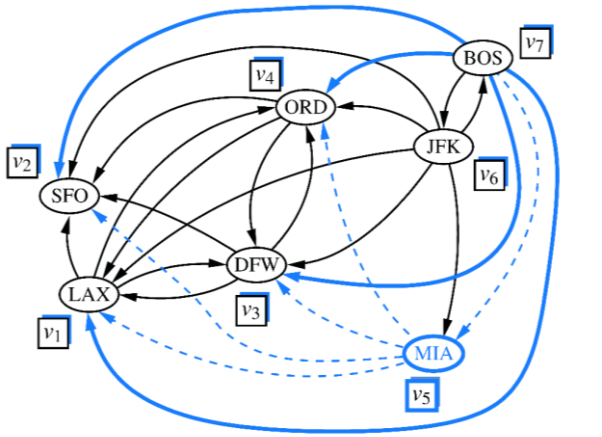
Transitive Closure

- **Transitive Closure**

Proposition 14.18: *For $i = 1, \ldots, n$, directed graph $\vec{G}_k$ has an edge (v_i, v_j) if and only if directed graph $\vec{G}$ has a directed path from v_i to v_j, whose intermediate vertices (if any) are in the set $\{v_1, \ldots, v_k\}$. In particular, $\vec{G}_n$ is equal to $\vec{G}^*$, the transitive closure of $\vec{G}$.*



(a)



(f)

- **Floyd-Warshall Algorithm**

Algorithm FloydWarshall($\vec{G}$):
Input: A directed graph $\vec{G}$ with n vertices
Output: The transitive closure $\vec{G}^*$ of $\vec{G}$

```
let v_1, v_2, ..., v_n be an arbitrary numbering of the vertices of G
G_0 = G
for k = 1 to n do
    G_k = G_{k-1}
    for all i, j in {1, ..., n} with i ≠ j and i, j ≠ k do
        if both edges (v_i, v_k) and (v_k, v_j) are in G_{k-1} then
            add edge (v_i, v_j) to G_k (if it is not already present)
return G_n
```

Code Fragment 14.9: Pseudocode for the Floyd-Warshall algorithm. This algorithm computes the transitive closure $\vec{G}^*$ of G by incrementally computing a series of directed graphs $\vec{G}_0, \vec{G}_1, \ldots, \vec{G}_n$, for $k = 1, \ldots, n$.

Transitive Closure

- **Floyd-Warshall Algorithm Example**

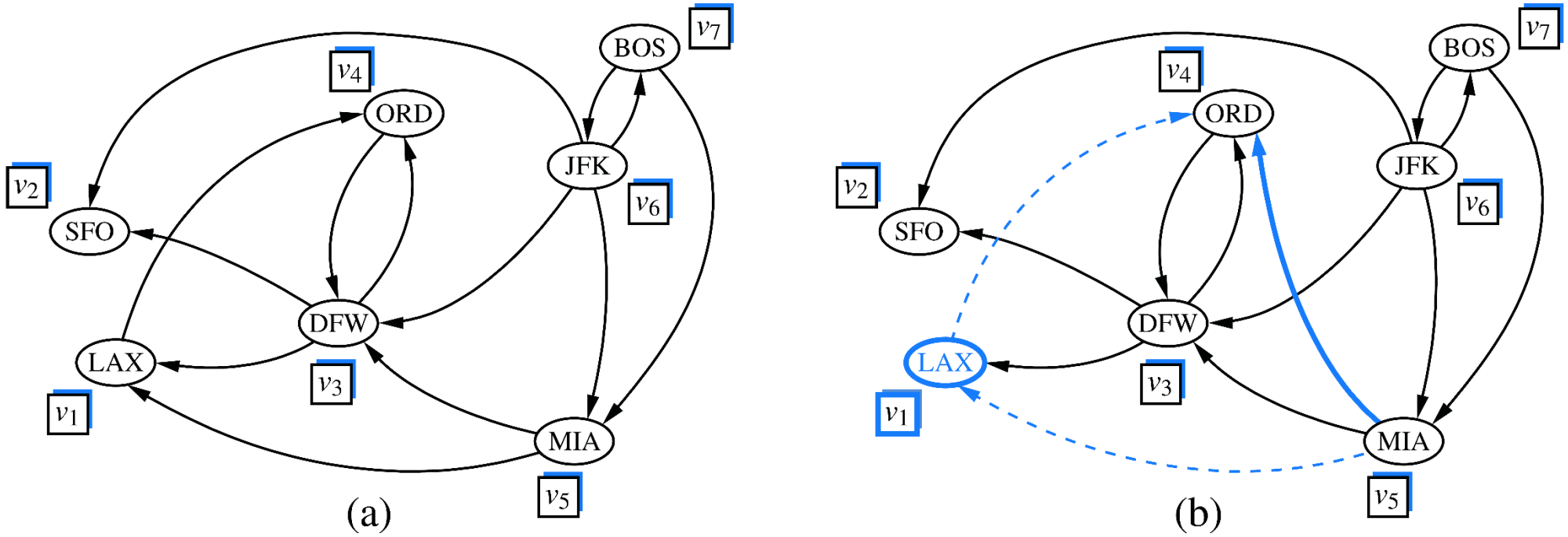


Figure 14.11: Sequence of directed graphs computed by the Floyd-Warshall algorithm: (a) initial directed graph $\vec{G} = \vec{G}_0$ and numbering of the vertices; (b) directed graph $\vec{G}_1$; (c) $\vec{G}_2$; (d) $\vec{G}_3$; (e) $\vec{G}_4$; (f) $\vec{G}_5$. Note that $\vec{G}_5 = \vec{G}_6 = \vec{G}_7$. If directed graph $\vec{G}_{k-1}$ has the edges (v_i, v_k) and (v_k, v_j), but not the edge (v_i, v_j), in the drawing of directed graph $\vec{G}_k$, we show edges (v_i, v_k) and (v_k, v_j) with dashed lines, and edge (v_i, v_j) with a thick line. For example, in (b) existing edges (MIA,LAX) and (LAX,ORD) result in new edge (MIA,ORD).

Transitive Closure

- **Floyd-Warshall Algorithm Example**

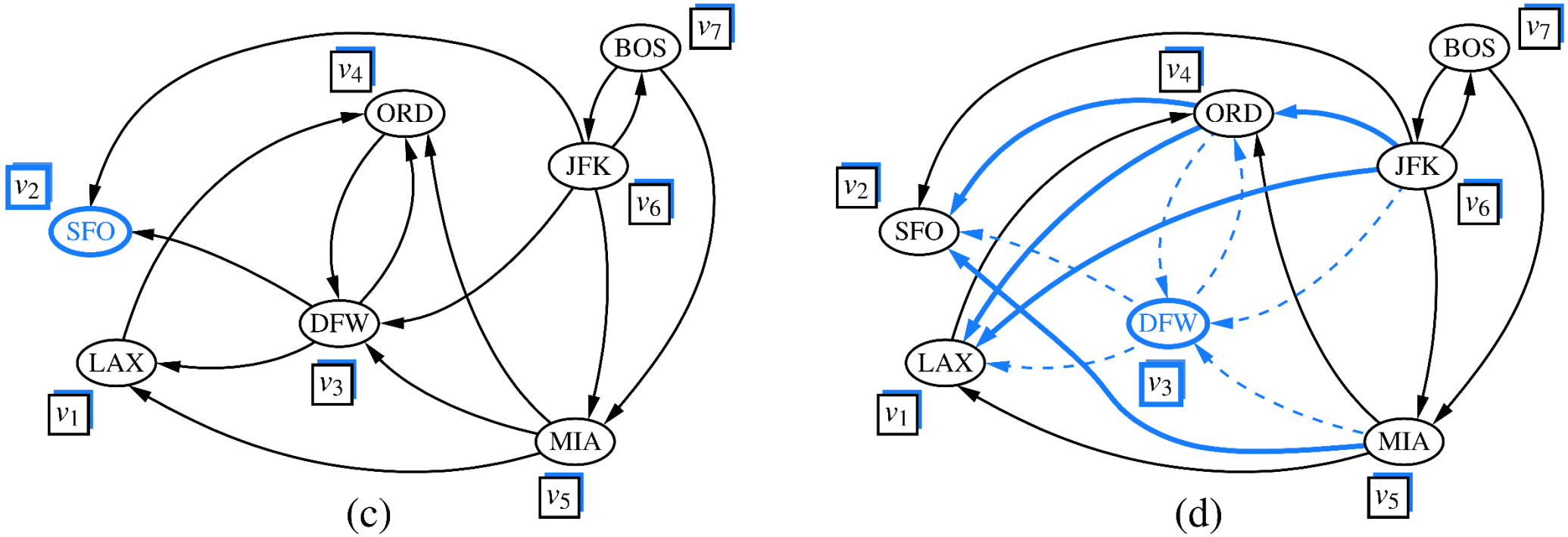


Figure 14.11: Sequence of directed graphs computed by the Floyd-Warshall algorithm: (a) initial directed graph $\vec{G} = \vec{G}_0$ and numbering of the vertices; (b) directed graph $\vec{G}_1$; (c) $\vec{G}_2$; (d) $\vec{G}_3$; (e) $\vec{G}_4$; (f) $\vec{G}_5$. Note that $\vec{G}_5 = \vec{G}_6 = \vec{G}_7$. If directed graph $\vec{G}_{k-1}$ has the edges (v_i, v_k) and (v_k, v_j), but not the edge (v_i, v_j), in the drawing of directed graph $\vec{G}_k$, we show edges (v_i, v_k) and (v_k, v_j) with dashed lines, and edge (v_i, v_j) with a thick line. For example, in (b) existing edges (MIA,LAX) and (LAX,ORD) result in new edge (MIA,ORD).

Transitive Closure

- **Floyd-Warshall Algorithm Example**

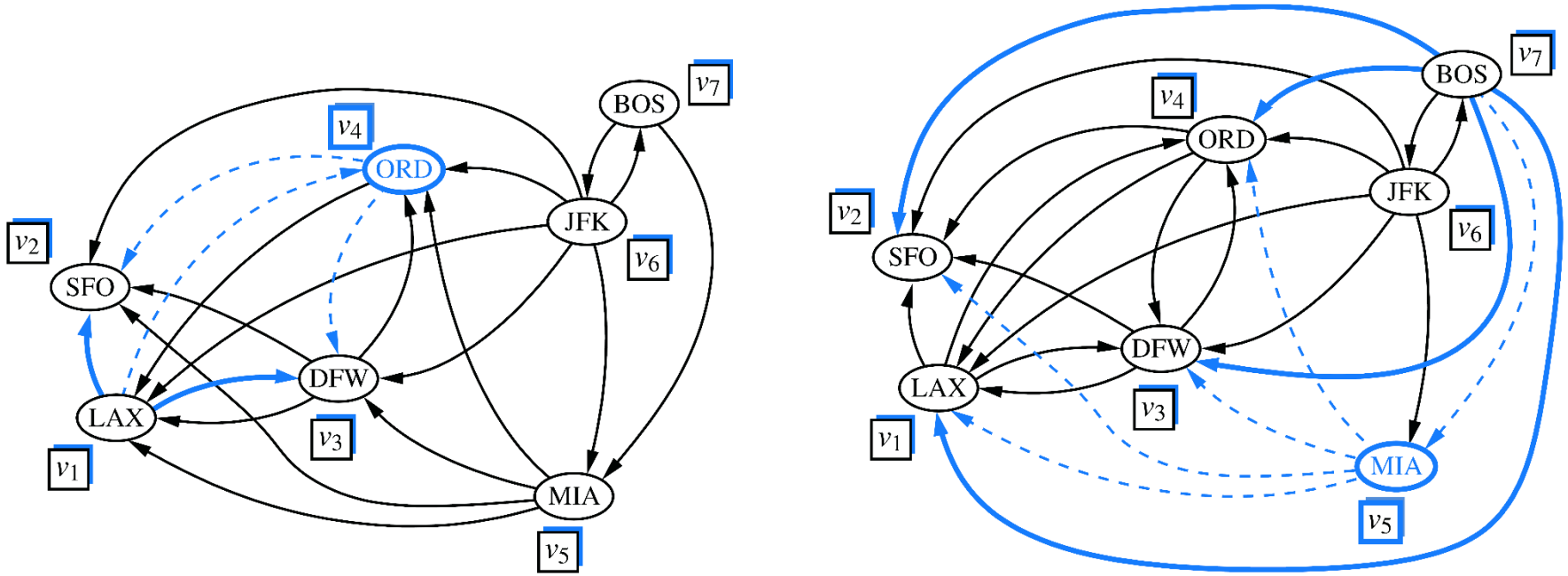
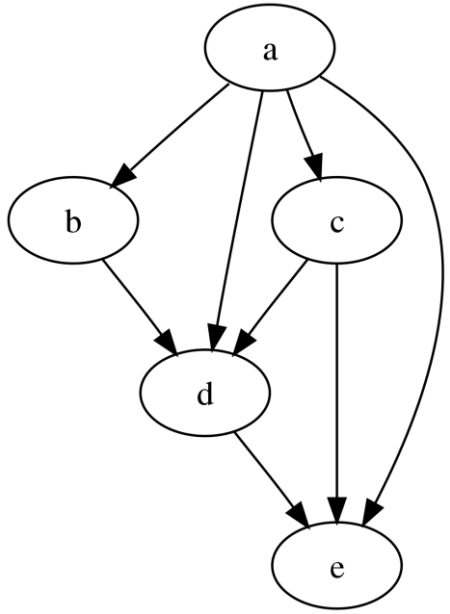


Figure 14.11: Sequence of directed graphs computed by the Floyd-Warshall algorithm: (a) initial directed graph $\vec{G} = \vec{G}_0$ and numbering of the vertices; (b) directed graph $\vec{G}_1$; (c) $\vec{G}_2$; (d) $\vec{G}_3$; (e) $\vec{G}_4$; (f) $\vec{G}_5$. Note that $\vec{G}_5 = \vec{G}_6 = \vec{G}_7$. If directed graph $\vec{G}_{k-1}$ has the edges (v_i, v_k) and (v_k, v_j), but not the edge (v_i, v_j), in the drawing of directed graph $\vec{G}_k$, we show edges (v_i, v_k) and (v_k, v_j) with dashed lines, and edge (v_i, v_j) with a thick line. For example, in (b) existing edges (MIA,LAX) and (LAX,ORD) result in new edge (MIA,ORD).

Directed Acyclic Graphs

- **Directed Acyclic Graph (DAG)**
  - A directed graph without directed cycles

- **Topological Ordering**
  - An ordering such that any directed path in a directed graph $\vec{G}$ traverses vertices in increasing order
  - An ordering $(v_1, \ldots, v_n)$ of n vertices of $\vec{G}$ such that $i < j$ for every edge (v_i, v_j)



https://en.wikipedia.org/wiki/Directed_acyclic_graph

(a) A C E B D F G H

(b) B A C D F E G H

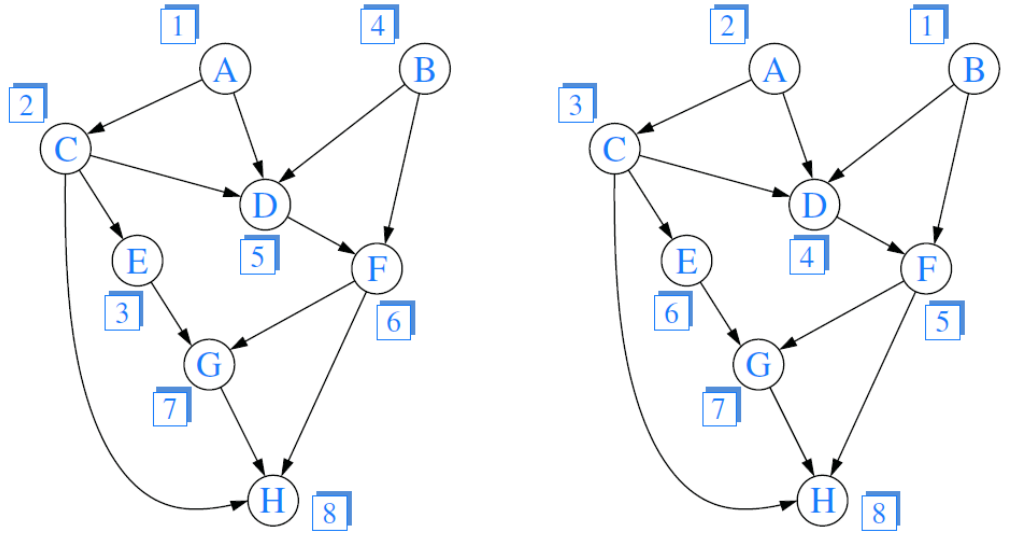


Figure 14.12: Two topological orderings of the same acyclic directed graph.

Directed Acyclic Graphs

- **Topological Sorting**
  - Computes a topological ordering of a digraph

Algorithm TopologicalSort(G):
Input: A digraph G with n vertices
Output: A topological ordering $v_1, \ldots, v_n$ of G

```
S ← an initially empty stack.
for all u in G.vertices() do
    Let incounter(u) be the in-degree of u.
    if incounter(u) = 0 then
        S.push(u)
i ← 1
while !S.empty() do
    u ← S.pop()
    Let u be vertex number i in the topological ordering.
    i ← i + 1
    for all outgoing edges (u, w) of u do
        incounter(w) ← incounter(w) − 1
        if incounter(w) = 0 then
            S.push(w)
```

Code Fragment 13.23: Pseudo-code for the topological sorting algorithm.

Directed Acyclic Graphs

- **Topological Sorting**

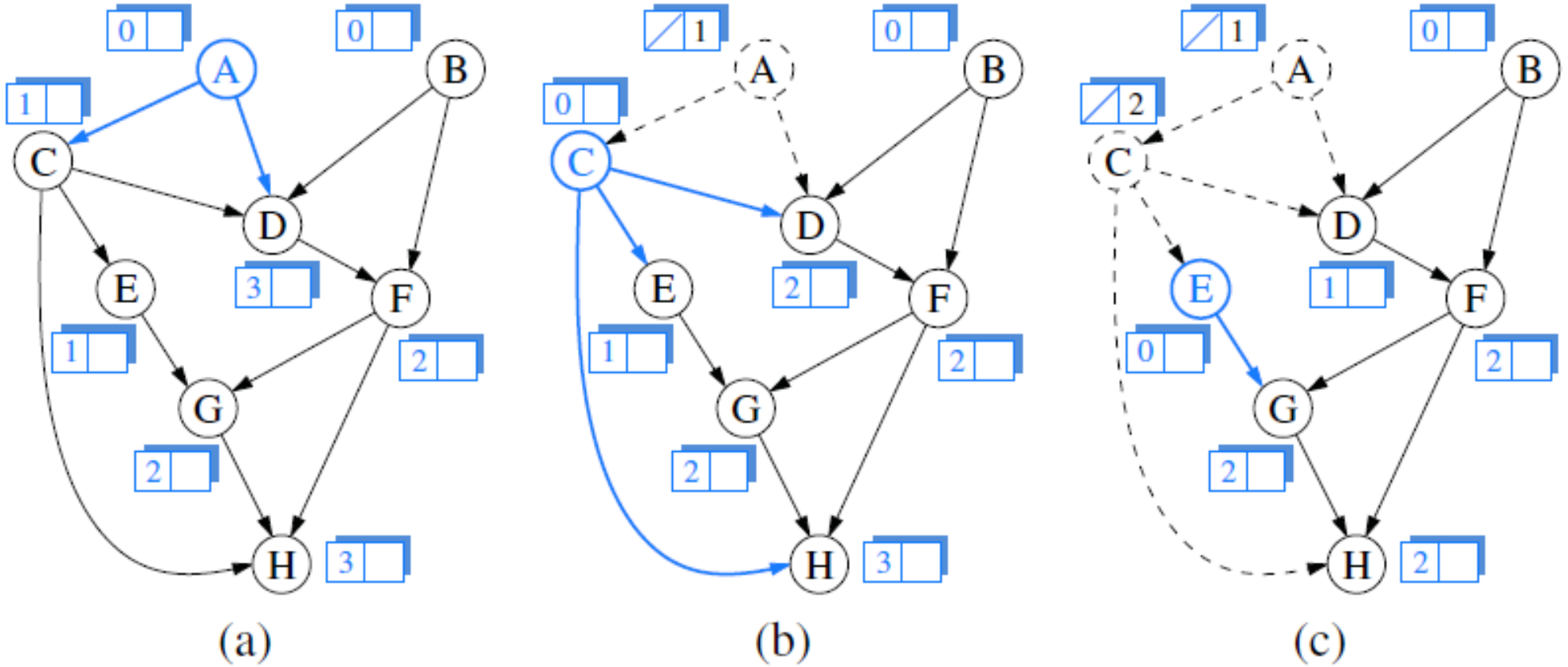


Figure 14.13: Example of a run of algorithm topologicalSort (Code Fragment 14.11). The label near a vertex shows its current inCount value, and its eventual rank in the resulting topological order. The highlighted vertex is one with inCount equal to zero that will become the next vertex in the topological order. Dashed lines denote edges that have already been examined, which are no longer reflected in the inCount values.

Directed Acyclic Graphs

- **Topological Sorting**

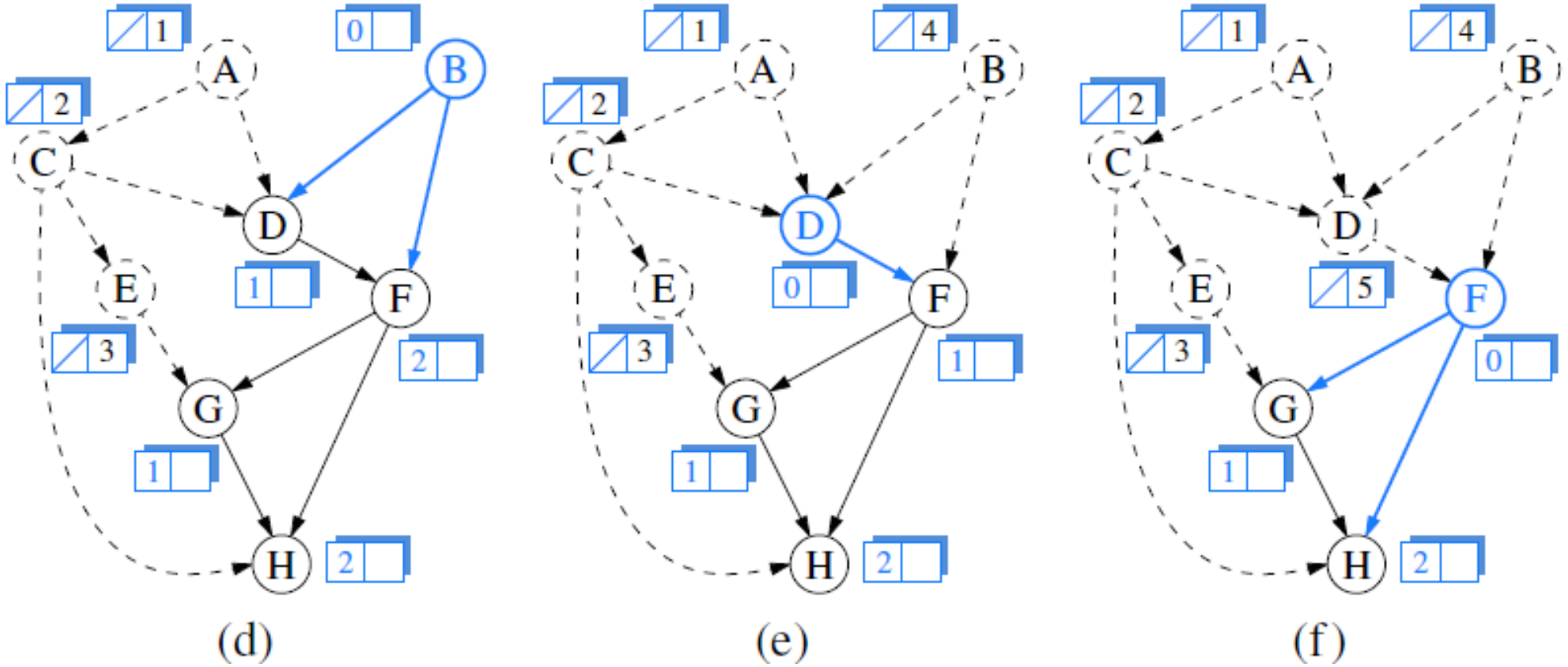


Figure 14.13: Example of a run of algorithm topologicalSort (Code Fragment 14.11). The label near a vertex shows its current inCount value, and its eventual rank in the resulting topological order. The highlighted vertex is one with inCount equal to zero that will become the next vertex in the topological order. Dashed lines denote edges that have already been examined, which are no longer reflected in the inCount values.

Directed Acyclic Graphs

- **Topological Sorting**

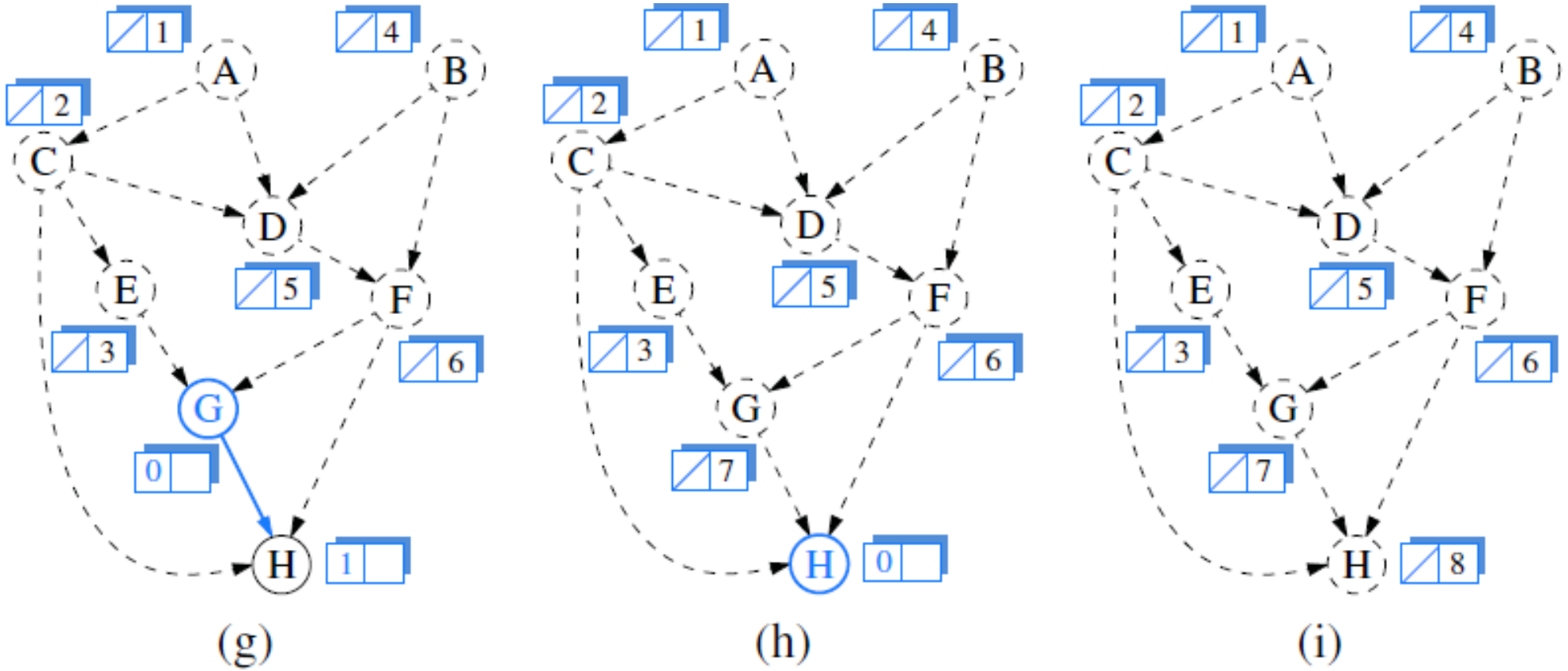


Figure 14.13: Example of a run of algorithm topologicalSort (Code Fragment 14.11). The label near a vertex shows its current inCount value, and its eventual rank in the resulting topological order. The highlighted vertex is one with inCount equal to zero that will become the next vertex in the topological order. Dashed lines denote edges that have already been examined, which are no longer reflected in the inCount values.

Shortest Paths

- **Weighted Graphs**
  - A graph G that has a numeric label $w(e)$ associated with each edge e, called the *weight* of edge e
  - The *length* $w(P)$ of a path P is the sum of the weights of the edges of P

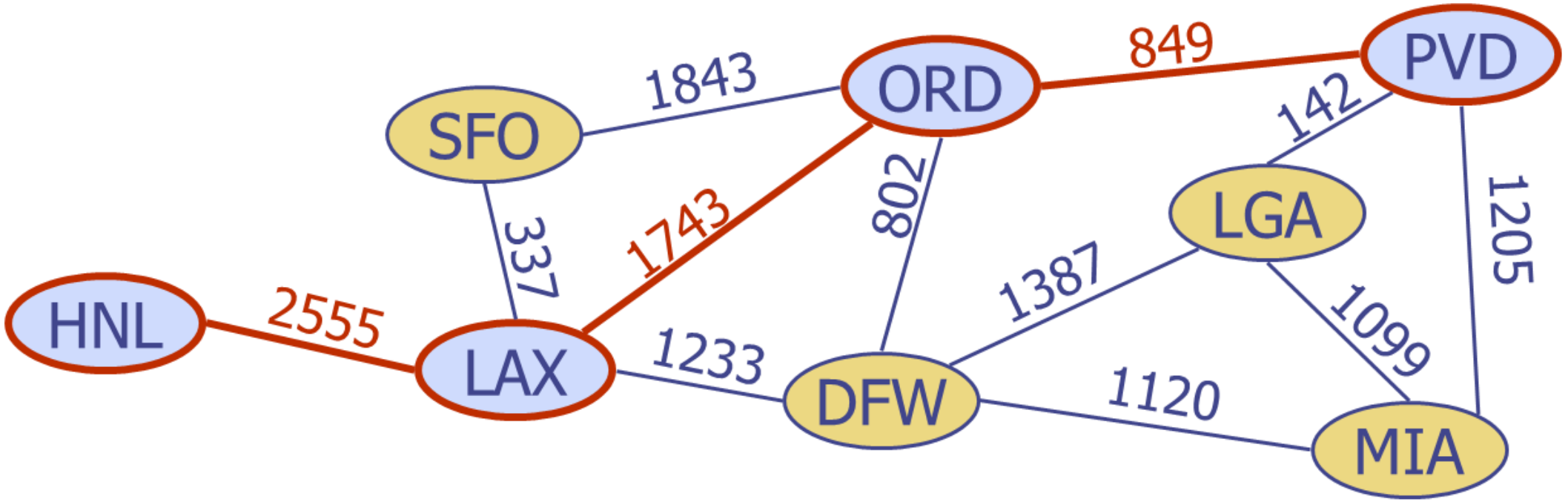
$$P = \big((v_0, v_1), (v_1, v_2), \ldots, (v_{k-1}, v_k)\big)$$

$$w(P) = \sum_{i=0}^{k-1} w\big((v_i, v_{i+1})\big)$$

  - The *distance* $d(v, u)$ from a vertex v to a vertex u in G is the length of a minimum length path (*i.e., shortest path*) from v to u (if such a path exists)
  - $d(v, u) = \infty$ if there is no path at all from v to u in G
  - $d(v, u)$ may not be defined if there is a cycle in G whose total weight is negative

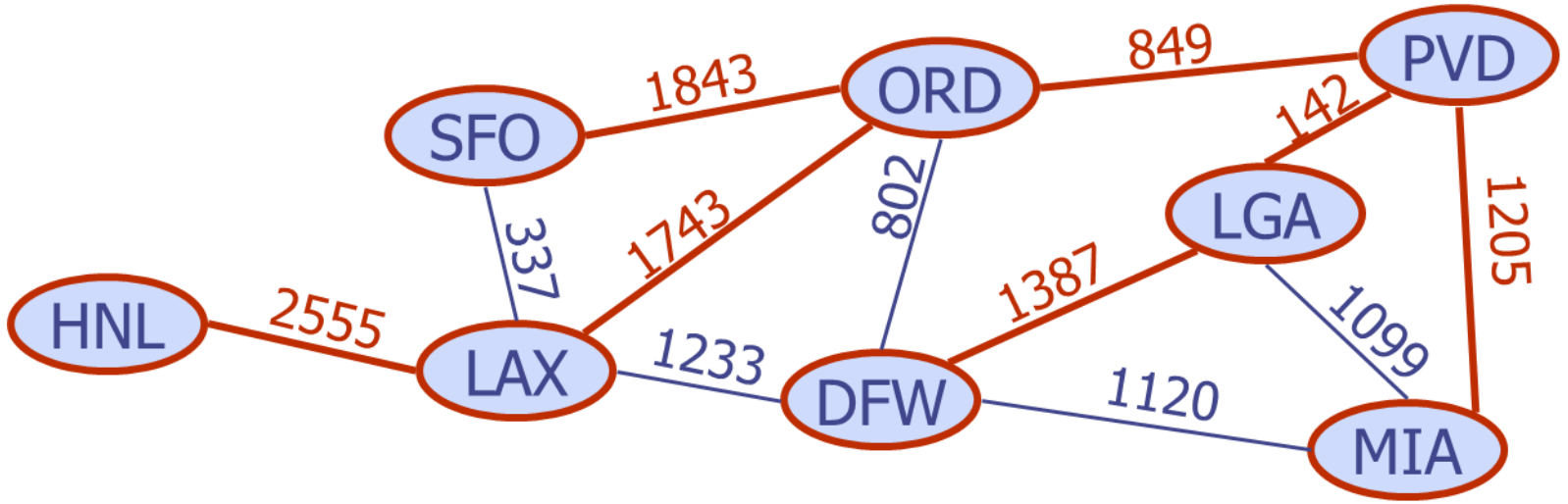
Shortest Paths

- **Shortest Path Example**
  - Shortest path between PVD and HNL



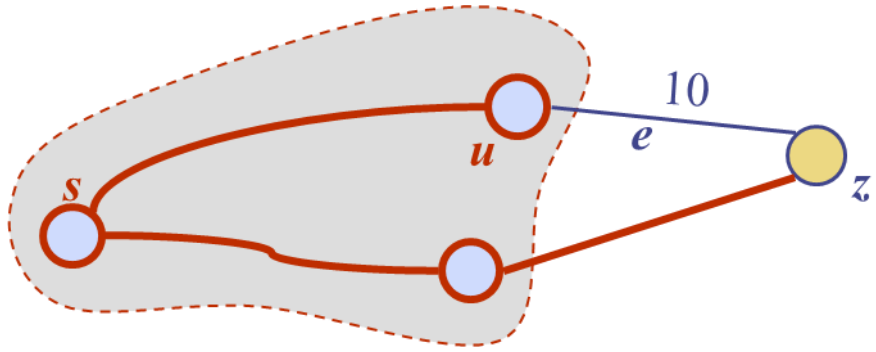
Shortest Paths

- **Shortest Path Properties**
  - A subpath of a shortest path is itself a shortest path
  - There is a tree of shortest paths from a start vertex to all the other vertices



Dijkstra's Algorithm

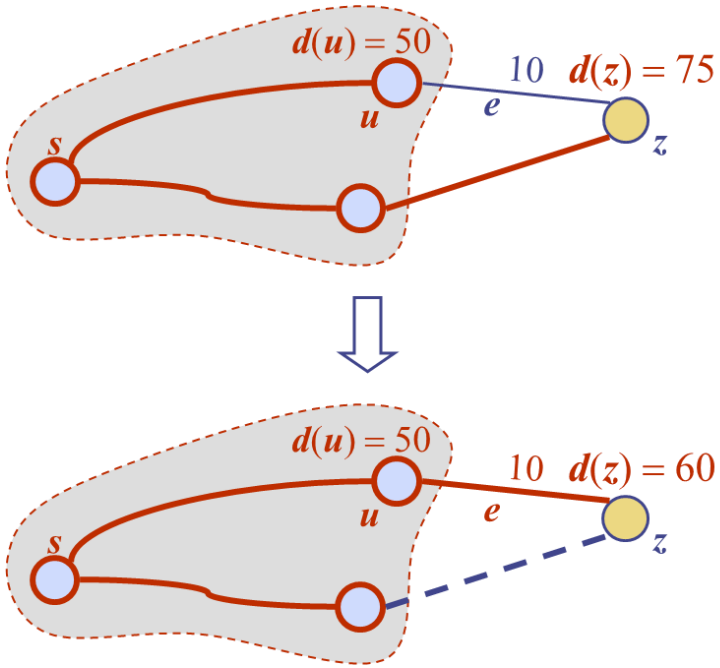
- **Dijkstra's Algorithm**
  - The greedy method to the single-source shortest-path problem
  - Computes the distances of all the vertices from a given start vertex v
  - A "weighted" breadth-first search starting at v
  - In each iteration, the next vertex chosen is the one closest to v
  - Assume that the input graph G is undirected and simple (*i.e.*, no self-loops and parallel edges)



Dijkstra's Algorithm

▪ Edge Relaxation

Let us define a label $D[v]$ for each vertex v in V, which we use to approximate the distance in G from s to v. The meaning of these labels is that $D[v]$ will always store the length of the best path we have found ***so far*** from s to v. Initially, $D[s] = 0$ and $D[v] = \infty$ for each $v \neq s$, and we define the set C, which is our "***cloud***" of vertices, to initially be the empty set. At each iteration of the algorithm, we select a vertex u not in C with smallest $D[u]$ label, and we pull u into C. (In general, we will use a priority queue to select among the vertices outside the cloud.) In the very first iteration we will, of course, pull s into C. Once a new vertex u is pulled into C, we update the label $D[v]$ of each vertex v that is adjacent to u and is outside of C, to reflect the fact that there may be a new and better way to get to v via u. This update operation is known as a ***relaxation*** procedure, for it takes an old estimate and checks if it can be improved to get closer to its true value. The specific edge relaxation operation is as follows:



Edge Relaxation:

if $D[u] + w(u,v) < D[v]$ **then**
$D[v] = D[u] + w(u,v)$

Dijkstra's Algorithm

- **Pseudocode**

Algorithm ShortestPath(G, s):
Input: A directed or undirected graph G with nonnegative edge weights, and a distinguished vertex s of G.
Output: The length of a shortest path from s to v for each vertex v of G.

```
Initialize D[s] = 0 and D[v] = ∞ for each vertex v ≠ s.
Let a priority queue Q contain all the vertices of G using the D labels as keys.
while Q is not empty do
    {pull a new vertex u into the cloud}
    u = value returned by Q.removeMin()
    for each edge (u, v) such that v is in Q do
        {perform the relaxation procedure on edge (u, v)}
        if D[u] + w(u, v) < D[v] then
            D[v] = D[u] + w(u, v)
            Change the key of vertex v in Q to D[v].
return the label D[v] of each vertex v
```

Code Fragment 14.12: Pseudocode for Dijkstra's algorithm, solving the single-source shortest-path problem for an undirected or directed graph.

Dijkstra's Algorithm

- **Example**

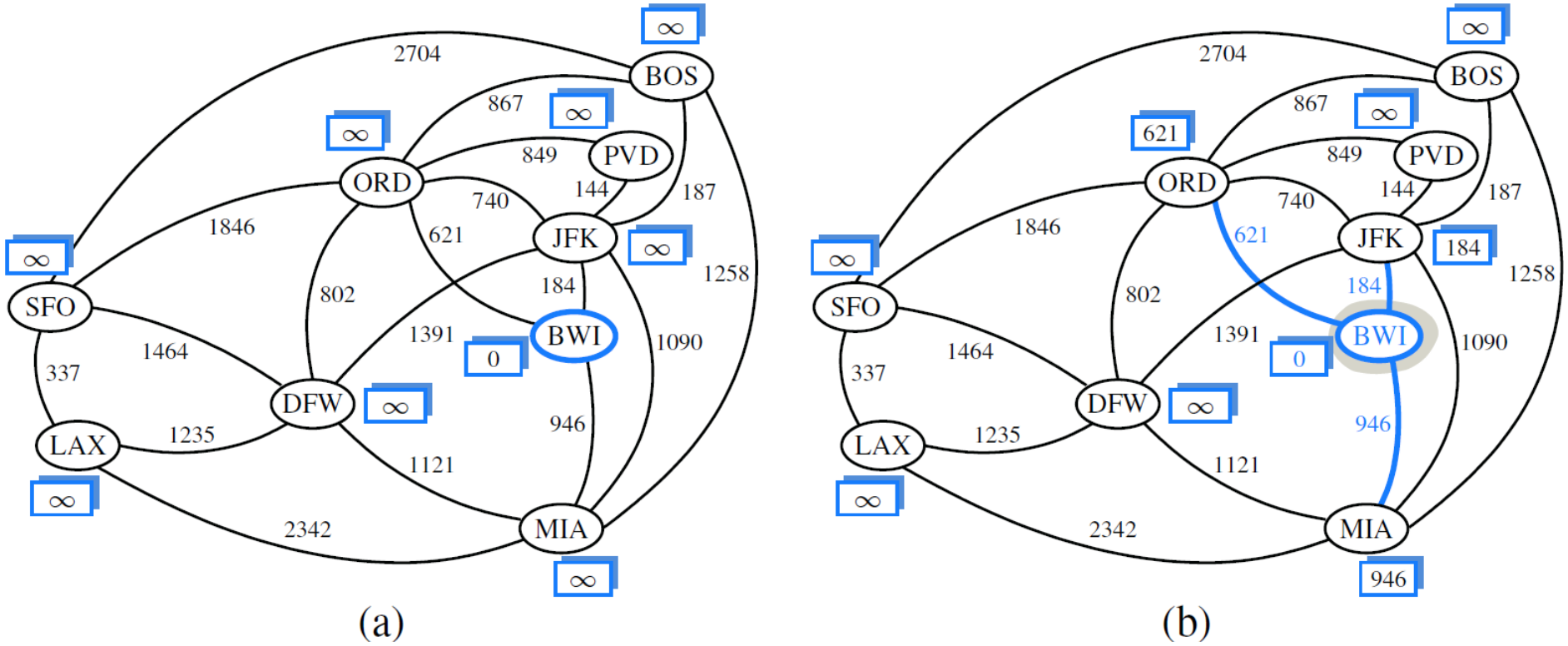


Figure 14.15: An example execution of Dijkstra's shortest-path algorithm on a weighted graph. The start vertex is BWI. A box next to each vertex v stores the label $D[v]$. The edges of the shortest-path tree are drawn as thick arrows, and for each vertex u outside the "cloud" we show the current best edge for pulling in u with a thick line. (Continues in Figure 14.16.)

Dijkstra's Algorithm

- **Example**

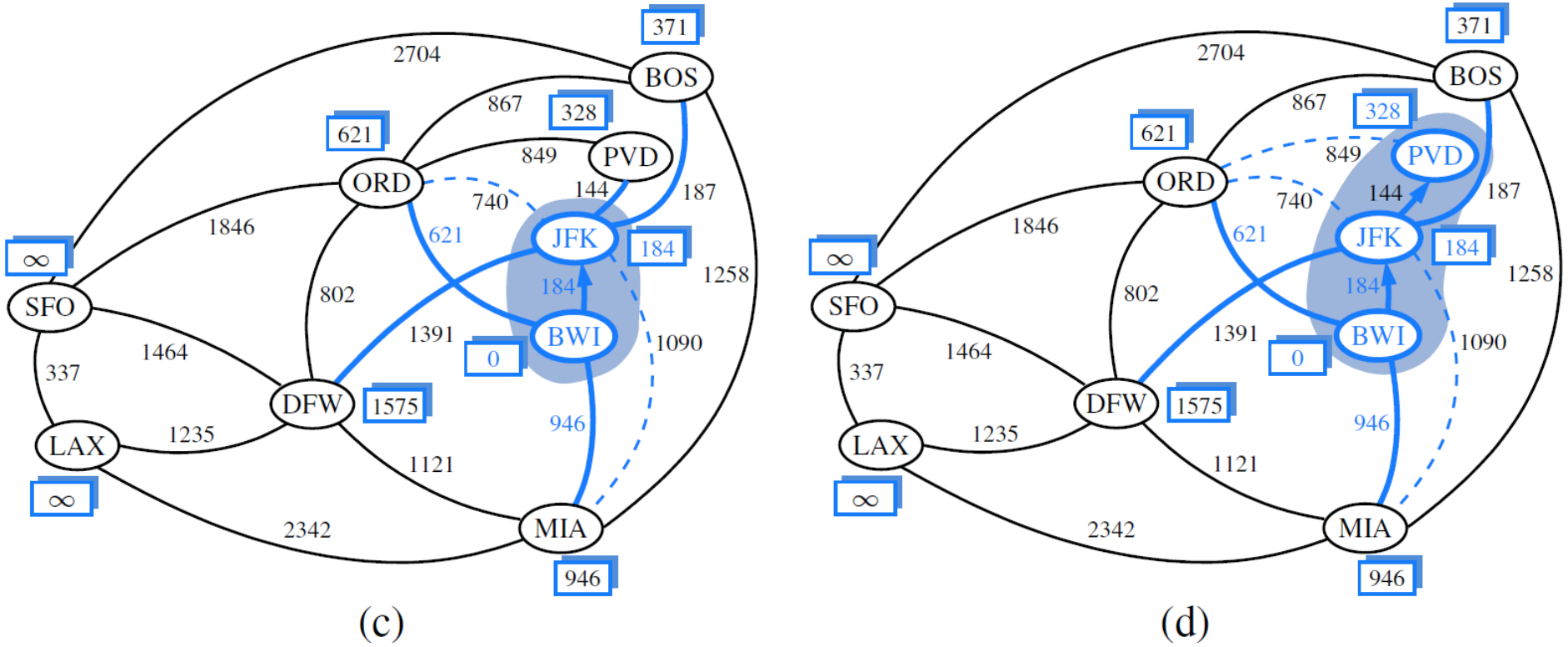


Figure 14.15: An example execution of Dijkstra's shortest-path algorithm on a weighted graph. The start vertex is BWI. A box next to each vertex v stores the label $D[v]$. The edges of the shortest-path tree are drawn as thick arrows, and for each vertex u outside the "cloud" we show the current best edge for pulling in u with a thick line. (Continues in Figure 14.16.)

Dijkstra's Algorithm

- **Example**

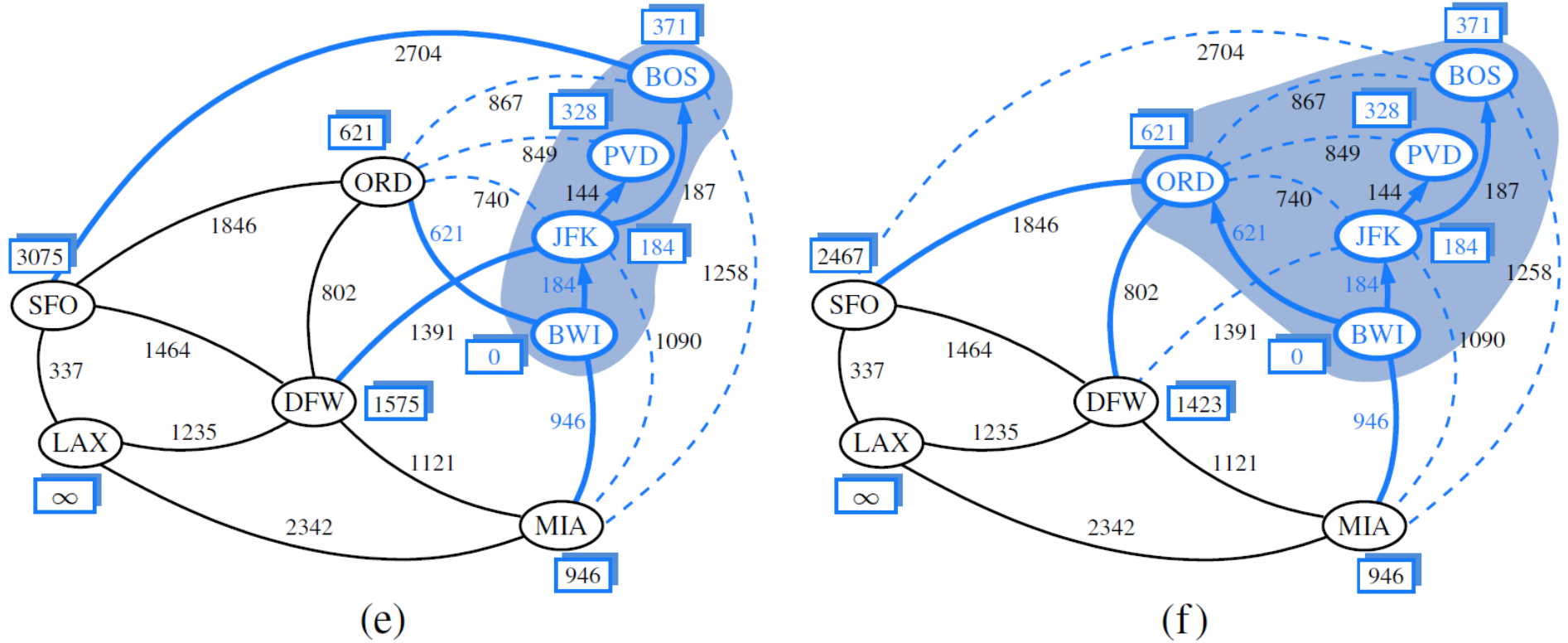


Figure 14.15: An example execution of Dijkstra's shortest-path algorithm on a weighted graph. The start vertex is BWI. A box next to each vertex v stores the label $D[v]$. The edges of the shortest-path tree are drawn as thick arrows, and for each vertex u outside the "cloud" we show the current best edge for pulling in u with a thick line. (Continues in Figure 14.16.)

Dijkstra's Algorithm

- **Example**

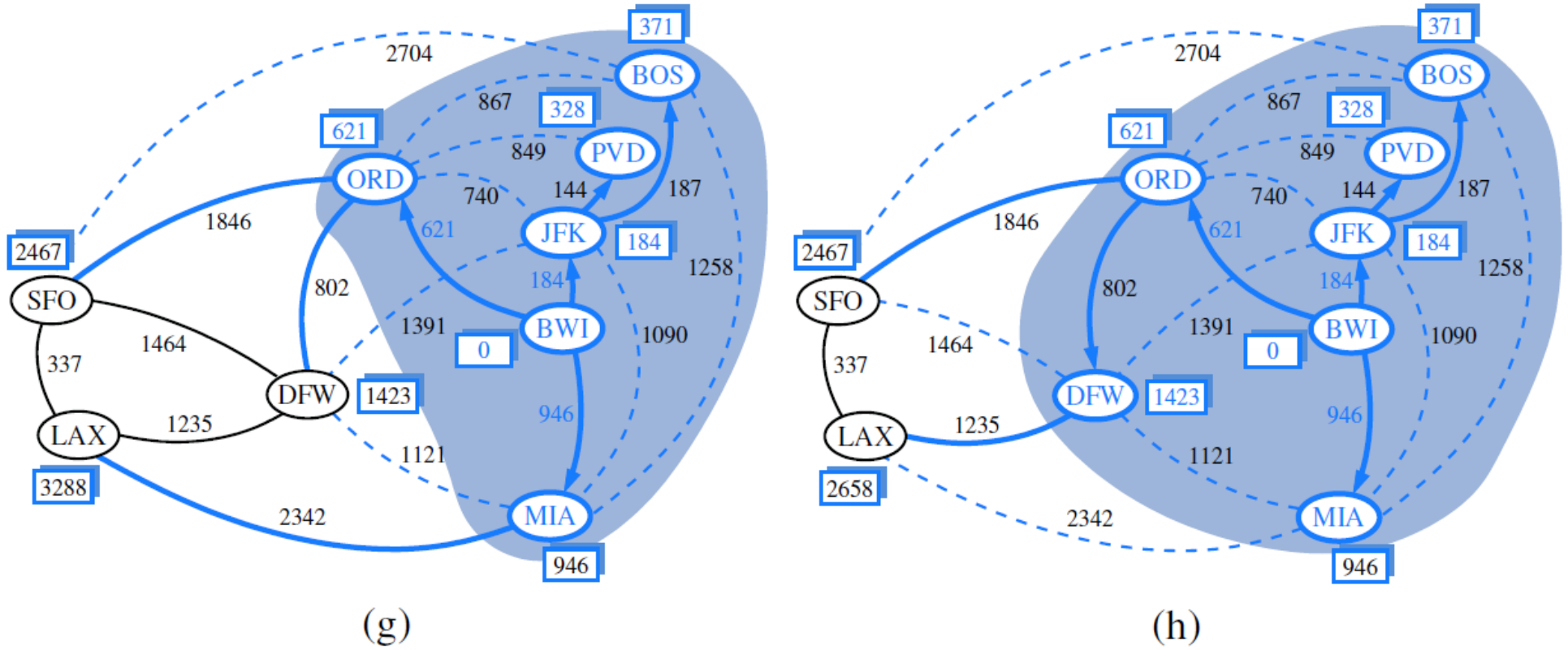


Figure 14.15: An example execution of Dijkstra's shortest-path algorithm on a weighted graph. The start vertex is BWI. A box next to each vertex v stores the label $D[v]$. The edges of the shortest-path tree are drawn as thick arrows, and for each vertex u outside the "cloud" we show the current best edge for pulling in u with a thick line. (Continues in Figure 14.16.)

Dijkstra's Algorithm

- **Example**

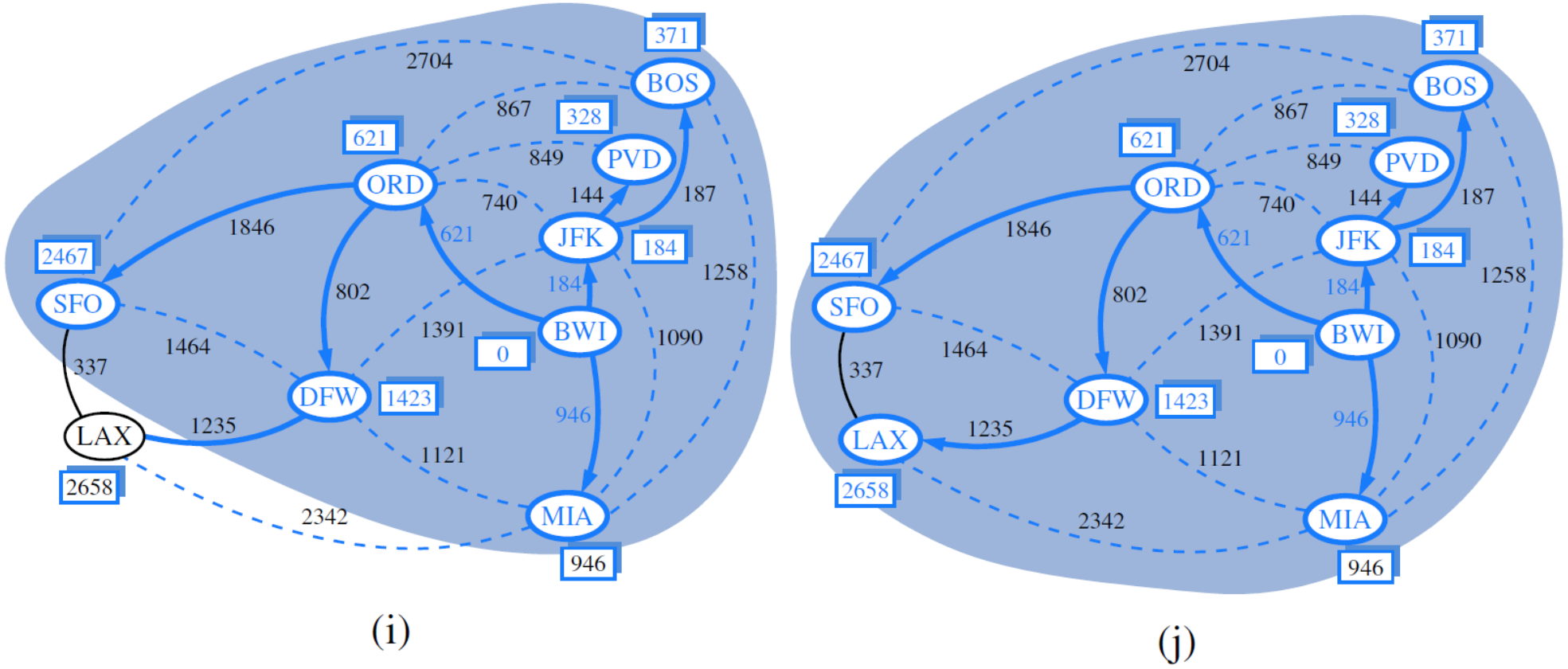


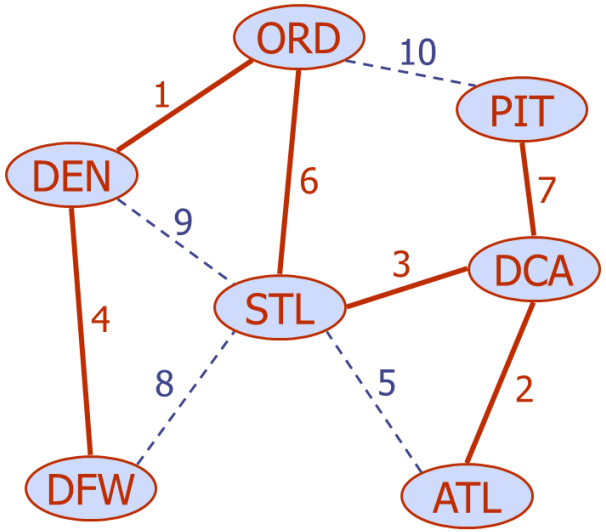
Figure 14.15: An example execution of Dijkstra's shortest-path algorithm on a weighted graph. The start vertex is BWI. A box next to each vertex v stores the label $D[v]$. The edges of the shortest-path tree are drawn as thick arrows, and for each vertex u outside the "cloud" we show the current best edge for pulling in u with a thick line. (Continues in Figure 14.16.)

Minimum Spanning Trees

- **Minimum Spanning Tree**
  - A tree T that contains all the vertices of a weighted graph G and has the minimum total weight over all such trees

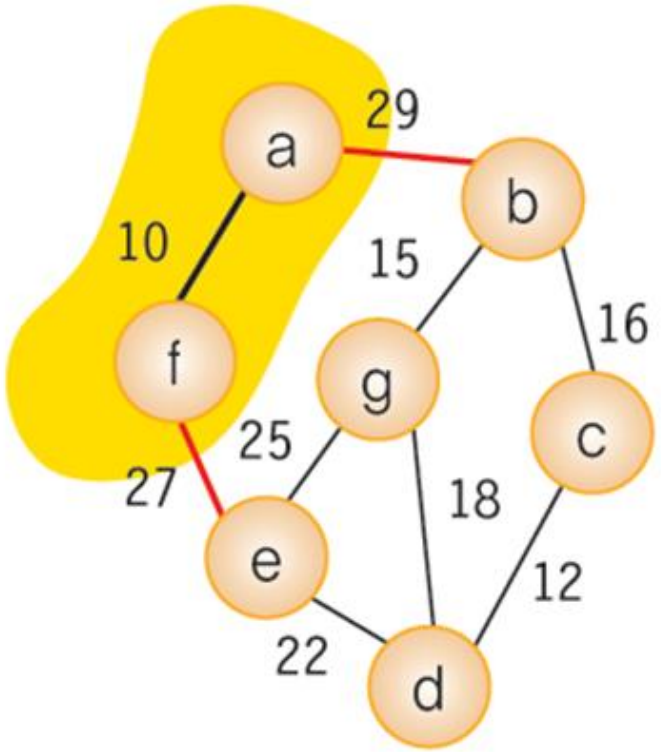
$$w(T) = \sum_{(v,u)\in T} w\big((v,u)\big)$$

  - Computing a spanning tree T with smallest total weight is the *minimum spanning tree* (*MST*) problem



Prim's Algorithm

- **Prim's Algorithm (a.k.a Prim-Jarnik Algorithm)**
  - A MST grows from a single cluster starting from some root vertex s
  - Chooses a minimum weight edge connecting u in the cloud C to v outside of C in each iteration



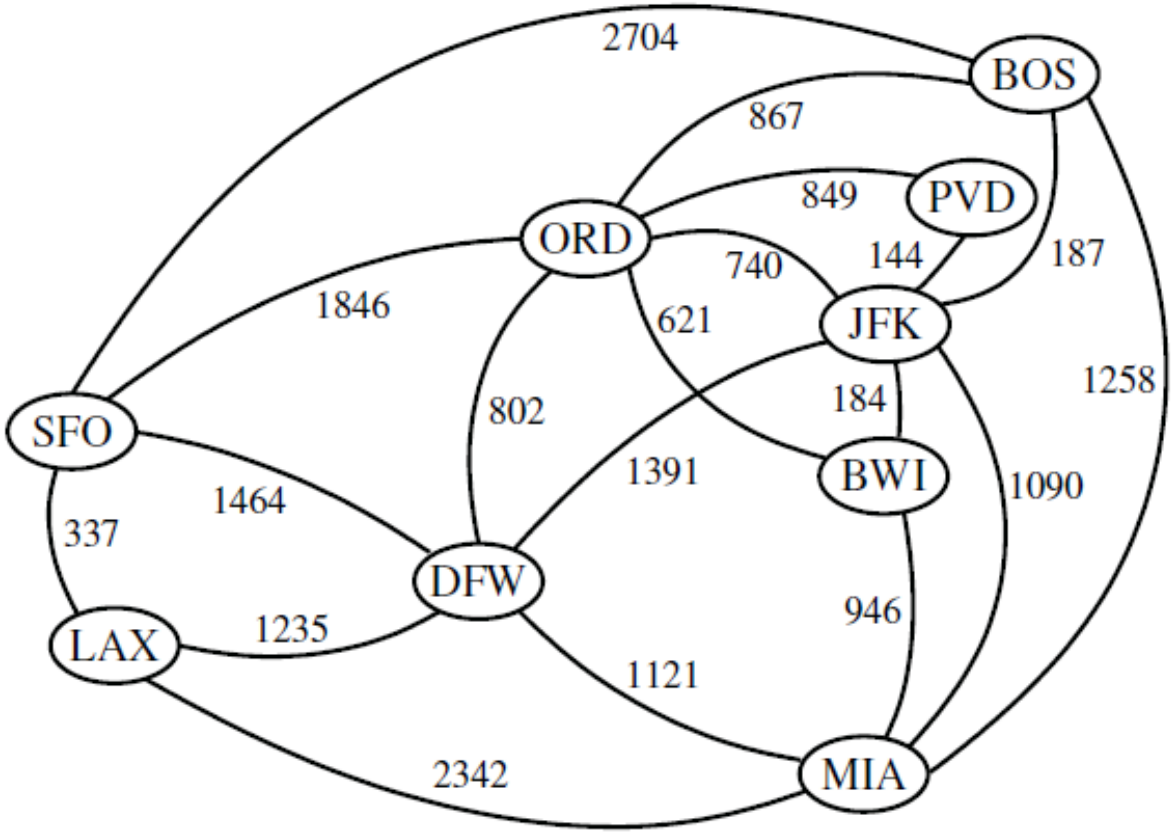
Algorithm PrimJarnik(G):
Input: An undirected, weighted, connected graph G with n vertices and m edges
Output: A minimum spanning tree T for G

Pick any vertex s of G
$D[s] = 0$
for each vertex $v \neq s$ **do**
  $D[v] = \infty$
Initialize $T = \emptyset$.
Initialize a priority queue Q with an entry $(D[v], v)$ for each vertex v.
For each vertex v, maintain connect(v) as the edge achieving $D[v]$ (if any).
while Q is not empty **do**
  Let u be the value of the entry returned by Q.removeMin( ).
  Connect vertex u to T using edge connect(e).
  for each edge $e' = (u, v)$ such that v is in Q **do**
    {check if edge (u, v) better connects v to T}
    if $w(u, v) < D[v]$ **then**
      $D[v] = w(u, v)$
      connect(v) $= e'$.
      Change the key of vertex v in Q to $D[v]$.
return the tree T

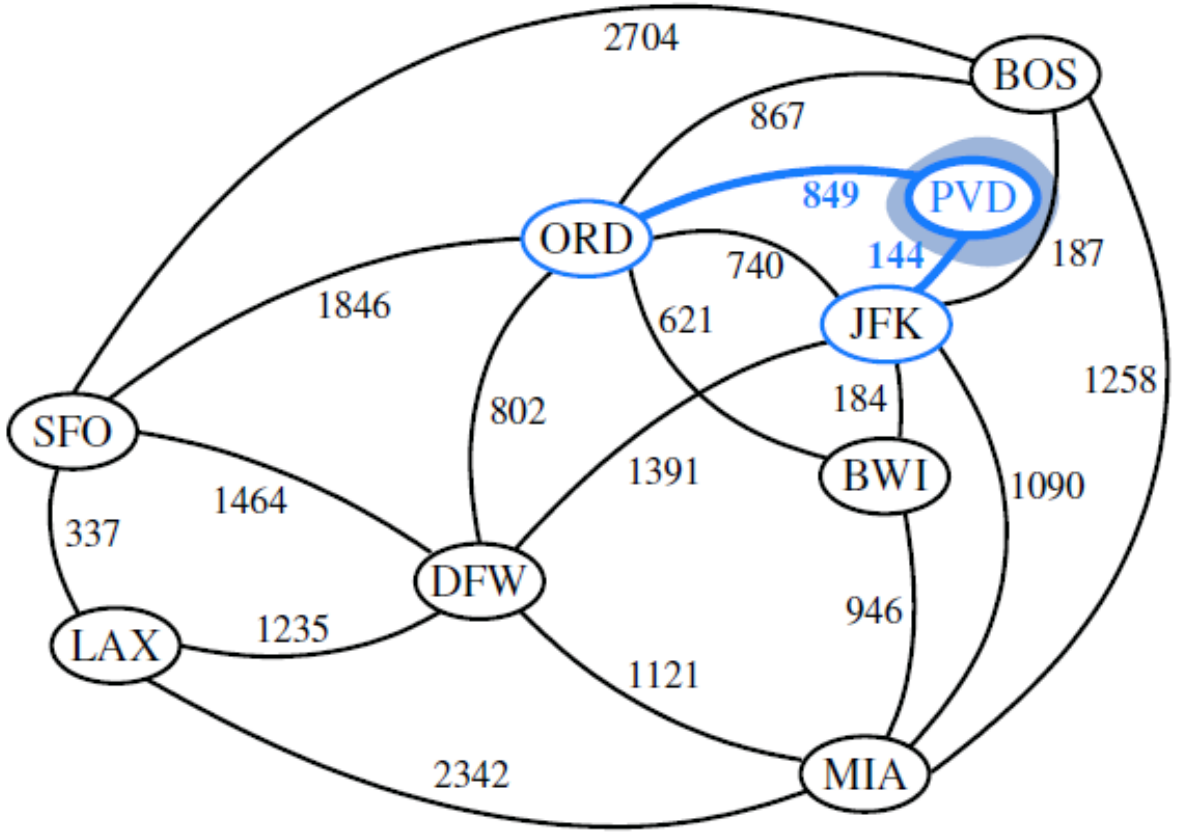
Code Fragment 14.15: The Prim-Jarník algorithm for the MST problem.

Prim's Algorithm

- **Example**



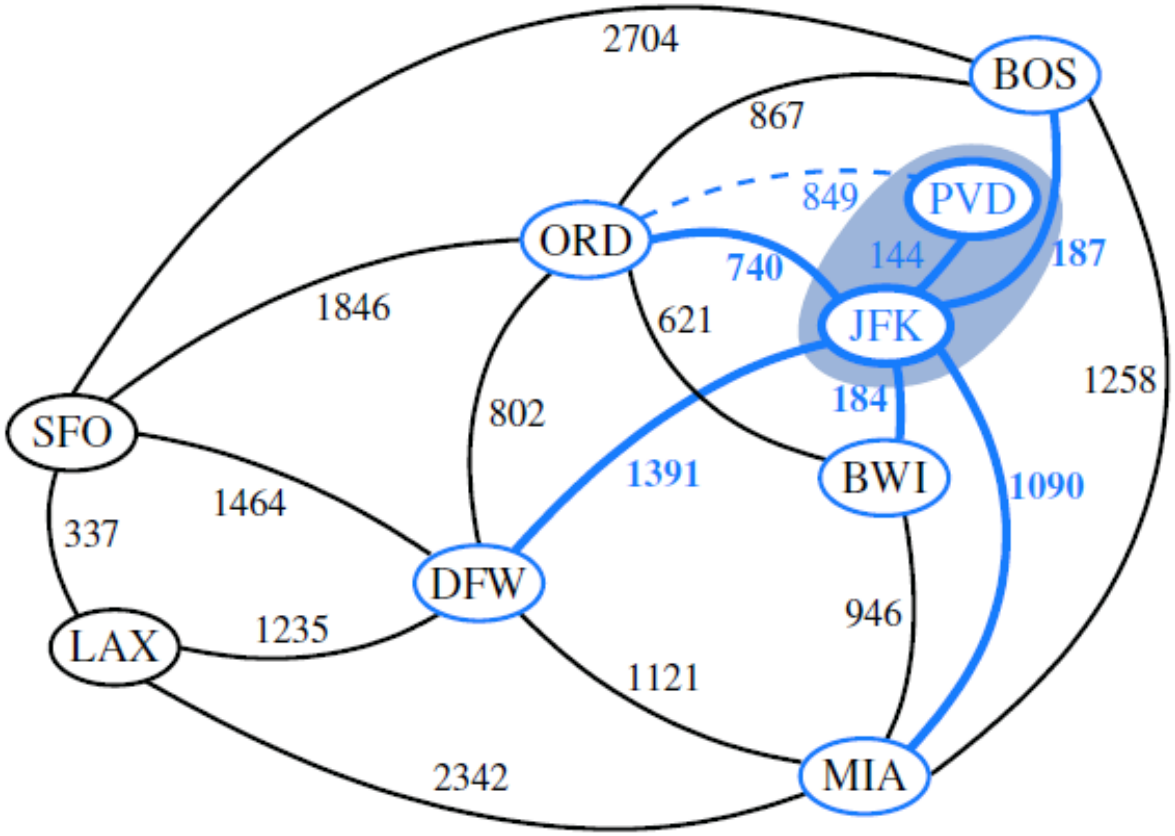
(a)



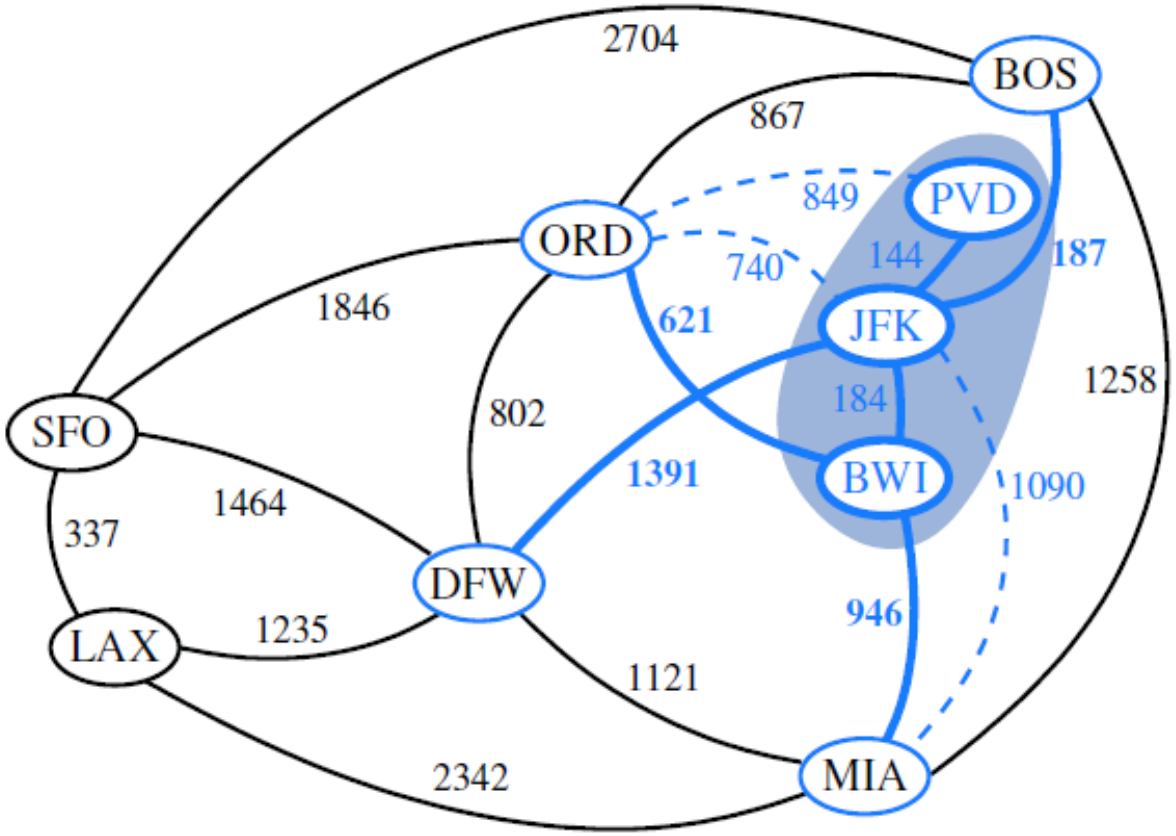
(b)

Prim's Algorithm

- **Example**



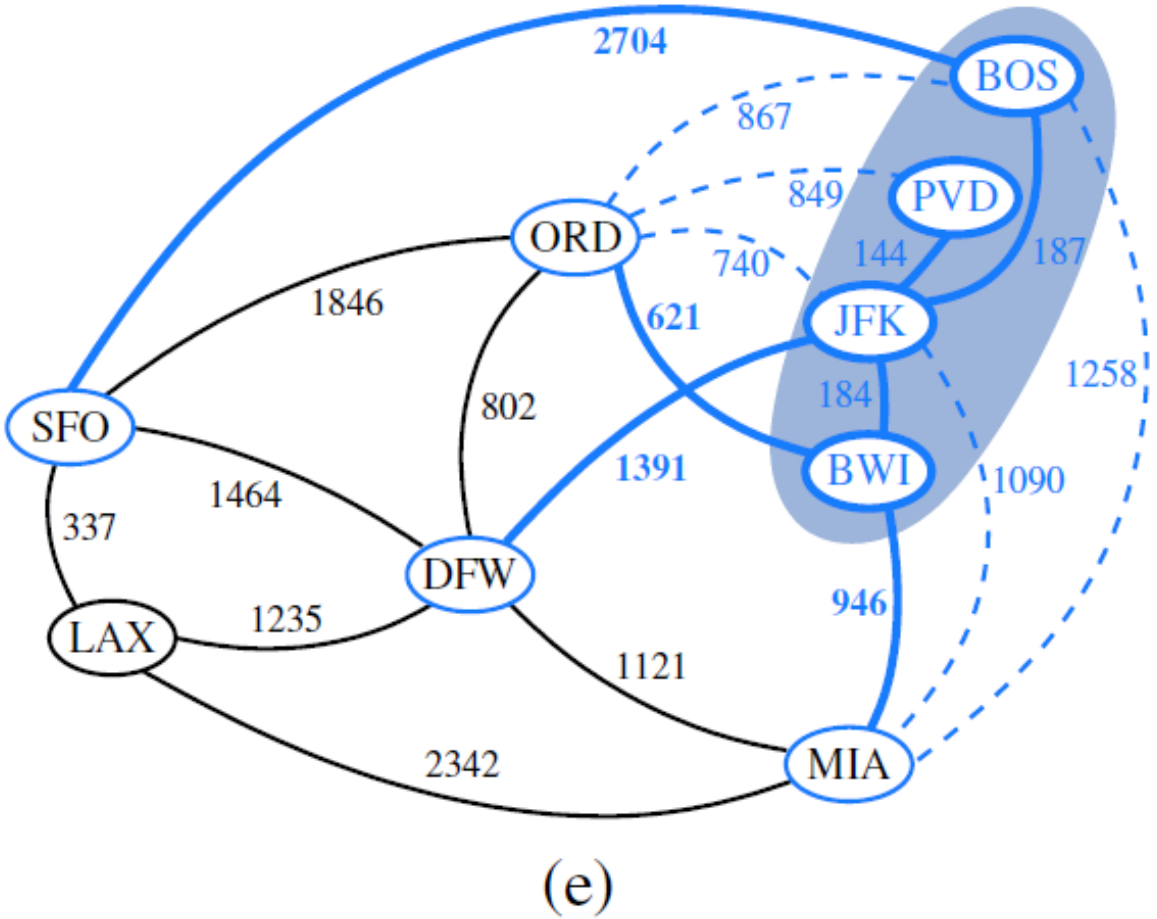
(c)



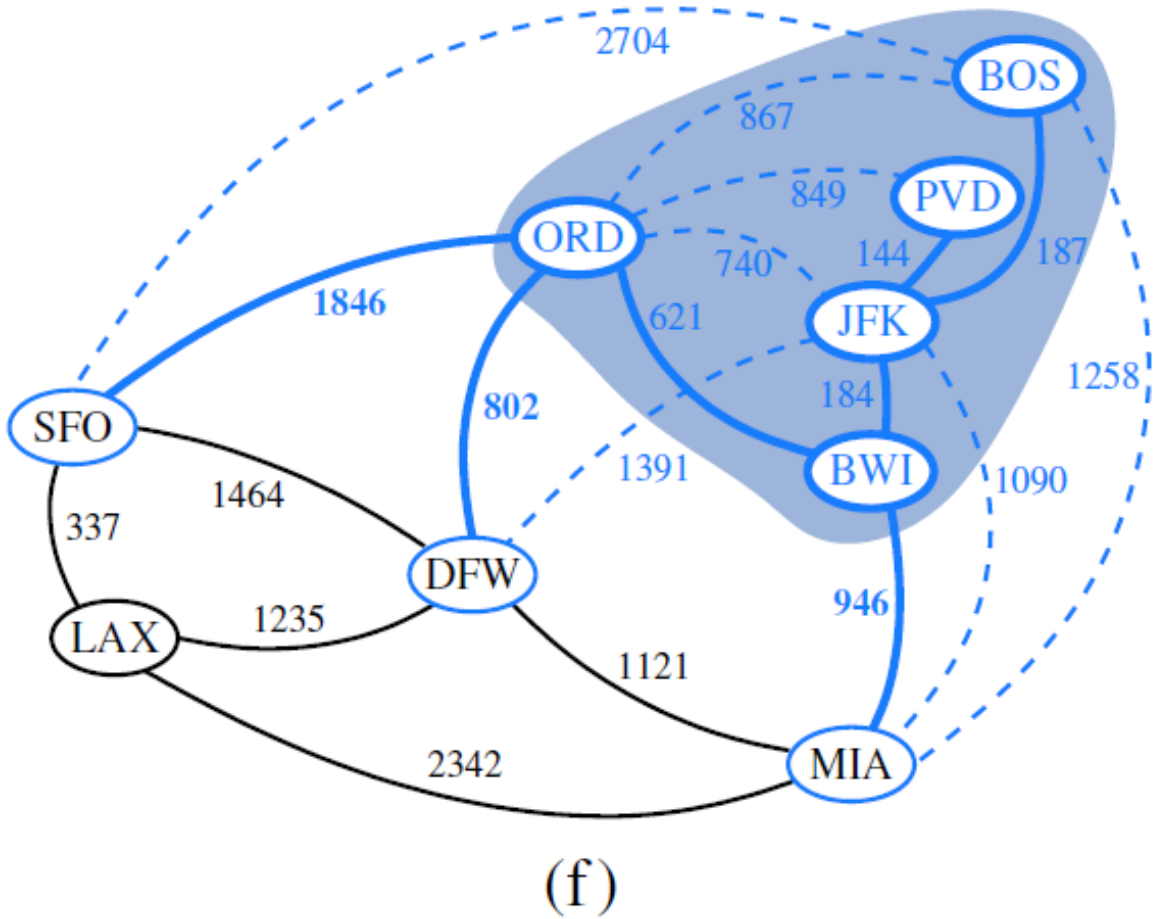
(d)

Prim's Algorithm

- **Example**



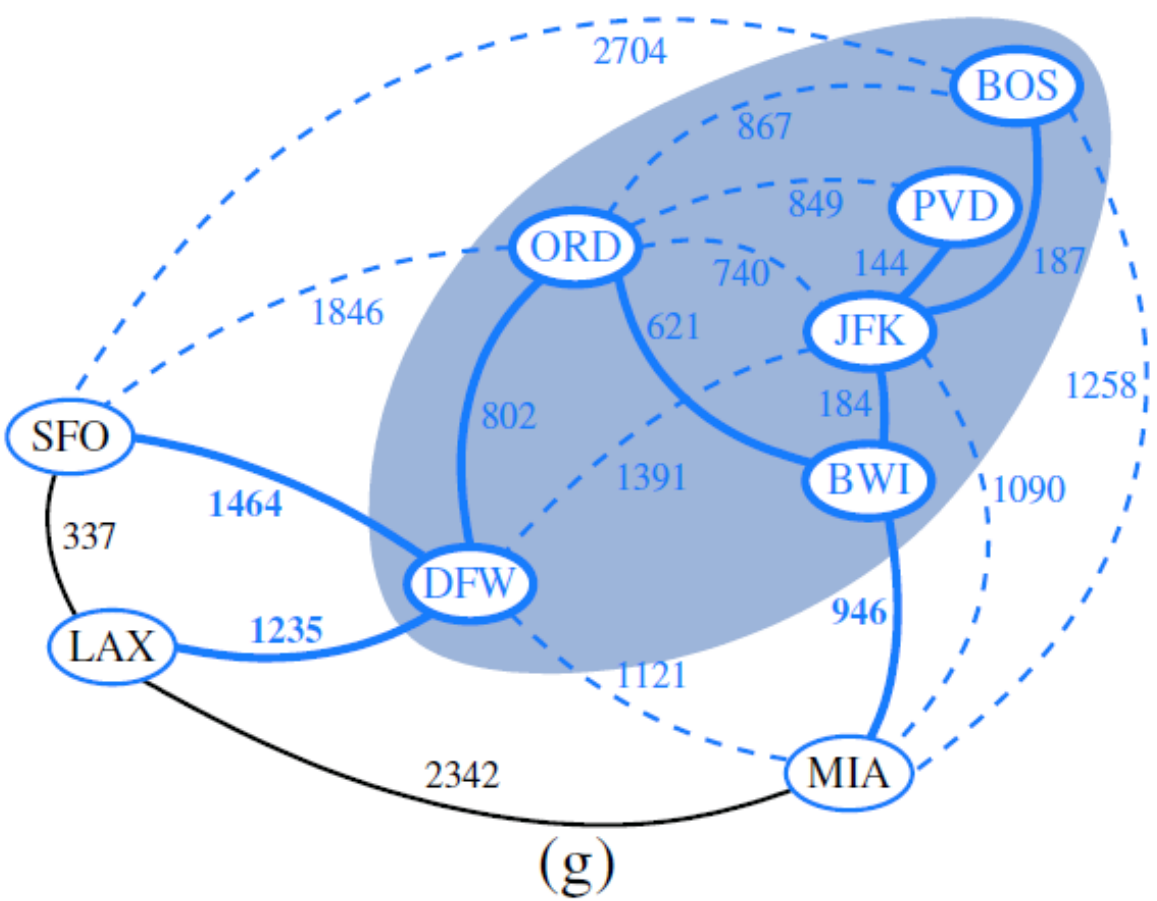
(e)



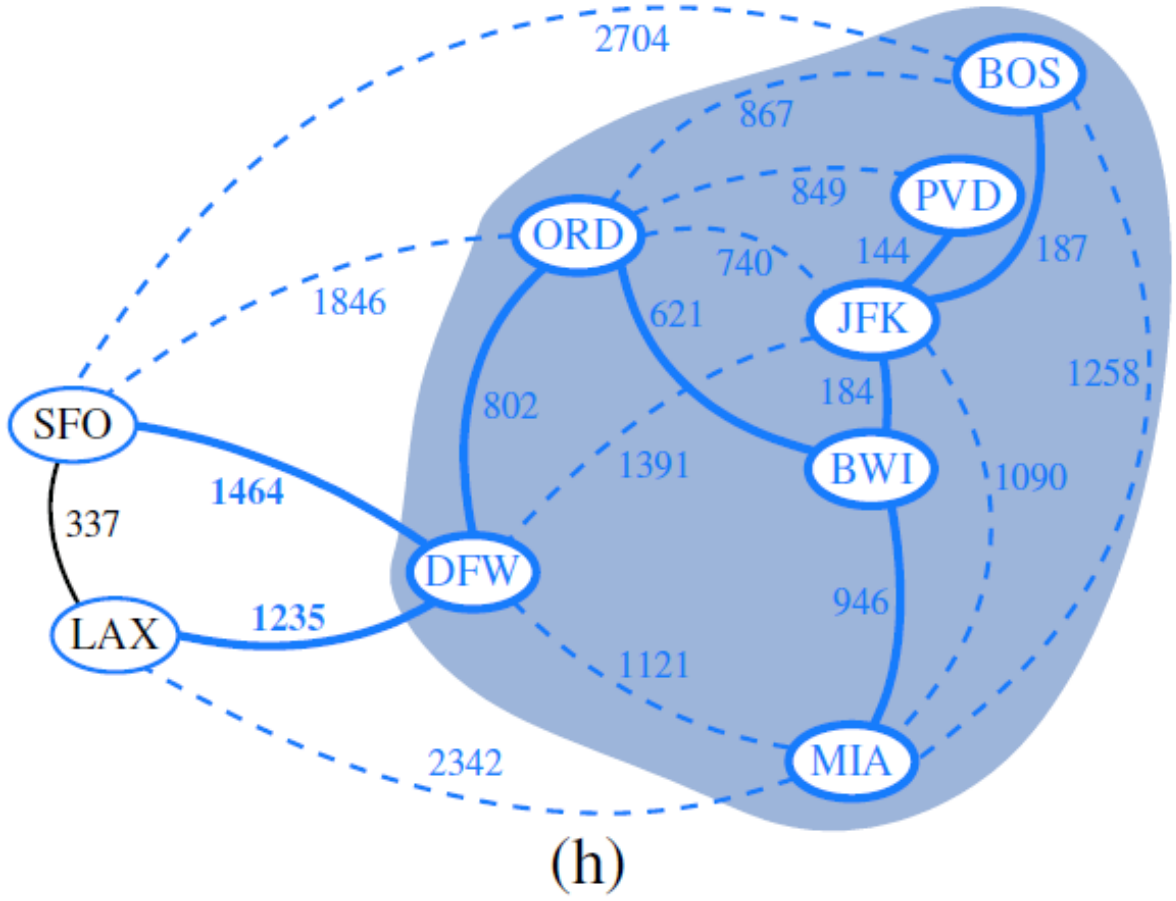
(f)

Prim's Algorithm

- **Example**



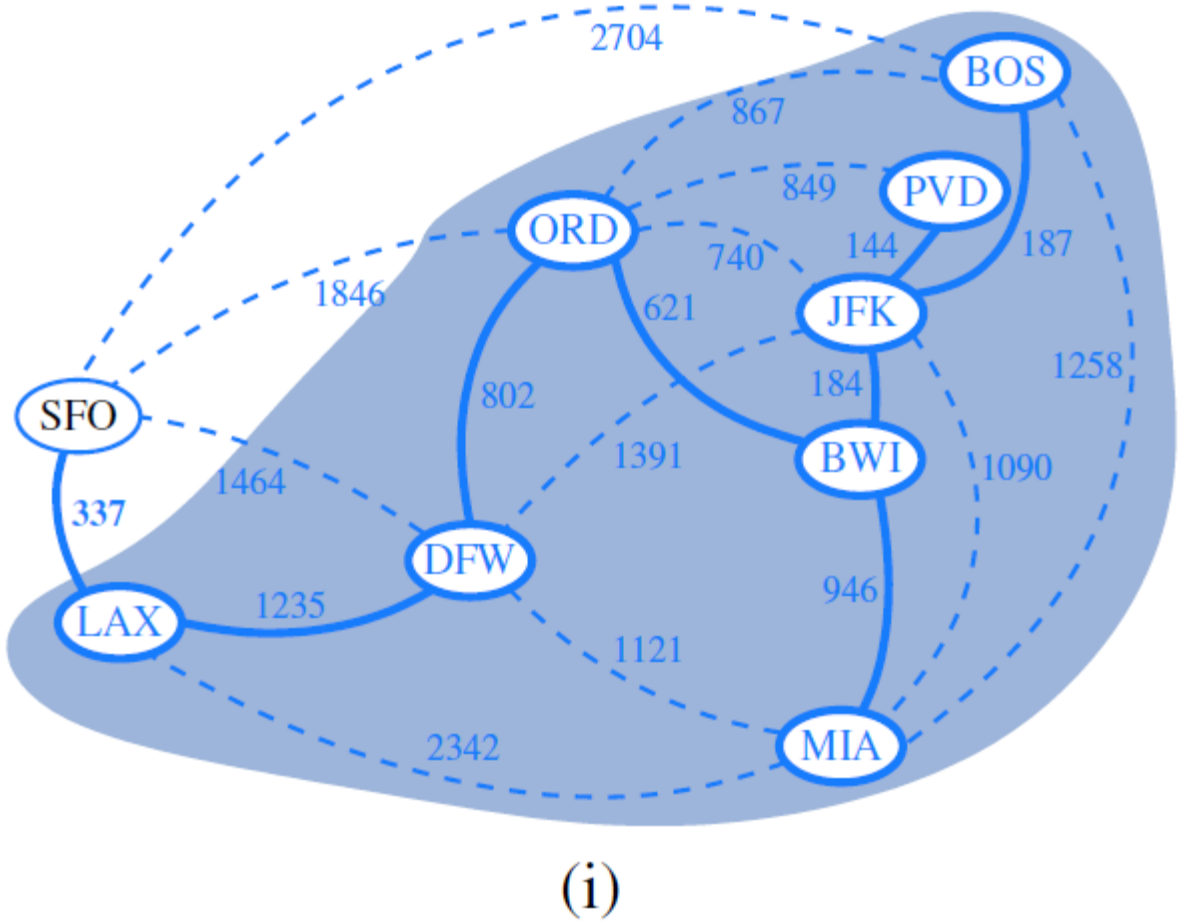
(g)



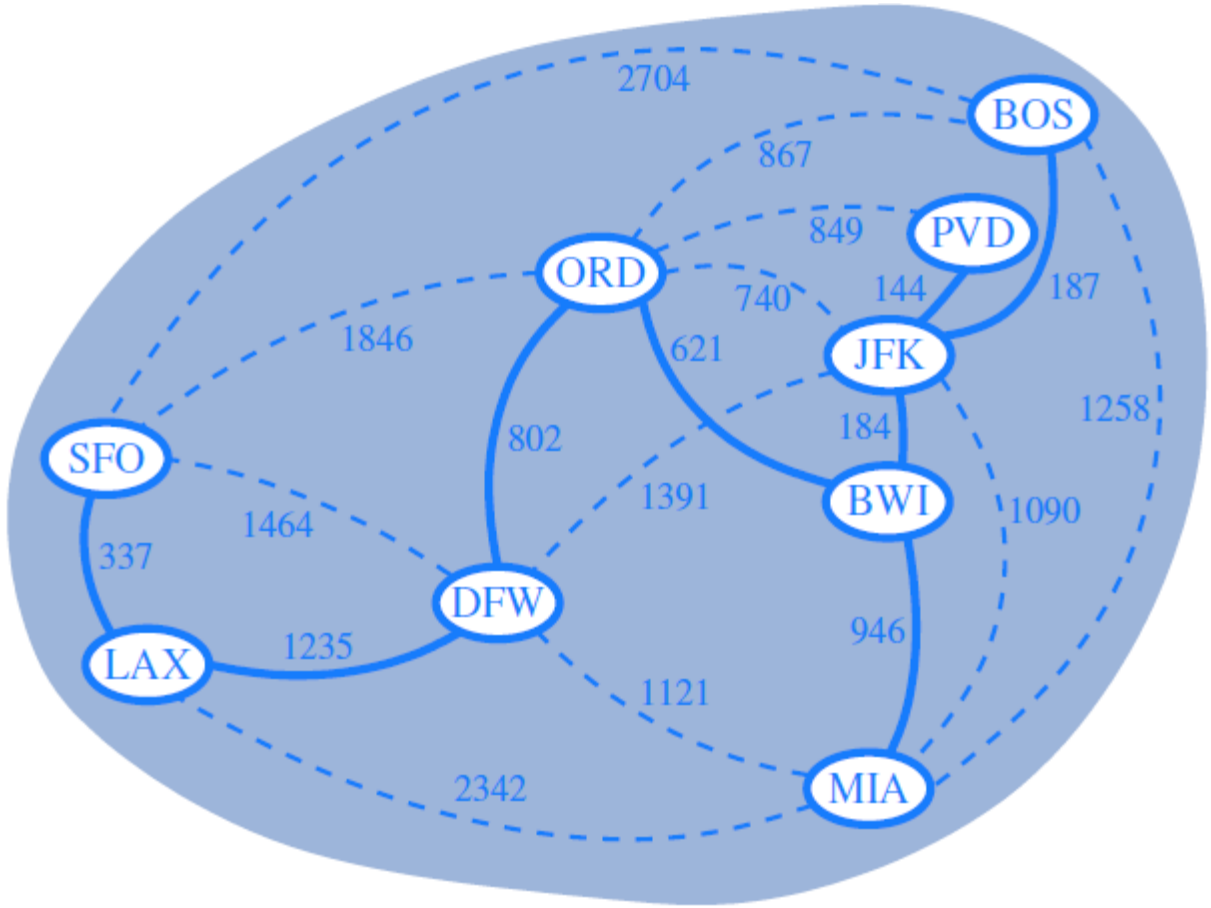
(h)

Prim's Algorithm

- **Example**



(i)



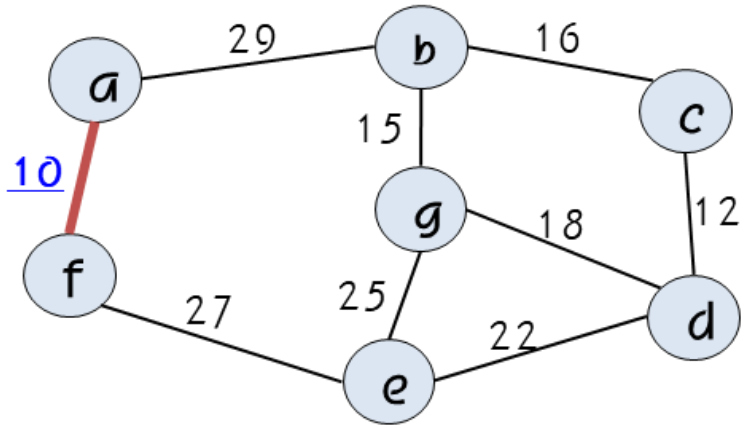
(j)

Kruskal's Algorithm

- **Kruskal's Algorithm**
  - Grows a single tree until it spans the graph
  - Maintains many small trees in a forest, and merges pairs of trees until a single tree spans the graph
  - In each iteration, chooses a minimum weight edge connecting two distinct clusters

Algorithm Kruskal(G):
Input: A simple connected weighted graph G with n vertices and m edges
Output: A minimum spanning tree T for G

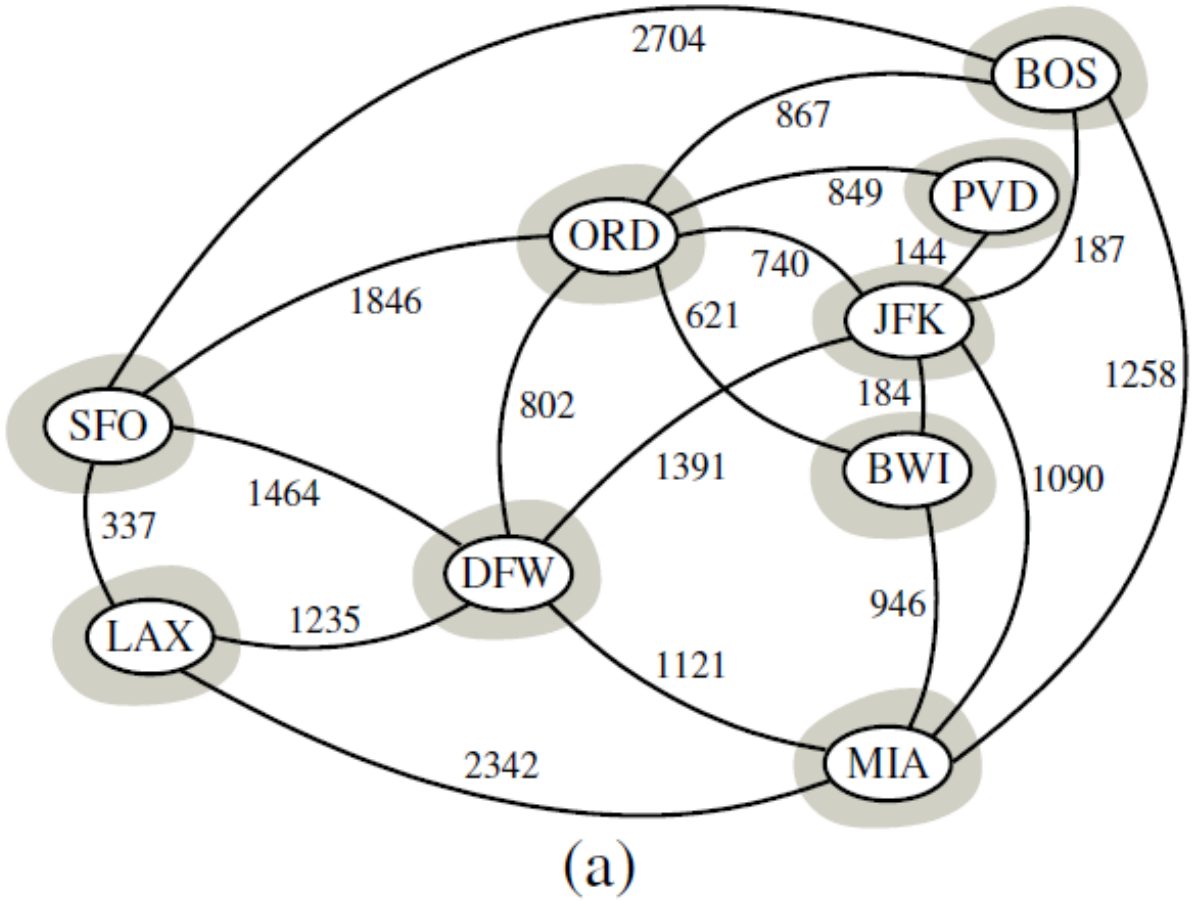
for each vertex v in G **do**
  Define an elementary cluster $C(v) = \{v\}$.
Initialize a priority queue Q to contain all edges in G, using the weights as keys.
$T = \emptyset$ {T will ultimately contain the edges of an MST}
while T has fewer than $n - 1$ edges **do**
  (u, v) = value returned by Q.removeMin()
  Let $C(u)$ be the cluster containing u, and let $C(v)$ be the cluster containing v.
  if $C(u) \neq C(v)$ **then**
    Add edge (u, v) to T.
    Merge $C(u)$ and $C(v)$ into one cluster.
return tree T



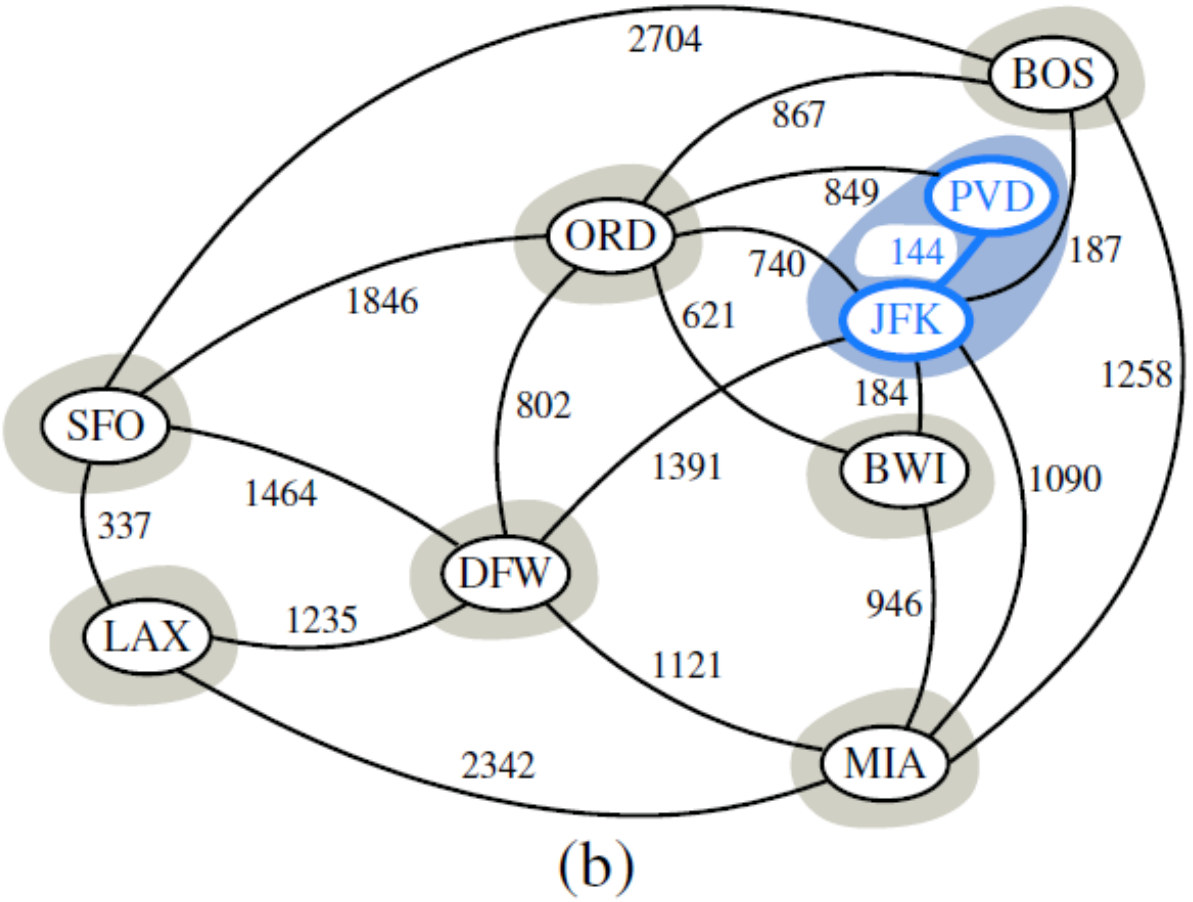
af	cd	bg	bc	dg	de	eg	ef	ab
10	12	15	16	18	22	25	27	29

Kruskal's Algorithm

- **Example**



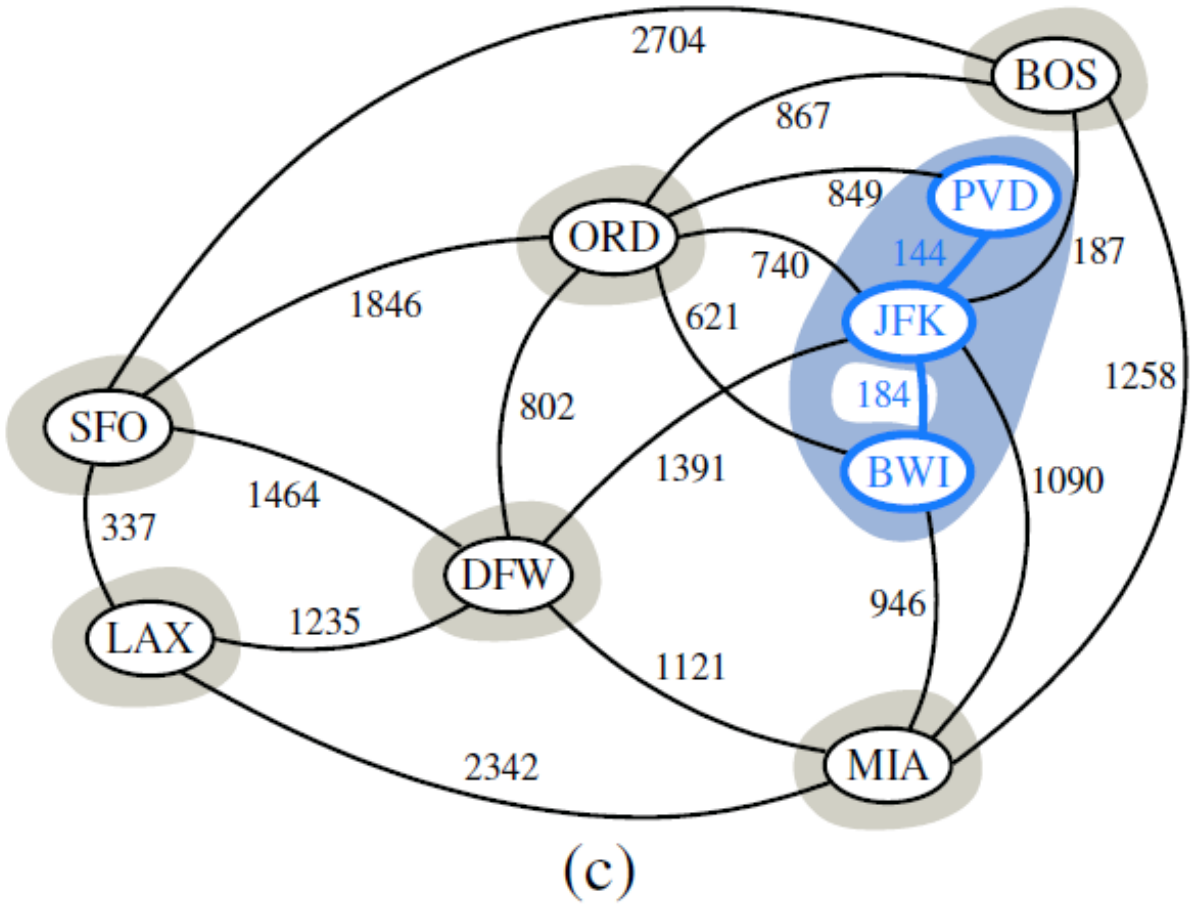
(a)



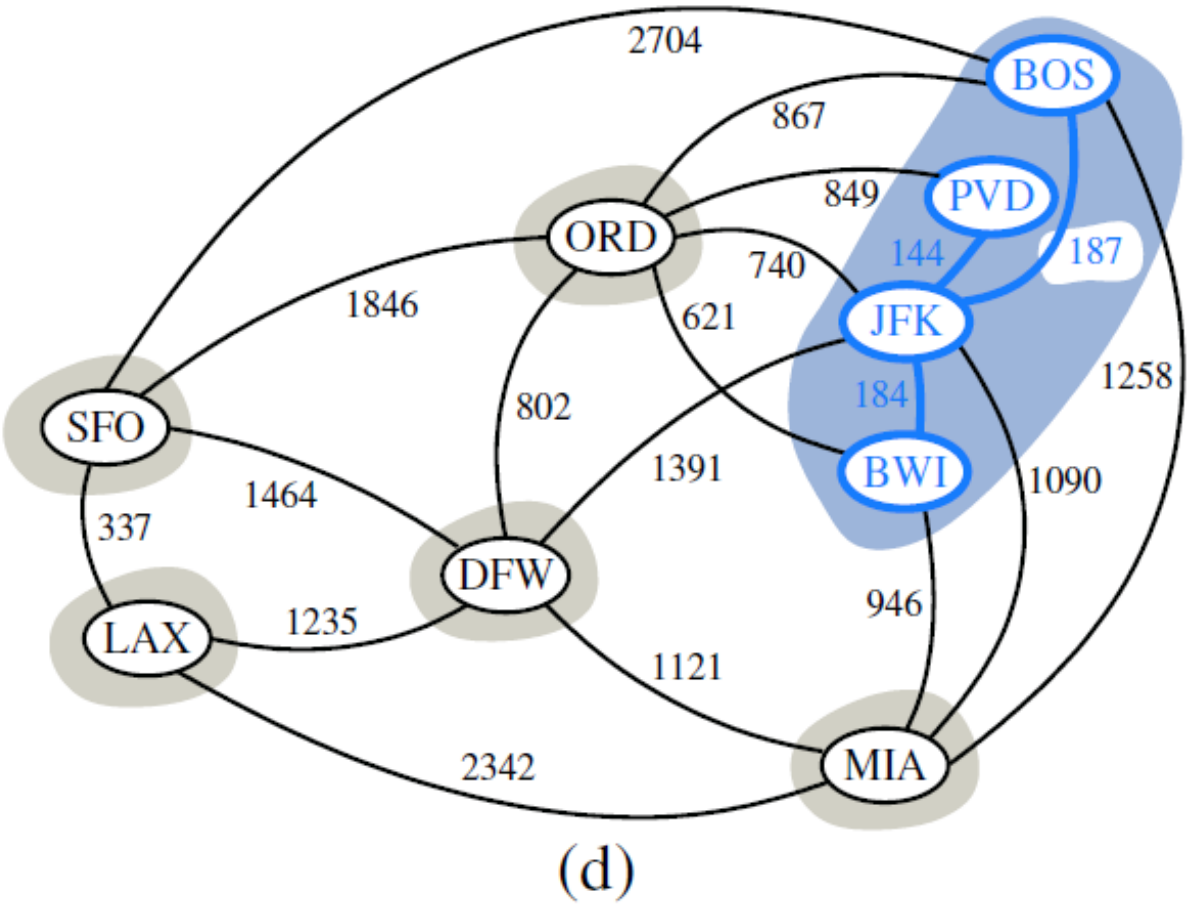
(b)

Kruskal's Algorithm

- **Example**



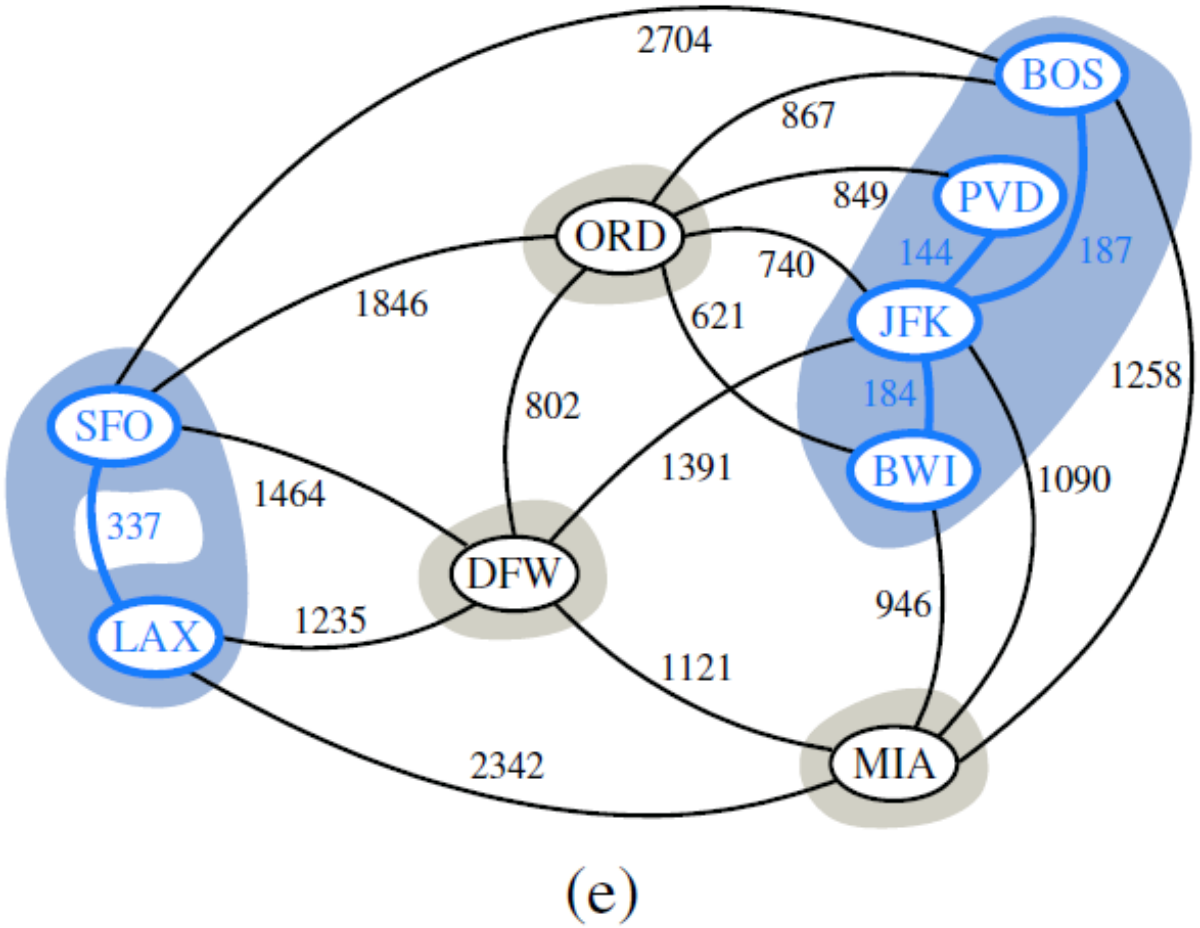
(c)



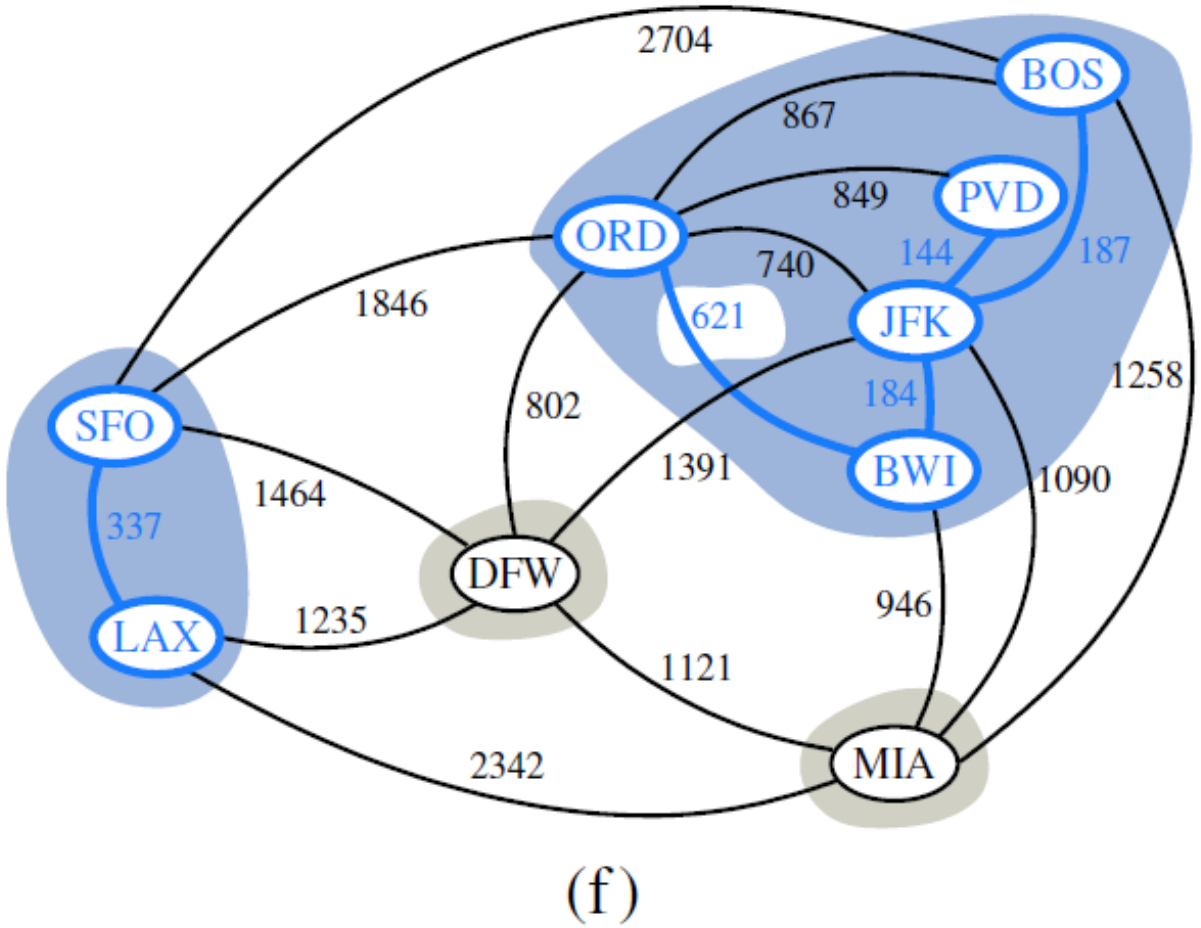
(d)

Kruskal's Algorithm

- **Example**



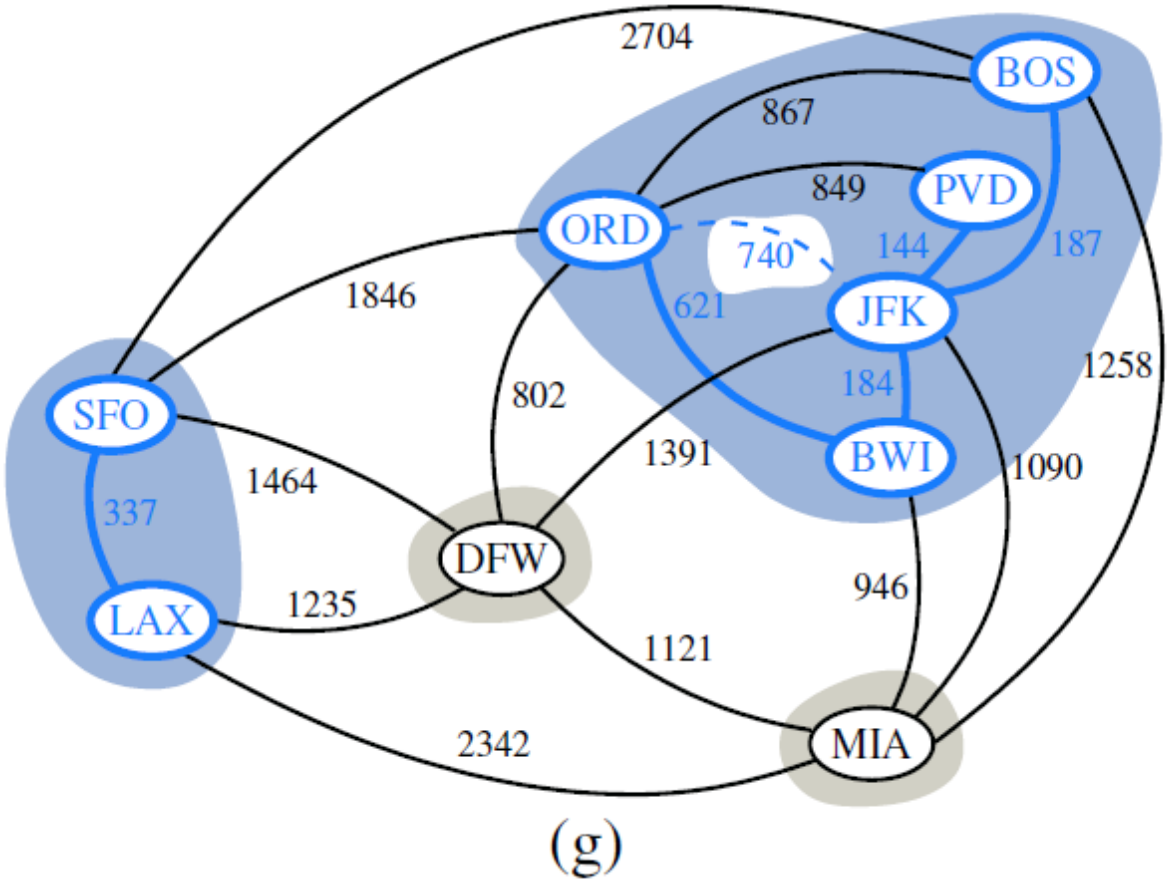
(e)



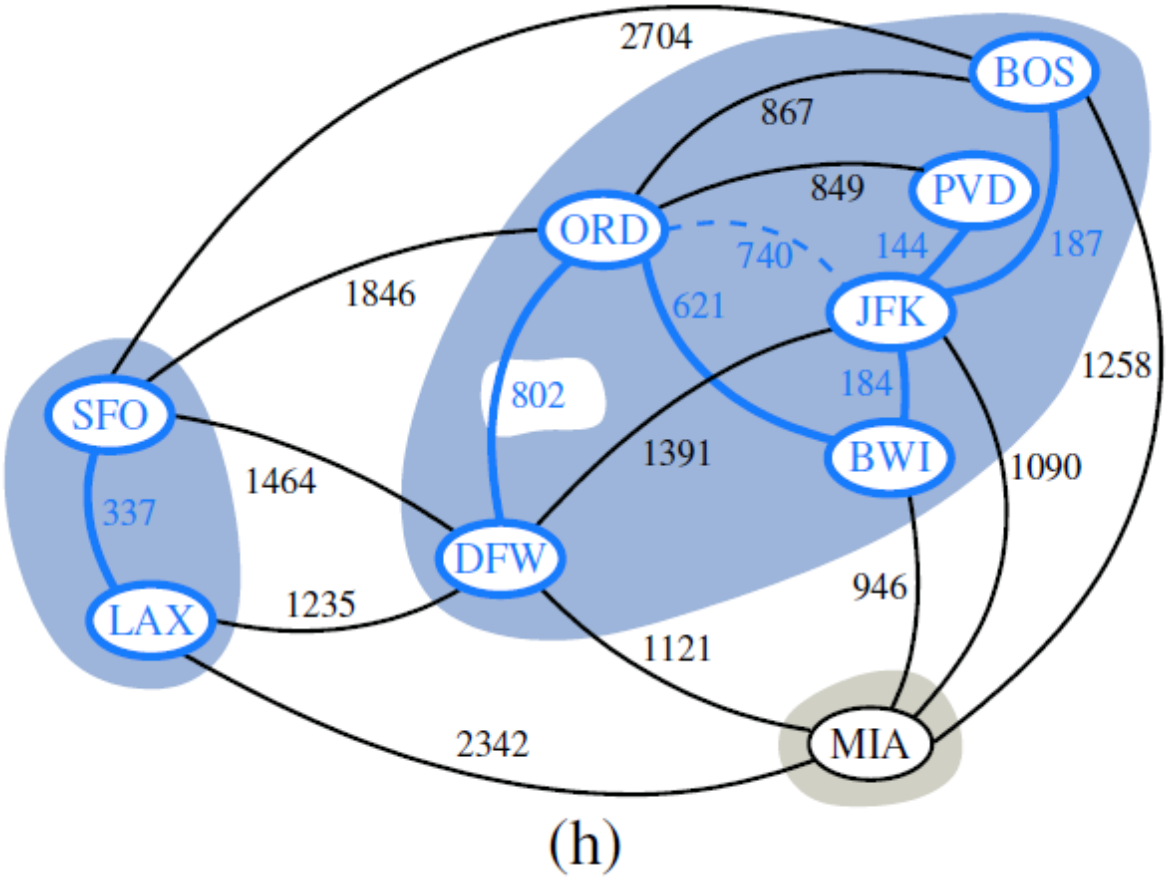
(f)

Kruskal's Algorithm

- **Example**



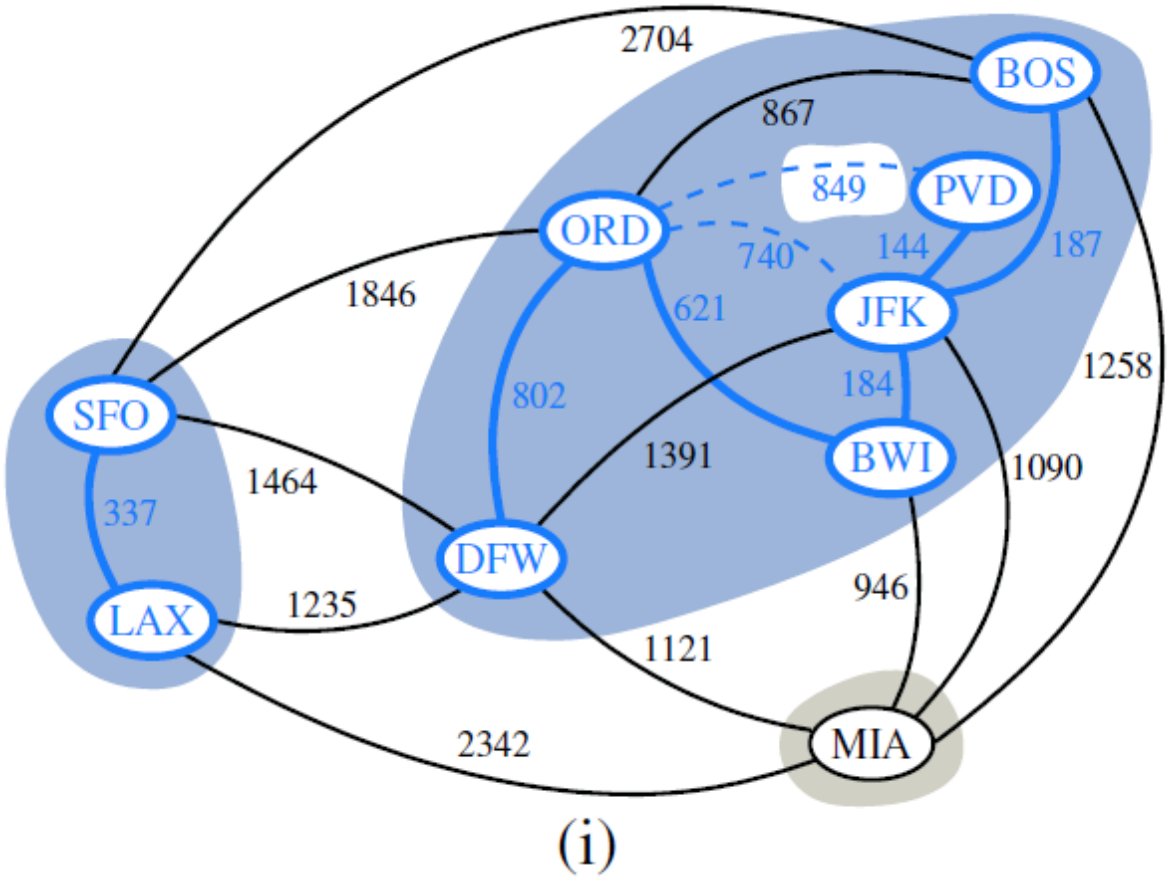
(g)



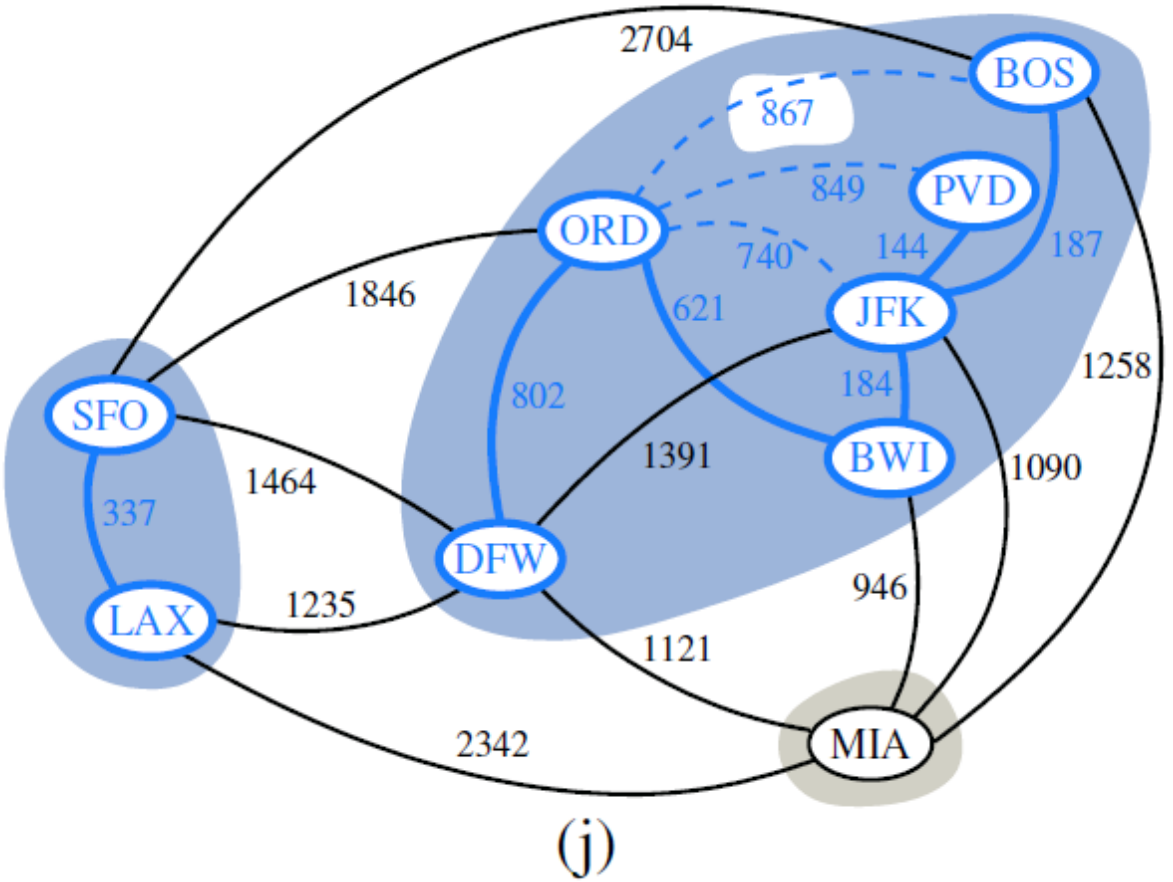
(h)

Kruskal's Algorithm

- **Example**



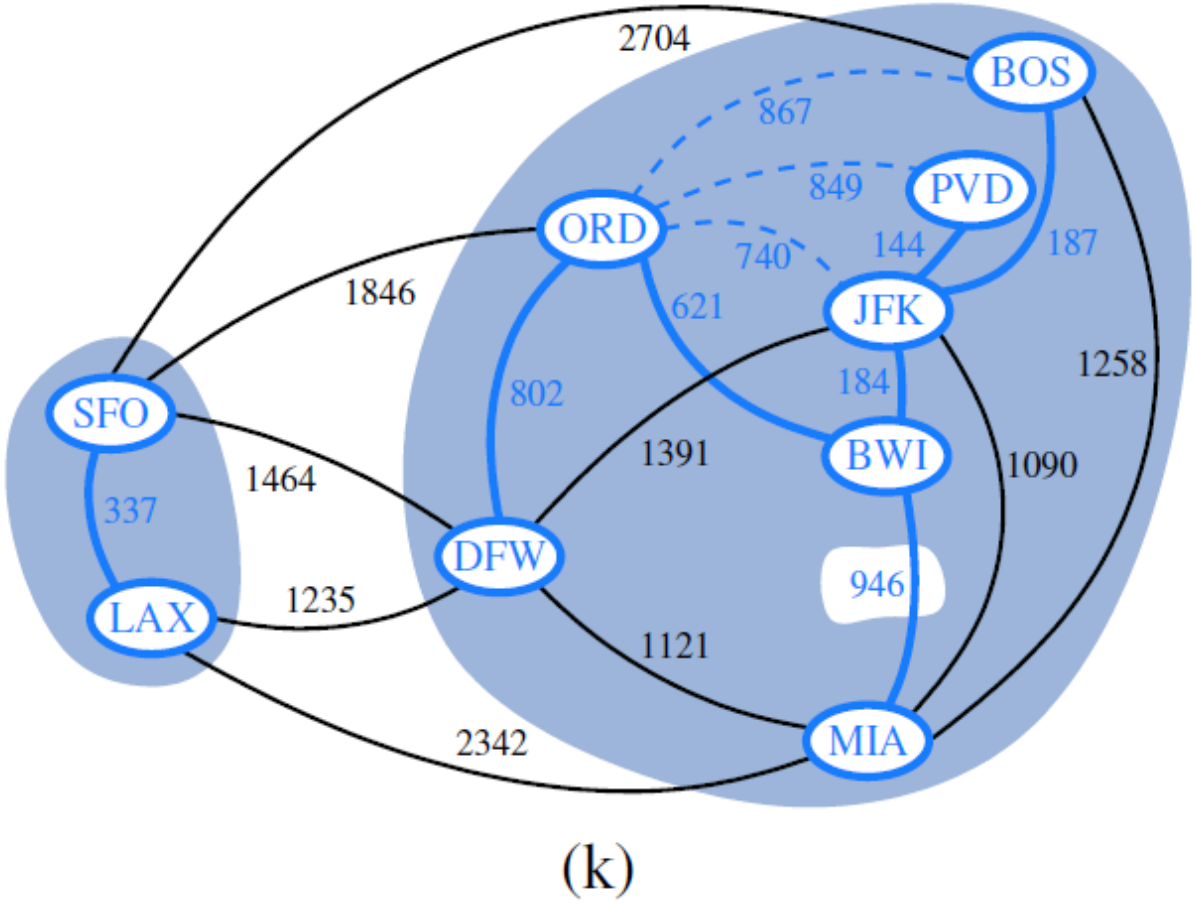
(i)



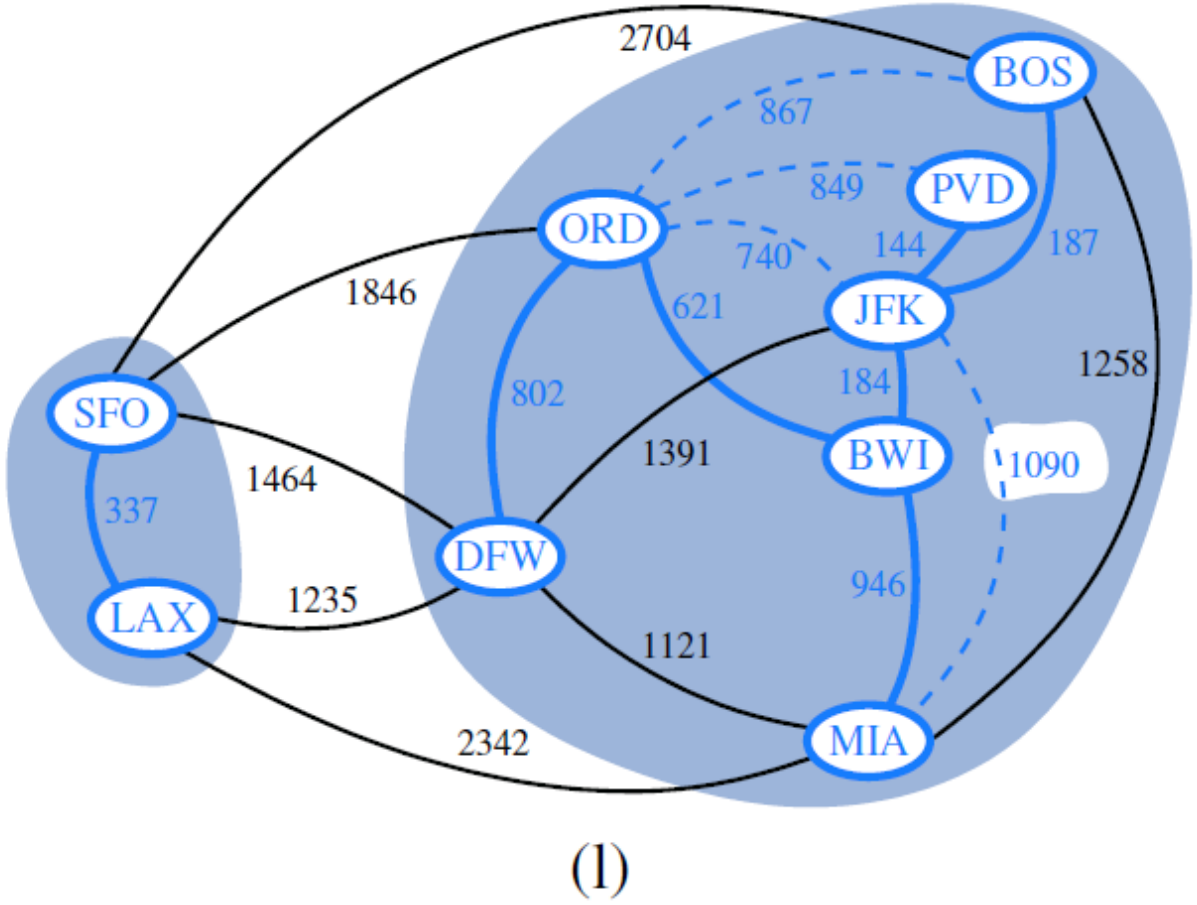
(j)

Kruskal's Algorithm

- **Example**



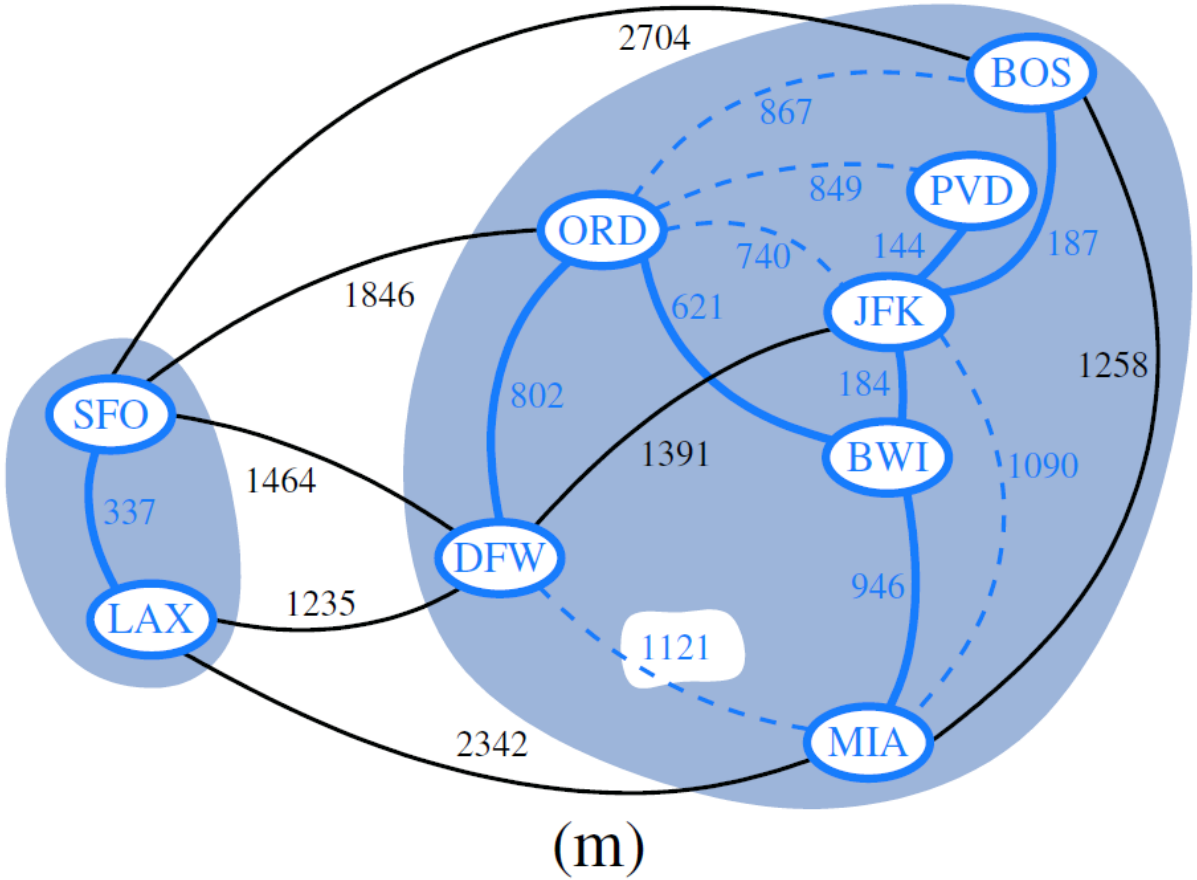
(k)



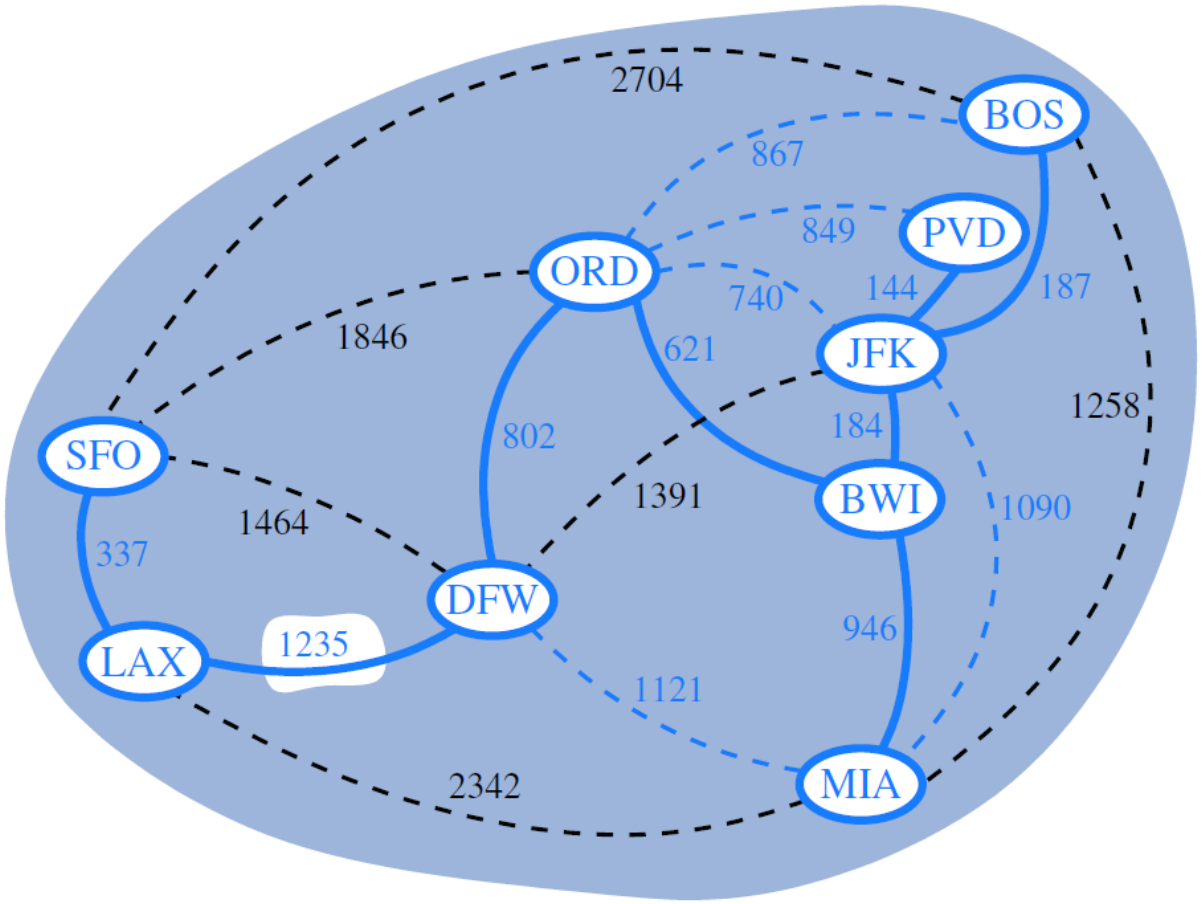
(l)

Kruskal's Algorithm

- **Example**



(m)



(n)